On Monte Carlo Tree Search with Multiple Objectives

Michael Painter

Pembroke College

University of Oxford

A thesis submitted for the degree of

Doctor of Philosophy

Trinity 2024

Acknowledgements

TODO: acknowledgements here

Abstract

TODO: abstract here

Contents

List of Figures

List of Tables

List of Code Listings

2.1.1 Definition 7
2.1.2 Definition 9
2.1.3 Definition 10
2.1.4 Definition 11
2.2.1 Definition 12
2.2.2 Definition 12
2.2.3 Definition 12
2.2.4 Definition 13
2.2.5 Definition 13
2.2.6 Definition 13
2.3.1 Definition 15
2.3.2 Definition 16
2.3.3 Definition 16
2.3.4 Definition 16
2.3.5 Definition 17
2.3.6 Definition 18
2.3.7 Definition 18
2.3.8 Definition 18
2.4.1 Definition 22
2.4.2 Definition 23
2.4.3 Definition 24
2.1 Pseudocode for running a trial in `THTS++`. 29
2.5.1 Definition 33
2.5.2 Definition 33
2.5.1 Theorem 33
2.6.1 Definition 34
2.6.2 Definition 34
2.6.3 Definition 35
2.6.4 Definition 36
2.6.5 Definition 36
2.6.6 Definition 36
2.6.7 Definition 37

2.6.8 Definition 37
2.6.9 Definition 37
2.6.10 Definition 38
2.6.11 Definition 39
2.6.12 Definition 40

4.1.1 Definition 56
4.1.2 Definition 57
4.1.3 Definition 58
4.2.1 Theorem 61
4.2.2 Theorem 61
4.2.3 Theorem 61
4.2.4 Theorem 61
4.2.5 Theorem 63
4.2.6 Theorem 64
4.2.7 Theorem 64
4.2.8 Theorem 64
4.3.1 Theorem 73
4.5.1 Definition 88
4.5.1 Lemma 88
4.5.1 Corollary 90
4.2.8 Theorem 91
4.2.5 Theorem 92
4.2.6 Theorem 92
4.5.2 Lemma 93
4.5.2 Corollary 94
4.5.3 Lemma 95
4.5.4 Lemma 96
4.5.5 Lemma 96
4.5.6 Lemma 97
4.5.7 Lemma 98
4.3.1 Theorem 99

List of Abbreviations

BTS . . . . . Boltzmann Tree Search.

CCS . . . . . Convex Coverage Set.

CHVI . . . . . Convex Hull Value Iteration.

CHVS . . . . . Convex Hull Value Set.

CMAB . . . . Contextual Multi-Armed Bandit (problem).

CZ . . . . . Contextual Zooming.

DENTS . . . . Decaying ENtropy Tree Search.

EMAB . . . . Exploring Multi-Armed Bandit (problem).

MAB . . . . . Multi-Armed Bandit (problem).

MCCS . . . . . Minimal Convex Coverage Set.

MCTS . . . . . Monte Carlo Tree Search.

MCTS-1 . . . . A specific and common presentation of Monte Carlo Tree Search, presented in Section **??**.

MDP . . . . . Markov Decision Process.

MENTS . . . . Maximum ENtropy Tree Search.

MOMDP . . . Multi-Objective Markov Decision Process.

MORL . . . . Multi-Objective Reinforcement Learning.

PF . . . . . Pareto Front.

PUCT . . . . . Predictor-UCT.

THTS . . . . . Trial-Based Heuristic Tree Search.

`THTS++` . . . . . An extension of THTS used in this thesis.

UCB . . . . . Upper Confidence Bound (algorithm).

UCT . . . . . Upper Confidence Bound Applied to Trees.

List of Notation

General Notation

$\mathbb{1}$ . . . . . The indicator function, where $\mathbb{1}(A) = 1$ when A is true, and $\mathbb{1}(A) = 0$ when A is false.

$\hat{A}$ . . . . . The hat notation is used to denote an estimate, i.e. $\hat{A}$ is an estimate for some value A or some optimal value A^*

$\bar{A}$ . . . . . The bar notation is used to denote sample averages, $\bar{A}$ is a sample average of a number of samples $A_1, ..., A_k$. i.e. $\bar{A} = \frac{1}{k}\sum_{i=1}^{k} A_i$.

$\tilde{A}$ . . . . . The tilde notation is used to denote a (function) approximation, such as a neural network.

$\mathcal{A}$ . . . . . Calligraphic font is used to denote variables that are sets or tuples (ordered sets).

$\mathbf{A}$ . . . . . Bold font is used to denote vectors, and A_i is used to denote the ith component of the vector $\mathbf{A}$.

$\texttt{A}$ . . . . . Typewriter text is used to refer to variables relating to an algorithm (or the analysis of an algorithm).

$\boldsymbol{\mathcal{A}}$ . . . . . The notations above will also be combined at times, so $\boldsymbol{\mathcal{A}}$ is used to denote a set that contains vectors.

$\mathbb{E}$ . . . . . The expectation operator. $\mathbb{E}[f]$ denotes the expectation of a distribution f, and $\mathbb{E}_{x\sim f}[g(x)]$ denotes the expected value of $g(x)$ under the distribution f.

$x \sim f$ . . . . . If $f : X \to \mathbb{R}$ specifies a probability distribution in some way (e.g. a probability density function or cumulative density function), then $x \sim f$ denotes that the variable x is sampled from the distribution f.

$O(f(x))$ . . . . Big-O notation: $g(x) = O(f(x))$ if there exist constants $c > 0$ and x_0 such that $|g(x)| \leq c \cdot f(x)$ for all $x \geq x_0$. Used to describe an asymptotic upper bound on the growth rate of a function.

$X_i \xrightarrow{p} X$ . . . . The sequence of random variables X_i converges in probability to the random variable X. $\Pr(|X_i - X| > \varepsilon) \to 0$ as $i \to \infty$ for all $\varepsilon > 0$.

$X_i \xrightarrow{as} X$ . . . . The sequence of random variables X_i converges almost surely to the random variable X. $\Pr(\lim_{i\to\infty} X_i = X) = 1$.

(Multi-Objective) Markov Decision Processes (Defined in Sections ?? and ??)

$\mathcal{A}$ . . . . . A (finite) set of actions.

a_t . . . . . The action at the tth timestep of a trajectory.

D . . . . . The dimension of rewards in a MOMDP.

H . . . . . The finite-horizon time bound of an MDP.

$\mathcal{M}$ . . . . . A Markov Decision Process, which is a tuple $\mathcal{M} = (\mathcal{S}, s_0, \mathcal{A}, R, p, H)$.

$\boldsymbol{\mathcal{M}}$ . . . . . A Multi-Objective Markov Decision Process, which is a tuple $\boldsymbol{\mathcal{M}} = (\mathcal{S}, s_0, \mathcal{A}, \mathbf{R}, p, H)$.

p . . . . . The next-state transition distribution of an MDP. $p(s'|s,a)$ is the probability of transitioning to state s' from state s by taking action a.

π . . . . . A policy, mapping a state $s \in \mathcal{S}$ to a probability distribution over actions $\mathcal{A}$. That is, $\pi(a|s)$ is the probability of taking action a at state s. $\pi(s) = a'$ is used a shorthand for $\pi(a|s) = \mathbb{1}[a = a']$.

R . . . . . The reward function of an MDP: $R(s,a)$ is the reward received from taking action a at state s.

$\mathbf{R}$ . . . . . The D dimensional reward function of a MOMDP: $\mathbf{R}(s,a) \in \mathbb{R}^D$ is the vector of rewards received from taking action a at state s.

r_t . . . . . The reward at the tth timestep of a trajectory.

$\mathcal{S}$ . . . . . A (finite) set of states.

$\mathcal{S}_0$ . . . . . The set of reachable states from the starting state s_0 in an MDP.

$\mathrm{Succ}(s,a)$ . . . . The set of successor states of a state-action pair (s,a), with respect to an MDP: $\mathrm{Succ}(s,a) = \{s' \in \mathcal{S} | p(s'|s,a) > 0\}$.

s_0 . . . . . $s_0 \in \mathcal{S}$ is the initial starting state of an MDP.

s_t . . . . . The state at the tth timestep of a trajectory.

`terminal` . . . . `terminal` $\subseteq \mathcal{S}$ is the set of terminal states. A state is terminal if the transition distribution and reward function of the MDP are such that once reached, the state will never be left and no more reward will be received.

τ . . . . . A trajectory, or sequence, of states, actions and rewards that are sampled according to a policy π and an MDP $\mathcal{M}$: $\tau = (s_0, a_0, r_0, s_1, ..., s_{H-1}, a_{H-1}, r_{H-1}, s_H)$.

τ . . . . . A multi-objective trajectory, of states, actions and rewards that are sampled according to a policy π and an MDP $\mathcal{M}$: $\tau = (s_0, a_0, \mathbf{r}_0, s_1, ..., s_{H-1}, a_{H-1}, \mathbf{r}_{H-1}, s_H)$.

$\tau_{i:j}$ . . . . . A truncated trajectory, starting at timestep i, and ending at timestep j: $\tau_{i:j} = (s_i, a_i, r_i, s_{i+1}, ..., s_{j-1}, a_{j-1}, r_{j-1}, s_j)$.

$\tau_{i:j}$ . . . . . A multi-objective truncated trajectory, starting at timestep i and ending at timestep j: $\tau_{i:j} = (s_i, a_i, \mathbf{r}_i, s_{i+1}, ..., s_{j-1}, a_{j-1}, \mathbf{r}_{j-1}, s_j)$.

Reinforcement Learning (Section ??)

$J(\pi)$ . . . . . The objective function for (standard) reinforcement learning: $J(\pi) = V^\pi(s_0; 0)$.

π^* . . . . . The optimal standard policy, that maximises the objective function $J(\pi)$.

Q^* . . . . . The optimal Q-value function. $Q^*(s, a; t)$ denotes the maximal expected value that can be achieved by any policy, starting from state $s_t = s$, with action $a_t = a$.

Q^π . . . . . The Q-value function of a policy π. $Q^\pi(s, a; t)$ denotes the expected cumulative reward that policy π will obtain, starting from state $s_t = s$, starting by taking action $a_t = a$.

V^* . . . . . The optimal value function. $V^*(s; t)$ denotes the maximal expected value that can be achieved by any policy, starting from state $s_t = s$.

V^π . . . . . The value of a policy π. $V^\pi(s; t)$ denotes the expected cumulative reward that policy π will obtain, starting from state $s_t = s$.

Maximum Entropy Reinforcement Learning (Section ??)

α . . . . . The temperature parameter, or, the coefficient of the entropy term in the maximum entropy (soft) objective.

$\mathcal{H}$ . . . . . The Shannon entropy, of a probability distribution or policy.

$J_{\text{sft}}(\pi)$ . . . . . The objective function for maximum entropy (soft) reinforcement learning: $J_{\text{sft}}(\pi) = V^\pi_{\text{sft}}(s_0; 0)$.

π^*_{sft} . . . . . The optimal soft policy, that maximises the soft objective function $J_{\text{sft}}(\pi)$.

Q^*_{sft} . . . . . The optimal soft Q-value function. $Q^*_{\text{sft}}(s, a; t)$ denotes the maximal expected soft value that can be achieved by any policy, from state $s_t = s$ and taking action $a_t = a$.

Q^π_{sft} . . . . . The soft Q-value function of a policy π. $Q^\pi_{\text{sft}}(s, a; t)$ is the expected value of the policy from $s_t = s$, starting with action $a_t = a$, with an addition of the entropy of the policy weighted by the temperature α.

V^*_{sft} . . . . . The optimal soft value function. $V^*_{\text{sft}}(s; t)$ denotes the maximal expected soft value that can be achieved by any policy, from state $s_t = s$.

V^π_{sft} . . . . . The soft value of a policy π. $V^\pi_{\text{sft}}(s; t)$ is the expected value of the policy from $s_t = s$, with an addition of the entropy of the policy weighted by the temperature α.

Trial Based Heuristic Tree Search (Section ??)

`backup_v` . . . . Updates the values at a decision node in the backup phase of a trial `THTS++`, using the decision node's children, the trajectory sampled for the trial and the heuristic value function.

`backup_q` . . . . Updates the values at a chance node in the backup phase of a trial `THTS++`, using the chance node's children, the trajectory sampled for the trial and the heuristic value function.

$H_{\texttt{THTS++}}$ . . . . . The planning horizon used in `THTS++`, with $H_{\texttt{THTS++}} \leq H$.

$N(s)$ . . . . . The number of visits at the decision node corresponding to state s.

$N(s, a)$ . . . . . The number of visits at the chance node corresponding to state-action pair (s, a).

`node`(s) . . . . . The decision node corresponding to the state s.

`node`(s)`.chldrn` The set of chance nodes that are children of `node`(s).

`node`(s)`.data` . The set of variables stored at decision node `node`(s), typically used for estimating values.

`node`(s, a) . . . The chance node corresponding to the state-action pair (s, a).

`node`(s, a)`.chldrn` The set of decision nodes that are children of `node`(s, a).

`node`(s, a)`.data` The set of variables stored at chance node `node`(s, a), typically used for estimating Q-values.

$\pi_{\text{heuristic}}$ . . . . . A heuristic policy, which can be used by the search policy to utilise prior knowledge of the environment.

π_{search} . . . . . The search policy used in `THTS++` to sample a trajectory in the selection phase.

$\hat{Q}_{\text{heuristic}}$ . . . . The heuristic action function used in `THTS++`, used to provide a Q-value estimate for any state-action pairs that aren't in the search tree.

`sample_context` A function used in `THTS++` that creates a context, or key-value store, and samples any initial values to be stored in the context.

`sample_outcome` A function used in `THTS++` to sample outcomes (successor states) from the environment (MDP).

`select_action` A function used in `THTS++` to select an action at a decision node.

`single_node_expansion` Specifies if `THTS++` samples a trajectory such that only one decision node is added to the search tree per trial. If not running with `single_node_expansion` set, `THTS++` samples a trajectory until a terminal state or the planning horizon $H_{\texttt{THTS++}}$ is reached.

$\mathcal{T}$ . . . . . The `THTS++` search tree. $\mathcal{T} \subseteq \mathcal{S} \cup \mathcal{S} \times \mathcal{A}$. Initially it consists of the root node, and new nodes are added from the trajectories/trials sampled in the selection phase.

`transposition_table` Specifies if `THTS++` uses a transposition table to store and re-use the decision nodes of the search tree when they are reached again via another path.

$\hat{V}_{\text{heuristic}}$ . . . . . The heuristic value function used in `THTS++`, used to initialise the value of a new decision node.

Upper Confidence Bound applied to Trees (Section ??)

b_{UCT} . . . . . The bias parameter used in UCT to control exploration.

`normalised_Q` . Specifies if the Q-values are linearly normalised (between 0 and 1) by the UCT search policy to select actions.

π_{rollout} . . . . . A policy used to sample a trajectory that is used to estimate the value of a state being added to a search tree.

π_{UCT} . . . . . The search policy used in UCT.

$\bar{Q}_{\text{UCT}}(s, a)$ . . . The sample-average value estimate used to estimate the Q-value at a chance node (s, a) in UCT.

$\bar{Q}^{\text{norm}}_{\text{UCT}}(s, a)$ . . . The optionally linearly normalised (between 0 and 1) Q-value. Used by the UCT search policy to select actions. If `normalised_Q` is False, then $\bar{Q}^{\text{norm}}_{\text{UCT}}(s, a) = \bar{Q}_{\text{UCT}}(s, a)$.

$\hat{V}^{\pi_{\text{rollout}}}(s)$ . . . An estimate of $V^{\pi_{\text{rollout}}}(s)$ computed using the return from a single trajectory sampled using policy π_{rollout}. It is used to initialise the value of a state being added to the search tree.

$\bar{V}_{\text{UCT}}(s)$ . . . . . The sample-average value estimate used to estimate the value at a decision node s in UCT.

Maximum ENtropy Tree Search (Section ??)

α_{MENTS} . . . . . The temperature parameter used in MENTS, which is the coefficient of the entropy terms in the maximum entropy objective.

ϵ_{MENTS} . . . . . An exploration parameter used in MENTS.

$\lambda(s, \epsilon)$ . . . . . A decaying function used to control and anneal exploration in MENTS. $\lambda(s, \epsilon) = \min\left(1, \frac{\epsilon}{\log(e+N(s))}\right)$.

$\tilde{\pi}$ . . . . . A policy neural network, used to initialise soft Q-value estimates in MENTS, if available.

π_{MENTS} . . . . . The search policy used in MENTS.

$\hat{Q}_{\text{MENTS}}(s, a)$ . . A soft Q-value estimate at the chance node (s, a) in MENTS.

$\tilde{V}$ . . . . . A function approximation to V^*.

$\hat{V}_{\text{MENTS}}(s)$ . . . A soft value estimate at the decision node s in MENTS.

Multi-Objective Reinforcement Learning (Section ??)

$CCS(\Pi)$ . . . . A convex coverage set of policies, which contains at least one policy that maximises the linear utility $u_{\text{lin}}(\cdot; \mathbf{w})$ for each weight vector $\mathbf{w}$.

$MCCS(\Pi)$ . . . The minimal convex coverage set of policies, which has no redundant policies and for each policy in $MCCS(\Pi)$ there is some weight that it uniquely (with respect to $MCCS(\Pi)$) obtains the optimal linear utility.

$CH(\Pi)$ . . . . . The undominated set of policies in Π (the set of all possible policies), with respect to the linear utility function u_{lin}. $CH(\Pi) = U(\Pi; u_{\text{lin}})$.

$CS(\Pi; u)$ . . . . A coverage set of policies, which contains at least one policy that maximises the utility $u(\cdot; \mathbf{w})$ for each weight vector $\mathbf{w}$.

`cvx_prune` . . . An operation that takes an arbitrary set of vectors, and returns the subset that lie at the vertices of the geometric (partial) convex hull.

Δ^D . . . . . The D dimensional simplex, or, the set of possible weightings over the D objectives of a MOMDP. $\Delta^D = \{\mathbf{w} \in \mathbb{R}^D | w_i \geq 0, \sum_i w_i = 1\}$.

Π . . . . . The set of all possible policies in a MOMDP.

$PF(\Pi)$ . . . . . The Pareto front of the set of policies Π.

$\mathbf{Q}^\pi$ . . . . . The multi-objective Q-value function of a policy π. $\mathbf{Q}^\pi(s, a; t)$ denotes the expected cumulative vector reward that policy π will obtain, starting from state $s_t = s$, starting by taking action $a_t = a$.

$U(\Pi; u)$ . . . . . The undominated set of policies in Π (the set of all possible policies), with respect to the utility function u. Each policy in $U(\Pi; u)$ achieves a maximal utility for some weighting over the objectives.

u . . . . . A utility function, mapping multi-objective values to scalar values, that depends on a weighting over objectives. The multi-objective value $\mathbf{V}^\pi(s; t)$ is mapped to $u(\mathbf{V}^\pi(s; t); \mathbf{w})$, where $\mathbf{w}$ is a weighting over the objectives.

u_{lin} . . . . . The linear utility function: $u_{\text{lin}}(\mathbf{v}; \mathbf{w}) = \mathbf{w}^\top \mathbf{v}$.

$\mathbf{V}^\pi$ . . . . . The multi-objective value of a policy π. $\mathbf{V}^\pi(s; t)$ denotes the expected cumulative reward that policy π will obtain, starting from state $s_t = s$.

$\boldsymbol{\mathcal{V}}\mathbf{als}(\Pi')$ . . . . The set of multi-objective values obtained by a set of policies Π'.

$\mathbf{w}$ . . . . . A weighting over objectives that quantifies preferences over the multiple objectives, to be used in a utility function.

1 Introduction

Contents

1.1 Overview 1
1.2 Contributions 2
1.3 Structure of Thesis 4
1.4 Publications 4

TODO: chapter structure (i.e. in the introduction section I give some background in the field(s), cover the main contributions of this thesis, etc, etc).

1.1 Overview

TODO: list

- Give some context around MCTS (and talk about exploration and exploitation), and why we might use it
  - Larger scale than tabular methods
  - Can do probability and theory stuff (and some explainability, by looking at stats in the tree the agent used)
  - Can use tree search with neural networks to get some of the above (and use for neural network training as in alpha zero)

- Argument from DENTS paper for exploration > exploitation (in context of planning in a simulator)
- Give high level overview of Multi-Objective RL, and why it can be useful
- Give an idea of how my work fits into MCTS and MORL as a whole
- Discuss research questions/issues with current literature (i.e. introduce some of the ideas from contributions section below)

1.2 Contributions

TODO: Extra pre-amble before questions?

The following questions, related to Monte Carlo Tree Search and multi-objective reinforcement learning, are considered throughout this thesis:

Q1 - Exploration: When planning in a simulator with limited time, how can Monte Carlo Tree Search algorithms prioritise exploration to make effective decisions?

Q1.1 - Entropy: Entropy is used as an exploration objective in reinforcement learning — under what conditions is it compatible with optimal decision-making?

Q1.2 - Multi-Objective Exploration: Can the exploration insights from (maximum-entropy) single-objective Monte Carlo Tree Search algorithms be applied to multi-objective domains?

Q2 - Stochastic Multi-Objective Domains: Multi-objective Monte Carlo Tree Search has been studied in deterministic domains — can these methods be generalised to stochastic settings?

Q3 - Scalability: How can the scalability of Monte Carlo Tree Search methods be improved, in both single- and multi-objective settings?

Q3.1 - Complexity: Monte Carlo Tree Search algorithms typically run linearly in the number of trials, actions and decision horizon — can this computational complexity be reduced?

Q3.2 - Multi-Objective Scalability: With respect to the size of environments, how scalable are multi-objective Monte Carlo Tree Search methods?

Q3.3 - Curse of Dimensionality: With respect to the number of objectives, to what extent do multi-objective Monte Carlo Tree Search methods suffer from the curse of dimensionality?

TODO: paragraphs pointing at the different chapters and how they answer the research questions.

TODO: some words about how below is the contributions we're making in this thesis and expand these bullets a bit more

- Max Entropy can be misaligned with reward maximisation (**Q1.1 - Entropy**)
- Boltzmann Search Policies - BTS and DENTS (**Q1.1 - Entropy**)
- Use the alias method to make faster algorithms (**Q2.1 - Complexity**)
- Ch5 will answer **Q2 - Stochastic Multi-Objective Domains**
- Simple regret (**Q1 - Exploration**)
- Use of contexts in THTS to make consistent decisions in each trial (**Q1.2 - Multi-Objective Exploration**)
- Contextual regret introduced in CHMCTS (**Q2.2 - Multi-Objective Scalability**)
- Contextual Zooming and CHMCTS (designed for **Q1.2 - Multi-Objective Exploration**, runtimes cover **Q2.3 - Curse of Dimensionality**)
- Simplex maps (**Q1.2 - Multi-Objective Exploration**, **Q2.2 - Multi-Objective Scalability**, **Q2.3 - Curse of Dimensionality**)

1.3 Structure of Thesis

TODO: a paragraph with a couple lines to a paragraph about each chapter. This is the high level overview/intro to the thesis paragraph. I.e. this section is "this is the story of my thesis in a page or two"

1.4 Publications

TODO: update final publication when submit

The work covered in this thesis also appears in the following publications:

- Painter, M; Lacerda, B; and Hawes, N. "Convex Hull Monte-Carlo Tree-Search." In *Proceedings of the international conference on automated planning and scheduling. Vol. 30. 2020*, ICAPS, 2020.
- Painter, M; Baioumy, M; Hawes, N; and Lacerda, B. "Monte Carlo Tree Search With Boltzmann Exploration." In *Advances in Neural Information Processing Systems, 36, 2023*, NeurIPS, 2023.
- Painter, M; Hawes, N; and Lacerda, B. "Simplex Maps for Multi-Objective Monte Carlo Tree Search." In *TODO*, **Under Review at conf_name**.

2
Background

Contents

2.1 Multi-Armed Bandits 6
2.1.1 K-Armed Bandits and the Upper Confidence Bound Algorithm 7
2.1.2 Exploring Bandits 8
2.1.3 Contextual Bandits 10
2.2 Markov Decision Processes 11
2.3 Reinforcement Learning 14
2.3.1 Maximum Entropy Reinforcement Learning 17
2.4 Trial-Based Heuristic Tree Search and Monte Carlo Tree Search 19
2.4.1 Notation 21
2.4.2 Trial Based Heuristic Tree Search 22
2.4.3 Upper Confidence Bound Applied to Trees (UCT) . . . 28
2.4.4 Predictor-UCT (PUCT) 31
2.4.5 Maximum Entropy Tree Search 32
2.5 Convergence of Random Variables 33
2.6 Multi-Objective Reinforcement Learning 34
2.6.1 Convex Hull Value Iteration 40

This chapter introduces the fundamental concepts which are used throughout this thesis, in particular Section **??** introduces the `THTS++` schema, which will be used throughout this thesis to define Monte Carlo Tree Search algorithms modularly.

Section **??** briefly covers Multi-Armed Bandit problems, which are commonly used to study decision making algorithms and as a building block in Monte Carlo

Tree Search methods. In particular, *Contextual* Multi-Armed Bandit problems (Section **??**) are covered, which will be relevant in the design and analysis of the multi-objective algorithms discussed in Chapters **??** and **??**.

Sections **??** and **??** outline the sequential decision-making framework used in this thesis. Additionally, the Maximum Entropy Reinforcement Learning framework is covered (Section **??**) which is a major topic discussed in Chapter **??**.

Section **??** covers the Monte Carlo Tree Search framework which will be used for solving sequential decision-making problems.

Finally, Section **??** covers the Multi-Objective Reinforcement Learning framework, which is used to study sequential decision-making algorithms with multiple objectives.

2.1 Multi-Armed Bandits

The Multi-Armed Bandit (MAB) problem [**?**] is a foundational problem considered in decision theory. In the K-armed bandit problem, an agent must choose which of the K arms to pull over multiple rounds. On each round, after pulling an arm, the agent receives a reward which is sampled from a probability distribution associated with the arm.

The MAB problem was first considered by **?**] in 1952, in the context of clinical trials. In a clinical trial, each patient is treated with one of two possible treatments, where the effectiveness of each treatment is unknown. In this setting, it is essential that the best treatment is given to as many patients as possible, while each treatment needs to be considered to determine its effectiveness. The balance between *exploring* to find the best treatment (arm) and *exploiting* the best treatment (arm) is known as the *exploration-exploitation trade-off*.

The remainder of this section will formalise the MAB problem, give details of the widely used Upper Confidence Bound (UCB) algorithm [**?**], and consider two variations on the MAB problem (Sections **??** and **??**) that are relevant in this thesis.

Parameters: K probability distributions for the rewards of each arm $f(1), \ldots, f(K)$.

- For each round $m = 1, 2, \ldots$:
  - the agent selects an arm $x^m \sim \sigma^m$ to pull;
  - the agent receives a reward $y^m \sim f(x^m)$ from the environment;
  - if the environment sends a stop signal, then the game ends, otherwise the next round starts.

Figure 2.1: The procedure of a multi-armed bandit problem, following a strategy σ.

2.1.1 K-Armed Bandits and the Upper Confidence Bound Algorithm

Figure **??** shows how the operation of a K-armed bandit problem proceeds. Let $f(1), \ldots, f(K)$ be the probability distributions for the rewards of each of the K arms, with expected values of $\mu(1), \ldots, \mu(K)$ respectively. On the mth round, if the arm $x^m \in \{1, \ldots, K\}$ is pulled, then a reward $y^m \sim f(x^m)$ is received. To specify a *strategy* σ, on each round m, a probability distribution σ^m over $\{1, \ldots, K\}$ is given, which is sampled to decide which arm to pull, i.e. $x^m \sim \sigma^m$.

Regret is a common metric used to analyse the performance of algorithms for MAB problems, which is the difference between the cumulative reward obtained and the maximal expected reward that could be obtained with full knowledge of $\{f(i)\}$.

Definition 2.1.1. *The* (cumulative) regret *of strategy σ after n rounds in the MAB process is:*

$$\texttt{cum_regr}_{\text{MAB}}(n, \sigma) = n\mu^* - \sum_{m=1}^{n} y^m, \tag{2.1}$$

where $\mu^* = \max_i \mu(i)$.

The expected regret, $\mathbb{E}[\texttt{cum_regr}_{\text{MAB}}(n, \sigma)]$, is often used to analyse the performance of algorithms for MAB problems. **?**] show a general lower bound of $\Omega(\log n)$ on the expected regret that an agent can achieve. **?**] introduces the Upper Confidence Bound (UCB) algorithm, which achieves a matching upper bound of $O(\log n)$ on the expected regret.

To define the UCB strategy, let $N^m(x)$ be the number of times that arm x has been pulled after m rounds, and let $\bar{y}^m(x)$ be the average reward obtained by pulling arm x after m rounds:

$$N^m(x) = \sum_{i=1}^{m} \mathbb{1}[x^i = x], \tag{2.2}$$

$$\bar{y}^m(x) = \frac{1}{N^m(x)} \sum_{i=1}^{m} y^i \mathbb{1}[x^i = x]. \tag{2.3}$$

The strategy followed by the UCB algorithm on the mth round is given by:

$$x^m_{\text{UCB}} = \arg\max_{i \in \{1,..,K\}} \bar{y}^{m-1}(i) + b_{\text{UCB}} \sqrt{\frac{\log(m)}{N^{m-1}(i)}}, \tag{2.4}$$

$$\sigma^m_{\text{UCB}}(x) = \mathbb{1}[x = x^m_{\text{UCB}}], \tag{2.5}$$

where b_{UCB} is a bias parameter used to control how much the strategy explores. In the first K rounds, UCB pulls each arm once to obtain an initial sample for each arm. Alternatively, the division by zero in Equation **??** can be considered to give a value of ∞, to give the same effect.

2.1.2 Exploring Bandits

In the pure exploration problem for K-armed bandits [**?**], the format of each round is changed slightly. The agent still pulls an arm each round and receives a reward from the environment, but after, the agent is given an opportunity to recommend which arms are the best. This better reflects scenarios where time is spent planning before any action is taken in the real world. Because the agent is only assessed on the quality of the recommendations, the agent is free to purely explore using its arm pulls and only needs to exploit when providing a recommendation.

Figure **??** gives an overview of the Exploring Multi-Armed Bandit (EMAB) problem. The agent follows an *exploration strategy* σ to pull an arm each round, and receives a reward from the environment. Then the agent provides a recommendation in the form of a probability distribution, or *recommendation strategy* ψ, over the K arms.

Algorithms for EMAB problems can be analysed by considering the expected regret of following the recommendation policy for one round, known as *simple regret*.

Parameters: K probability distributions for the rewards of each arm $f(1), \ldots, f(K)$.

- For each round $m = 1, 2, \ldots$:
  - the agent selects an arm $x^m \sim \sigma^m$ to pull;
  - the agent receives a reward $y^m \sim f(x^m)$ from the environment;
  - the agent outputs a recommendation distribution ψ^m, over the arms $\{1, \ldots, K\}$;
  - if the environment sends a stop signal, then the game ends, otherwise the next round starts.

Figure 2.2: The procedure of an exploring multi-armed bandit problem, following an exploration strategy σ, and recommendation strategy ψ.

Definition 2.1.2. *The* simple regret *of the recommendation strategy ψ is:*

$$\texttt{sim_regr}_{\text{EMAB}}(\psi) = \mathbb{E}_{i \sim \psi}[\mu^* - \mu(i)]. \tag{2.6}$$

?] analyse a range of exploration and recommendation strategies in their work. Two of the recommendation strategies considered are the *empirical best arm* (EBA) and *most played arm* (MPA):

$$x^m_{\text{EBA}} = \arg\max_{i \in \{1,..,K\}} \bar{y}^m(i), \tag{2.7}$$

$$\psi^m_{\text{EBA}}(x) = \mathbb{1}[x = x^m_{\text{EBA}}], \tag{2.8}$$

$$x^m_{\text{MPA}} = \arg\max_{i \in \{1,..,K\}} N^m(i), \tag{2.9}$$

$$\psi^m_{\text{MPA}}(x) = \mathbb{1}[x = x^m_{\text{MPA}}]. \tag{2.10}$$

In addition to the UCB exploration strategy (Equation **??**), a *uniform exploration strategy* is considered:

$$x^m_{\text{uniform}} = (m \mod K) + 1, \tag{2.11}$$

$$\sigma^m_{\text{uniform}}(x) = \mathbb{1}[x = x^m_{\text{uniform}}]. \tag{2.12}$$

It is shown that exploring with UCB leads to a polynomial upper bound on the simple regret for both recommendation strategies. That is, when ψ^m_{EBA}

and ψ^m_{MPA} are computed using visit counts and sample averages from following σ_{UCB}, then $\texttt{sim_regr}_{\text{EMAB}}(\psi^m_{\text{EBA}}) = O(m^{-k_{\text{UCB,EBA}}})$ and $\texttt{sim_regr}_{\text{EMAB}}(\psi^m_{\text{MPA}}) = O(m^{-k_{\text{UCB,MPA}}})$ for some appropriate constants $k_{\text{UCB,EBA}}, k_{\text{UCB,MPA}} > 0$.

When the uniform exploration strategy is used instead, and ψ^m_{EBA} is computed similarly using σ_{uniform}, then $\texttt{sim_regr}_{\text{EMAB}}(\psi^m_{\text{EBA}}) = O(e^{-k_{\text{uniform}}m})$ for an appropriate constant $k_{\text{uniform}} > 0$.

2.1.3 Contextual Bandits

The Contextual Multi-Armed Bandit (CMAB) problem [**?** **?** **?**] is another extension of the MAB problem, where the agent is provided with a context $w \in \mathcal{W}$ at the start of each round. After the arm $x \in \mathcal{X}$ is selected, the reward is sampled from the distribution $f(w, x)$, with mean $\mu(w, x)$, which also depends on the context. For each round m, a strategy maps contexts to arms via $\sigma^m(w)$. Figure **??** gives an overview of the CMAB problem.

It is assumed that the *similarity space* $\mathcal{W} \times \mathcal{X}$ is *compact*, and the mean reward function μ is Lipschitz continuous. This requires a *metric* d over the similarity space.

Definition 2.1.3. *d is a* metric *over the similarity space $\mathcal{W} \times \mathcal{X}$ if it satisfies the following conditions for any $z, z', z'' \in \mathcal{W} \times \mathcal{X}$ with $z \neq z' \neq z''$:*

$$d(z, z) = 0, \tag{2.13}$$

$$d(z, z') > 0, \tag{2.14}$$

$$d(z, z') = d(z', z), \tag{2.15}$$

$$d(z, z'') \leq d(z, z') + d(z', z''). \tag{2.16}$$

μ is then Lipschitz continuous if the following condition holds:

$$|\mu(x, y) - \mu(x', y')| \leq d((x, y), (x', y')). \tag{2.17}$$

The performance of CMAB algorithms is analysed with *contextual regret*, a notion of regret that accounts for the contexts drawn.

Parameters: the set of arms $\mathcal{X}$, the set of contexts $\mathcal{W}$ and a mapping from $\mathcal{W} \times \mathcal{X}$ to probability distributions: $f(w, x)$.

- For each round $m = 1, 2, \ldots$:
  - the agent receives a context $w^m \in \mathcal{W}$ from the environment;
  - the agent selects an arm $x^m \sim \sigma^m(w^m)$ to pull;
  - the agent receives a reward $y^m \sim f(w^m, x^m)$ from the environment;
  - if the environment sends a stop signal, then the game ends, otherwise the next round starts.

Figure 2.3: The procedure of a contextual multi-armed bandit problem, following a strategy σ.

Definition 2.1.4. *The* (cumulative) contextual regret *of the strategy σ over n rounds in the process given in Figure* **??** *is defined as:*

$$\texttt{ctx_regr}_{\text{CMAB}}(n, \sigma) = \sum_{m=1}^{n} \mu^*(w^m) - y^m, \tag{2.18}$$

where $\mu^*(w) = \sup_i \mu(w, i)$.

The *contextual zooming* algorithm [**?**] is an algorithm for CMAB problems which is presented where relevant in TODO: ref to where it got updated and placed.

2.2 Markov Decision Processes

Markov decision processes (MDPs) provide a mathematical description of sequential decision making problems [**?**]. This section provides definitions and notation related to MDPs used throughout this thesis.

MDPs are used throughout this thesis to describe the environments in which agents act. An MDP contains, among other things, a set of states and actions. States are sampled according to a transition distribution that depends on the current state and current action being taken (the Markov assumption). Whenever an action is taken, the agent receives a reward according to a reward function that depends on the current state and action.

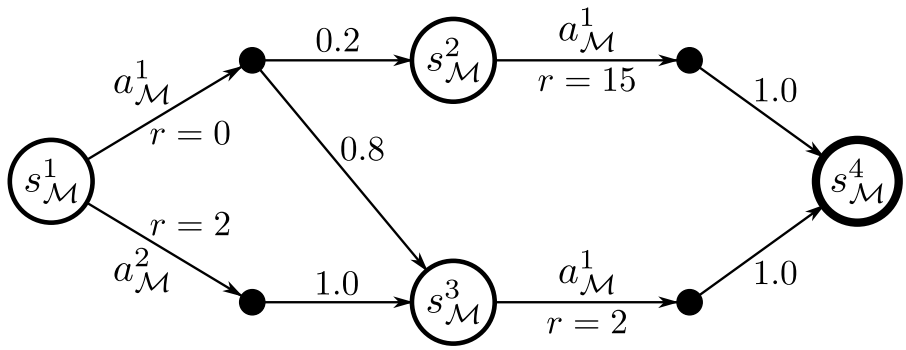


Figure 2.4: An example MDP $\mathcal{M}$, where $s_0 = s^1_{\mathcal{M}}$, the finite horizon is $H = 2$, and terminal states are marked with a thicker border. Edges are appropriately marked with actions and rewards, or marked with transition probabilities.

In this thesis, unless otherwise stated, MDPs are assumed to have finite state and action spaces, and *trajectories* (sequences of states, actions and rewards) are of finite length.

Definition 2.2.1. *A* (Finite-Horizon) Markov decision process *(MDP) is a tuple* $\mathcal{M} = (\mathcal{S}, s_0, \mathcal{A}, R, p, H)$*, where* $\mathcal{S}$ *is a set of states,* $s_0 \in \mathcal{S}$ *is an initial state,* $\mathcal{A}$ *is a set of actions,* $R(s, a)$ *is the reward function* $\mathcal{S} \times \mathcal{A} \to \mathbb{R}$*,* $p(s'|s, a)$ *is a next-state transition distribution* $\mathcal{S} \times \mathcal{A} \to (\mathcal{S} \to [0, 1])$ *and* $H \in \mathbb{N}$ *is the time horizon.*

An example MDP is shown in Figure **??**. The set of *successor states* is the set of states that can be reached after taking an action from the current state of the MDP:

Definition 2.2.2. *The set of* successor states $\mathrm{Succ}(s, a)$ *of a state-action pair* (s, a)*, with respect to an MDP, is defined as:*

$$\mathrm{Succ}(s, a) := \{s' \in \mathcal{S} | p(s'|s, a) > 0\}. \tag{2.19}$$

The set of reachable states from the starting state s_0 can then be defined as the smallest set closed under the successor relation:

Definition 2.2.3. *The set of* reachable states $\mathcal{S}_0$ *from the starting state* s_0 *is the smallest set satisfying:*

$$s_0 \in \mathcal{S}_0, \qquad s \in \mathcal{S}_0 \Rightarrow \bigcup_{a \in \mathcal{A}} \mathrm{Succ}(s, a) \subseteq \mathcal{S}_0. \tag{2.20}$$

In this thesis, MDPs may contain *trap states*, which are states that, once reached, can never be left and from which no further reward is obtained.

Definition 2.2.4. *A state $s \in \mathcal{S}$ is a* terminal *state (equivalently, a* trap *state), if the transition distribution always returns the same state (i.e. $p(s|s,a) = 1$ for all actions a), and if no more reward is obtained from that state ($R(s,a) = 0$ for all a). The set of terminal states is denoted* `terminal` $\subseteq \mathcal{S}$.

A *policy* maps each state to a probability distribution over the action space, and defines a strategy that an agent will follow in an MDP. Often it is desirable to define deterministic policies, which always produce the same action when given the same state, and can be represented as *one-hot* distributions.

Definition 2.2.5. *A* (stochastic) policy $\pi : \mathcal{S} \to (\mathcal{A} \to [0,1])$ *is a mapping from states to a probability distribution over actions. $\pi(a|s)$ is the probability of sampling action a at state s. The policy π must satisfy: $\sum_{a \in \mathcal{A}} \pi(a|s) = 1$ and $\pi(a|s) \geq 0$ for all $s \in \mathcal{S}$ and $a \in \mathcal{A}$.*

Additionally, a deterministic policy *is defined as a one-hot policy, that is, the policy π is deterministic if it can be written as $\pi(a|s) = \mathbb{1}[a = a']$ for some $a' \in \mathcal{A}$.*

Moreover, the following notations are used for policies:

- *$a \sim \pi(\cdot|s)$ denotes sampling an action a from the distribution $\pi(\cdot|s)$;*
- *$\pi(s) = a'$ is a shorthand to define the deterministic policy $\pi(a|s) = \mathbb{1}[a = a']$;*
- *$\pi(s)$ denotes the action a' of the deterministic policy $\pi(a|s) = \mathbb{1}[a = a']$.*

Given an MDP $\mathcal{M}$ and a policy π, a *trajectory* is a sampled sequence of states, actions and rewards, where the policy is used to sample actions and the MDP is used to sample the next state. A trajectory simulates one possible sequence that could occur if an agent follows policy π in $\mathcal{M}$.

Definition 2.2.6. *A* trajectory *τ is a sequence of states, actions and rewards that is induced by a policy π and MDP $\mathcal{M}$. Let the trajectory be $\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \ldots, s_{H-1}, a_{H-1}, r_{H-1}, s_H)$, where $a_t \sim \pi(\cdot|s_t)$, $r_t = R(s_t, a_t)$ and $s_{t+1} \sim p(\cdot|s_t, a_t)$.*

The following notation is also used for trajectories:

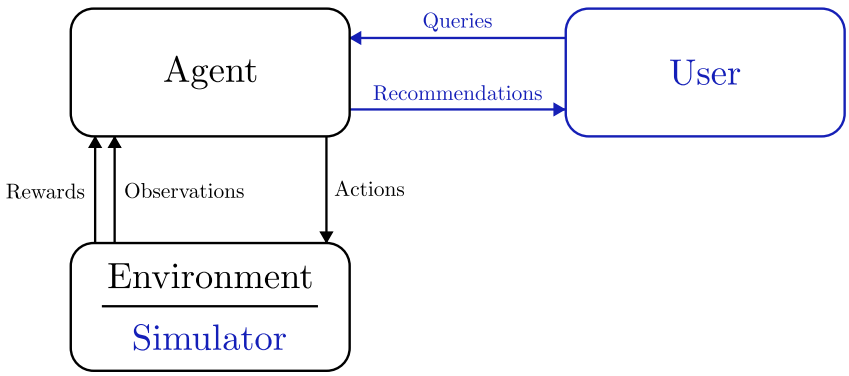


Figure 2.5: An overview of reinforcement learning. Depicted in black is the standard agent-environment interface for reinforcement learning [**?**], where an agent can perform actions in an environment and is given feedback in the form of observations and rewards. In blue the environment is replaced by a simulated environment which the agent can use for planning, and is queried by a user for recommendations on how to act in the real environment.

- *$\tau \sim \pi$ denotes a trajectory that is sampled using the policy π, where the MDP $\mathcal{M}$ is implicit;*
- *$\tau_{i:j}$ denotes the* truncated trajectory *$\tau_{i:j} := (s_i, a_i, r_i, s_{i+1}, \ldots, s_{j-1}, a_{j-1}, r_{j-1}, s_j)$, between the timesteps $0 \leq i < j \leq H$ inclusive;*
- *$\tau_{:j} := \tau_{0:j}$ denotes a trajectory that is truncated on only one end;*
- *$s \in \tau$ is used as a shorthand for $s \in \{s_i | i = 0, \ldots, H\}$;*
- *$(s, a) \in \tau$ is used as a shorthand for $(s, a) \in \{(s_i, a_i) | i = 0, \ldots, H-1\}$;*
- *and finally, when states, actions and rewards are timestep-indexed (i.e. s_t, a_t, r_t), they always correspond to a trajectory.*

2.3 Reinforcement Learning

This section provides a brief introduction to fundamental concepts and definitions in reinforcement learning and motivates an alternative lens on reinforcement learning considered in this thesis. Typically reinforcement learning is presented as a sequence of agent-environment interactions, where the agent takes actions in an environment

and receives *observations* and *rewards* for those actions [?]. The objective of the agent is to maximise the rewards that it receives.

However, in the modern era of reinforcement learning, the agent is often equipped with a simulator in which it can plan, learn and explore. The "target" environment that the user cares about may be the same simulated environment, or a real-world environment where actions carry genuine costs, unlike interactions with the simulator. After the learning phase, one typically cares only about the policy that the agent recommends at the end, rather than the rewards accrued during planning.

Figure **??** depicts the classical reinforcement learning setting (in black) where the agent interacts with the environment directly, and this alternative setting (in blue) where the agent is equipped with a simulator to plan and explore, until a user queries the agent for a recommendation. This perspective shift changes the nature of the exploration-exploitation trade off, almost separating the two issues. During the learning phase there is a greater emphasis on exploring, while recommendations are concerned with pure exploitation. This perspective on reinforcement learning motivates the research questions around exploration: **Q1 - Exploration**.

Following on from Section **??**, the remainder of this section defines *value functions* and the objectives of reinforcement learning and covers *Value Iteration*, a tabular dynamic programming approach to reinforcement learning. Finally, Section **??** covers *maximum-entropy reinforcement learning.*

The value of a policy π is the expected cumulative reward that is obtained by following the policy:

Definition 2.3.1. *The* value *of a policy π from state s at time t is:*

$$V^{\pi}(s;t) = \mathbb{E}_{\tau\sim\pi}\left[\sum_{i=t}^{H-1} r_i \middle| s_t = s\right]. \tag{2.21}$$

The Q-value *of a policy π, from state s, with action a, at time t is:*

$$Q^{\pi}(s,a;t) = R(s,a) + \mathbb{E}_{s'\sim p(\cdot|s,a)}[V^{\pi}(s';t+1)]. \tag{2.22}$$

From the definition of the value functions, the optimal value functions can be defined by taking the maximum value over all policies:

Definition 2.3.2. *The* Optimal (Q-)Value *of a state(-action pair) is defined as:*

$$V^*(s;t) = \max_\pi V^\pi(s;t) \tag{2.23}$$

$$Q^*(s,a;t) = \max_\pi Q^\pi(s,a;t). \tag{2.24}$$

Value functions can also be used to define an objective function:

Definition 2.3.3. *The* (standard) reinforcement learning objective function $J(\pi)$ *is defined as:*

$$J(\pi) = V^\pi(s_0;0). \tag{2.25}$$

The objective of (standard) reinforcement learning can then be stated as finding $\max_\pi J(\pi)$.

The *optimal policy* is the policy that maximises the objective function J, and can be shown to be deterministic [**?**]:

Definition 2.3.4. *The* optimal (standard) policy π^* *is the policy maximising the standard objective function:*

$$\pi^* = \arg\max_\pi J(\pi) \tag{2.26}$$

or equivalently, the optimal standard policy can be expressed in terms of the optimal Q-value function:

$$\pi^*(s) = \arg\max_a Q^*(s,a). \tag{2.27}$$

It can be shown that the optimal (Q-)value functions satisfy the *Bellman equations* [**?** **?**]:

$$V^*(s;t) = \max_{a\in\mathcal{A}} Q^*(s,a;t), \tag{2.28}$$

$$Q^*(s,a;t) = R(s,a) + \mathbb{E}_{s'\sim p(\cdot|s,a)}[V^*(s';t+1)]. \tag{2.29}$$

The Bellman equations admit a *dynamic programming* approach which can be used to compute the optimal value functions, known as *Value Iteration* [**?** **?**]. In

Value Iteration, a table of value estimates $\hat{V}(s;t)$ is kept for each s, t. Given any initial estimate of the value function $\hat{V}^0$, the *Bellman backup* operations are:

$$\hat{V}^{k+1}(s;t) = \max_{a \in \mathcal{A}} \hat{Q}^{k+1}(s,a;t), \tag{2.30}$$

$$\hat{Q}^{k+1}(s,a;t) = \mathbb{E}_{s' \sim p(\cdot|s,a)}[R(s,a) + \hat{V}^k(s';t+1)]. \tag{2.31}$$

In each iteration of Value Iteration, these values are computed for all $s \in \mathcal{S}$, $a \in \mathcal{A}$ and $t \in \{0, 1, \ldots, H\}$. The value estimates computed from the Bellman backups will converge to the optimal values in a finite number of iterations: $V^k \to V^*$.[1]

2.3.1 Maximum Entropy Reinforcement Learning

In *maximum-entropy reinforcement learning*, the objective function is altered by the addition of an entropy term. This is motivated by the desire to learn stochastic behaviours that better explore large state spaces and are more robust under uncertainty by learning multiple solutions rather than a single deterministic solution [**?**].

Let $\mathcal{H}$ denote the (Shannon) entropy function [**?**]:

$$\mathcal{H}(\pi(\cdot|s)) = -\mathbb{E}_{a \sim \pi(\cdot|s)}[\log \pi(a|s)] = -\sum_{a \in \mathcal{A}} \pi(a|s) \log \pi(a|s). \tag{2.32}$$

In the maximum entropy objective, the relative weighting of entropy terms is included using a coefficient α, called the *temperature*. In the maximum entropy objective, analogues of the value functions can be defined, which are typically referred to as *soft (Q-)values*, and similarly the maximum entropy objective is often referred to as the *soft objective*. Where disambiguation is necessary, the *standard (Q-)value* (Definition **??**) and *standard objective* (Definition **??**) will be used to refer to the standard reinforcement learning counterparts.

Definition 2.3.5. *The* soft value *of a policy π from state s at time t is:*

$$V^{\pi}_{\text{sft}}(s;t) = \mathbb{E}_{\tau \sim \pi}\left[\sum_{i=t}^{H-1} r_i + \alpha \mathcal{H}(\pi(\cdot|s_i)) \middle| s_t = s\right]. \tag{2.33}$$

[1] Strictly speaking, if backups are performed in reverse order from $t = H$ to $t = 0$, convergence is achieved in a single iteration. The iterative presentation here is adopted to more closely resemble the backup procedures used later in the thesis, where the transition distribution p is unknown.

The soft Q-value *of a policy π, from state s, with action a, at time t is:*

$$Q^{\pi}_{\text{sft}}(s, a; t) = R(s, a) + \mathbb{E}_{s' \sim p(\cdot|s,a)}[V^{\pi}_{\text{sft}}(s'; t+1)]. \tag{2.34}$$

The optimal soft (Q-)values can be defined by taking the maximum over policies, similar to the standard setting:

Definition 2.3.6. *The* optimal soft (Q-)Value *of a state(-action pair) is defined as:*

$$V^{*}_{\text{sft}}(s; t) = \max_{\pi} V^{\pi}_{\text{sft}}(s; t), \tag{2.35}$$

$$Q^{*}_{\text{sft}}(s, a; t) = \max_{\pi} Q^{\pi}_{\text{sft}}(s, a; t). \tag{2.36}$$

In maximum entropy reinforcement learning, the objective is to find a policy with maximal soft value.

Definition 2.3.7. *The* maximum entropy (or soft) reinforcement learning objective function $J_{\text{sft}}(\pi)$ *is defined as:*

$$J_{\text{sft}}(\pi) = V^{\pi}_{\text{sft}}(s_0; 0). \tag{2.37}$$

The objective of maximum entropy (or soft) reinforcement learning can then be stated as finding $\max_{\pi} J_{\text{sft}}(\pi)$.

The optimal soft policy is defined as the policy that maximises the soft objective function J_{sft}, and it can be computed uniquely from the optimal soft (Q-)values.

Definition 2.3.8. *The* optimal soft policy π^{*}_{sft} *is the policy maximising the soft objective function:*

$$\pi^{*}_{\text{sft}} = \arg\max_{\pi} J_{\text{sft}}(\pi). \tag{2.38}$$

*Given V^{*}_{sft} and Q^{*}_{sft} the optimal soft policy is known to take the form [**?**]:*

$$\pi^{*}_{\text{sft}}(a|s; t) = \exp\left(\left(Q^{*}_{\text{sft}}(s, a; t) - V^{*}_{\text{sft}}(s; t)\right)/\alpha\right). \tag{2.39}$$

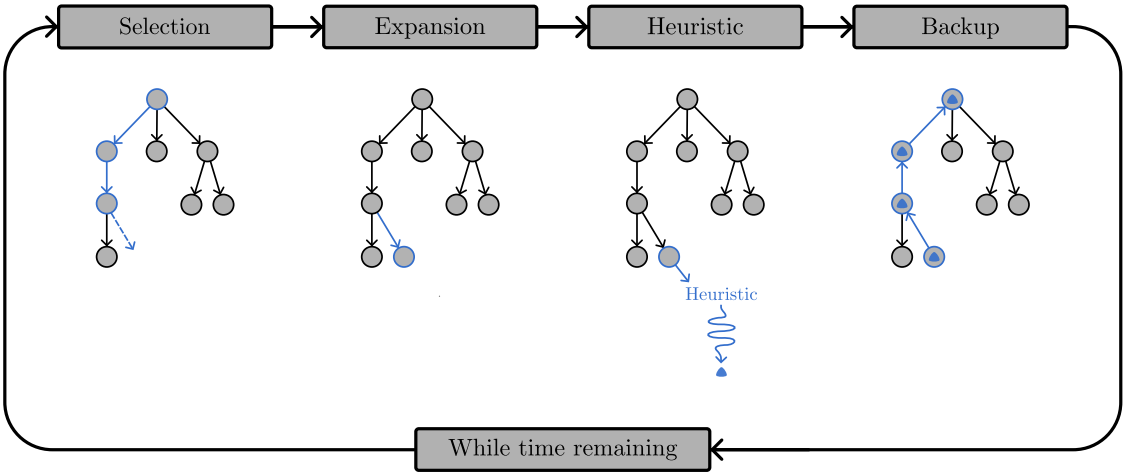


Figure 2.6: Overview of one trial of MCTS-1, depicting the four commonly presented stages [?]. First, in the selection phase a search policy is used to sample a path down the tree. Next a new node is added to the search tree. Then the new node is initialised using a heuristic, often using a *simulation* using a *rollout policy* as depicted. Finally, values (triangles) are updated through *backups* along the path sampled in the selection phase.

The *soft Bellman equations* are the maximum entropy analogues of the Bellman equations. These equations differ from equations (??) and (??) by the replacement of the max operation with a *softmax* or *log-sum-exp* operation, which is why the maximum entropy analogues are referred to as the *soft* versions of their standard reinforcement learning counterparts. The *soft Bellman equations* are [?]:

$$V^*_{\text{sft}}(s;t) = \alpha \log \sum_{a \in \mathcal{A}} \exp\left(Q^*_{\text{sft}}(s,a;t)/\alpha\right), \tag{2.40}$$

$$Q^*_{\text{sft}}(s,a;t) = R(s,a) + \mathbb{E}_{s' \sim p(\cdot|s,a)}[V^*_{\text{sft}}(s';t+1)]. \tag{2.41}$$

Analogous to standard reinforcement learning, the soft Bellman equations can be used in *soft Bellman backups* for a *Soft Value Iteration* algorithm [?]:

$$\hat{V}^{k+1}_{\text{sft}}(s;t) = \alpha \log \sum_{a \in \mathcal{A}} \exp\left(\frac{1}{\alpha}\hat{Q}^{k+1}_{\text{sft}}(s,a;t)\right), \tag{2.42}$$

$$\hat{Q}^{k+1}_{\text{sft}}(s,a;t) = R(s,a) + \mathbb{E}_{s' \sim p(\cdot|s,a)}[\hat{V}^{k}_{\text{sft}}(s';t+1)]. \tag{2.43}$$

2.4 Trial-Based Heuristic Tree Search and Monte Carlo Tree Search

When considering MDPs with large state-action spaces, using tabular dynamic programming methods becomes infeasibly slow. One approach for handling large

state-action spaces is to use *heuristic methods* that consider a subset of the state-action space and utilise heuristic estimates for the value of states. Often these heuristic methods build a *search tree* from the initial state of the MDP.

Monte Carlo Tree Search (MCTS) refers to heuristic algorithms that build a search tree using *Monte Carlo Trials*, where nodes are added to the tree from randomly sampled trajectories. Two advantages of using MCTS methods in the modern day are: that they allow for statistical analysis, which may provide *probabilistic guarantees* on the performance of the algorithm; and that they offer interpretability, as the search tree can be inspected post-hoc to identify why the algorithm made certain decisions.

MCTS trials are commonly presented using four stages, selection, expansion, simulation/heuristic and backup phases [**?**], which are depicted in Figure **??**. To avoid confusion with the broader definition of MCTS above, this thesis refers to this specific form of MCTS as MCTS-1. Each of the four phases of MCTS-1 are described in more detail below:

1. *Selection phase* - a *search policy* is used to traverse the search tree from the root node until it reaches a state not contained in the search tree;

2. *Expansion phase* - a new child node is added to the search tree;

3. *Simulation/Heuristic phase* - the new child node's value is initialised using a *heuristic*, often taking the form of a Monte Carlo return, obtained using a *simulation* with a *rollout policy*;

4. *Backup phase* - the value from the heuristic or simulation is used to back up values through the path traversed in the selection phase.

Trial-Based Heuristic Tree Search (THTS) [**?**] generalises heuristic tree search methods used for planning, such as Trial-Based Real Time Dynamic Programming [**?**] and LAO* [**?**]. However, THTS differs slightly from how MCTS-1 is presented above. In a THTS trial, after the expansion/heuristic phases are run, the trial

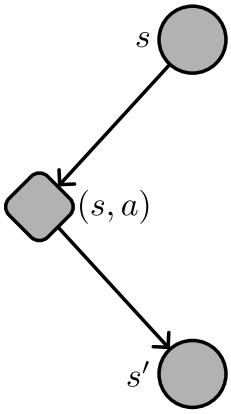


Figure 2.7: Tree diagrams notation, circles denote *decision nodes* that are associated with states, diamonds denote *chance nodes* that are associated with state-action pairs and arrows denote parent/child relationships in the tree.

"switches back to the selection phase and alternates between those phases", until the planning horizon is reached.

`THTS++` [?] is introduced in Section **??**, which is an open-source, parallelised extension of the THTS schema — which also generalises MCTS-1 algorithms — and was built to facilitate the work in this thesis. Section **??** covers the Upper Confidence Bound applied to Trees (UCT) algorithm [? ?], which is a commonly used MCTS algorithm, and Section **??** presents the Maximum ENtropy Tree Search (MENTS) algorithm [?], which is related to the algorithms introduced in Chapter **??** of this thesis.

2.4.1 Notation

To simplify notation in the presentation and analysis of `THTS++` algorithms, this thesis assumes that states and state-action pairs have a one-to-one correspondence with nodes in the search tree. This *unique state assumption* is purely to simplify notation for a clean presentation, and any results discussed in this thesis generalise to settings where this assumption does not hold.

Specifically, this allows the notation for value functions to avoid explicitly writing the timestep parameter, so that $V^\pi(s)$ can be written instead of $V^\pi(s; t)$. With the exception of Section **??**, the timestep parameter is dropped for the remainder of this thesis.

THTS++ builds a *search tree* $\mathcal{T}$ incrementally by running *trials* from the root node and adding them to the tree until a stopping criterion is met. Trees consist of *decision nodes* that correspond to states $s \in \mathcal{T}$, and *chance nodes* that correspond to state-action pairs $(s, a) \in \mathcal{T}$. For brevity, s refers to both states and decision nodes, and (s, a) refers to both state-action pairs and chance nodes. Where necessary for clarity, the notation $\texttt{node}(s)$ and $\texttt{node}(s, a)$ is used to explicitly refer to nodes rather than states or state-action pairs. At the start of search, the search tree is initialised to contain only the root node, $\mathcal{T} \leftarrow \{s_0\}$. Figure **??** gives an example of how decision and chance nodes are depicted as circles and diamonds in diagrams throughout this thesis.

Definition 2.4.1. *A* search tree $\mathcal{T}$ *is a subset of the state and state-action spaces,* $\mathcal{T} \subseteq \mathcal{S} \cup \mathcal{S} \times \mathcal{A}$*, where for each* $s \in \mathcal{T}$*, there exists some truncated trajectory* $\tau_{:h}$ *such that* $s_h = s$*, each* $s' \in \tau_{:h}$ *is also in the tree,* $s' \in \mathcal{T}$*, and each* $s', a' \in \tau_{:h}$ *is also in the tree,* $(s', a') \in \mathcal{T}$*.*

Each decision and chance node stores some algorithm-dependent data, typically including but not limited to a *value estimate* and a *visit count*. Let $N(s)$ and $N(s, a)$ denote the number of times $\texttt{node}(s)$ and $\texttt{node}(s, a)$ have been visited, or equivalently, the number of times s and (s, a) appear in trajectories sampled in THTS++ trials. $\texttt{node}(s)\texttt{.data}$ denotes the set of variables stored at the decision node $\texttt{node}(s)$, and $\texttt{node}(s, a)\texttt{.data}$ denotes the set of variables stored at the chance node $\texttt{node}(s, a)$. For example $N(s) \in \texttt{node}(s)\texttt{.data}$ and $N(s, a) \in \texttt{node}(s, a)\texttt{.data}$.

Additionally, let $\texttt{node}(s)\texttt{.chldrn}$ be a mapping from actions to the chance nodes that are children of $\texttt{node}(s)$. Likewise, $\texttt{node}(s, a)\texttt{.chldrn}$ is a mapping from outcomes (successor states) to the decision nodes that are children of $\texttt{node}(s, a)$. For example, $\texttt{node}(s)\texttt{.chldrn}[a] = \texttt{node}(s, a)$.

2.4.2 Trial Based Heuristic Tree Search

THTS++ [**?**] provides a modular framework for defining and implementing MCTS algorithms. It extends the Trial-Based Heuristic Tree Search (THTS) schema

[?] in a way that incorporates MCTS-1 algorithms and more modern MCTS algorithms that use a neural network or heuristic in place of a rollout simulation [? ?]. It is largely presented in a single-threaded manner, which is sufficient for the purposes of this thesis.

Each trial has an associated *context*, which is a *per-trial* key-value store that can be used to store any algorithm-dependent data. The algorithms defined in Sections ?? and ?? will not make use of this context, but it will be used in Chapters ?? and ?? for multi-objective MCTS algorithms where *contextual tree search* is discussed. Contexts allow algorithms to store additional information within a trial without altering the interface. Beyond the scope of this thesis, contexts enable algorithms that require additional per-trial state to be implemented in `THTS++`, such as Partially Observable Monte Carlo Planning [?], where a state is sampled from a *belief state* at the beginning of each trial.

Definition 2.4.2. *A* context *is an arbitrary key-value store used to store any per-trial data.*

The main four steps performed during a `THTS++` trial are (depicted in Figure ??):

1. *Context Sampling* - a *context* is sampled for the trial;

2. *Selection Step* - starting from a decision node s, an action is selected to reach a chance node (s, a), and an outcome is sampled to reach the next decision node s'. (s, a) and s' are added to the search tree $\mathcal{T}$ if they were not already.

3. *Heuristic Step* - any new nodes added from the selection step are initialised using a *heuristic function*;

4. *Backup Step* - value estimates in the search tree are updated, along the path sampled in the selection step(s).

Collectively, the selection and heuristic steps are referred to as the *selection phase* of the trial, and the backup steps are collectively referred to as the *backup phase*.

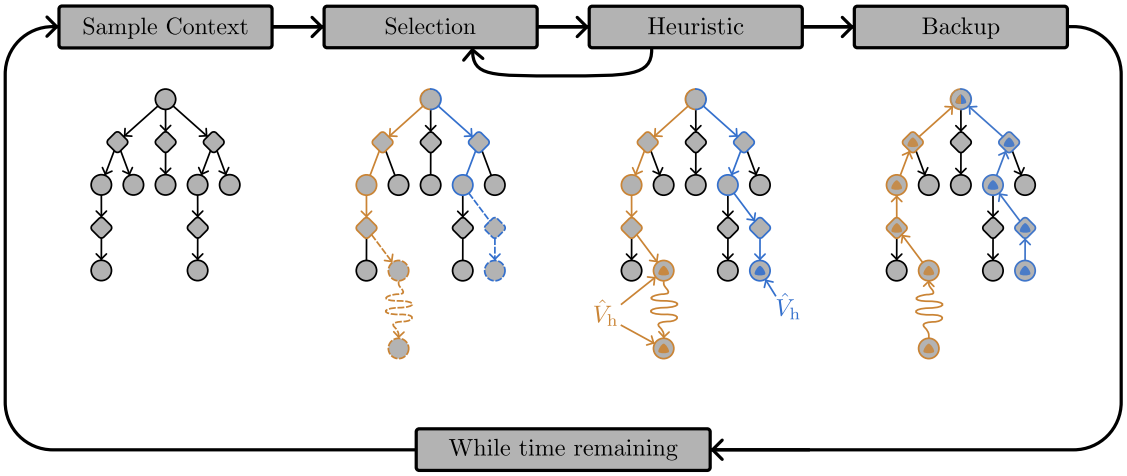


Figure 2.8: Overview of one trial of `THTS++`, blue shows an example when `single_node_expansion` is True, and orange shows an example when `single_node_expansion` is False. From left to right: first a context is sampled, which stores any necessary per-trial state (not depicted) and the search tree at the beginning of the trial is shown; second shows the selection phase, where a trajectory is sampled and where dashed lines indicate any new nodes to be added; third shows that new nodes are initialised using the heuristic function $\hat{V}_{\text{heuristic}}$ (written as $\hat{V}_h$ for short); and finally on the right, shows the backup phase, where the arrow directions are changed to show that information is being propagated back up the tree to update the values in the nodes (triangles).

In `THTS++` there are two options for when to progress from the selection phase to the backup phase, which is indicated by the boolean flag `single_node_expansion`. The selection phase will always end if a terminal state is reached, which includes reaching the planning horizon. When `single_node_expansion` is True, the selection phase will end earlier, once a new decision node is added to the search tree. Figure **??** shows an example of a trial when `single_node_expansion` is True (blue), and when it is False (orange).

Definition 2.4.3. *When* `single_node_expansion` *is True, the selection phase will end once a new decision node is added to the search tree.*

If the search is equipped with an informative heuristic function, then running with `single_node_expansion` set to True can allow the algorithm to avoid wasting resources (time and memory) on parts of the search tree that are not promising. Moreover, if the search is using a large planning horizon or the search will run

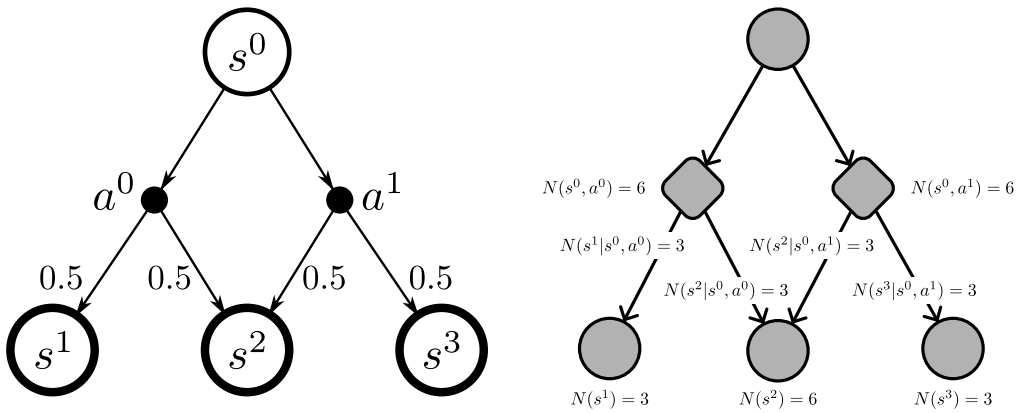


Figure 2.9: A complication arising from the use of a transposition table. Many MCTS algorithms use the ratio $\frac{N(s')}{N(s,a)}$ as an empirical estimate of transition probability $p(s'|s,a)$. Left: an example MDP with horizon $H = 1$ where using this ratio to estimate transition probabilities gives the incorrect ratios. Right: the corresponding *acyclic graph* created by running `THTS++` on the MDP with `transposition_table` set to True and sampling actions uniformly. After 12 trials, the resulting visit counts at the leaf nodes are $N(s^1) = 3, N(s^2) = 6, N(s^3) = 3$. The ratio of visit counts $\frac{N(s^1)}{N(s^0,a^1)} = \frac{1}{2}$ and $\frac{N(s^2)}{N(s^0,a^1)} = 1$, which do not form an (empirical) probability distribution, and even if normalised do not give the correct uniform distribution over successor states. To fix this issue, the *conditional visit counts* $N(s'|s,a)$ for $s' \in \text{Succ}((,s),a)$ are needed instead, that keep track of the number of times s' was sampled from the chance node (s,a). In this example, the conditional visit counts at node (s^0,a^0) would be $N(s^1|s^0,a^0) = 3$ and $N(s^2|s^0,a^0) = 3$, and give the correct ratios of $\frac{N(s^1|s^0,a^0)}{N(s^0,a^0)} = \frac{N(s^2|s^0,a^0)}{N(s^0,a^0)} = \frac{1}{2}$. That is, the ratios $\frac{N(s'|s,a)}{N(s,a)}$ correctly maintain an empirical estimate for the probability $p(s'|s,a)$, even when using a transposition table.

for a long time, then it may be necessary to run with `single_node_expansion` set to True to avoid running out of memory.

In contrast, a random *rollout* is often used in MCTS-1 algorithms when no informative heuristic is available (for example Section **??**). In such cases, where memory allows, running with `single_node_expansion` set to False can be beneficial, because it avoids throwing away potentially useful information from states visited during the rollout [**?** **?**].

If the unique state assumption (Section **??**) is set aside, environments may contain *transpositions*, where the same state can be reached from multiple different paths. When the `transposition_table` flag is set, a *transposition table* is used to re-use the `node`(s) when the state s is reached again via another path [**?** **?** **?**].

Many MCTS algorithms, whether explicitly or implicitly, use the ratio of visit

counts, $\frac{N(s')}{N(s,a)}$, as an empirical estimate of the transition distribution $p(s'|s,a)$. Figure **??** illustrates the issue that arises with this when using a transposition table and how it can be resolved, following an argument similar to that of **?**].

To run an algorithm in the `THTS++` schema, the following *modules* and *parameters* need to be specified. Algorithms specify the operation of each module:

Value Estimates: What data will be stored at each decision and chance node. That is, `node`(s)`.data` and `node`(s,a)`.data` need to be defined. It is assumed that `node`(s)`.data` and `node`(s,a)`.data` always contain the visit counts $N(s)$, and $N(s,a)$ respectively;

Initialisation: Two functions `init_v` and `init_q` which specify how to initialise `node`(s)`.data` and `node`(s,a)`.data` respectively;

Context Sampler: A function `sample_context` which creates a context key-value store, and samples any initial values to be stored in the context;

Action Selection (Search Policy): A function `select_action` which selects an action at a decision node. In this thesis, this will be specified as a function π_{search}, and `select_action` will return an action sampled from the search policy: $a \sim \pi_{\text{search}}(s)$;

(Optional) Outcome Sampler: A function `sample_outcome` that samples outcomes from chance nodes. By default, this will sample a state from the transition distribution of the MDP: $s' \sim p(\cdot|s,a)$ and this default will always be used in this thesis[2];

Backup functions: Two functions `backup_v` and `backup_q` which specify how to update `node`(s)`.data` and `node`(s,a)`.data` respectively.

Parameters are set on a per-environment basis:

[2]One may want to use a different `sample_outcome` function to implement *double progressive widening* [**?**] for example.

(Optional) Heuristic functions: The functions $\hat{V}_{\text{heuristic}}$, $\hat{Q}_{\text{heuristic}}$ and $\pi_{\text{heuristic}}$ are the heuristic functions. $\hat{V}_{\text{heuristic}}$ is used to initialise value estimates in `node`(s)`.data`. $\hat{Q}_{\text{heuristic}}$ and $\pi_{\text{heuristic}}$ can be used by the search policy to incorporate prior knowledge about unexplored actions. If not specified then by default $\hat{V}_{\text{heuristic}}$ and $\hat{Q}_{\text{heuristic}}$ are set to zero, and $\pi_{\text{heuristic}}$ is set to a uniform distribution;

Single Node Expansion: A boolean flag, `single_node_expansion`, specifying whether `THTS++` should only expand one decision node per trial;

Transposition Table: A boolean flag, `transposition_table`, specifying whether `THTS++` should use a transposition table to re-use nodes when the same state is reached via another path;

Planning horizon: A (problem dependent) planning horizon for the tree search $H_{\texttt{THTS++}}$, which is no longer than the horizon of the MDP, $H_{\texttt{THTS++}} \leq H$[3].

Given all of the modules and parameters are specified, the full operation of a trial in `THTS++` is described below. Additionally, pseudocode is given in Listing **??**.

Sample Context: At the start of the trial, a context is sampled using the `sample_context` function, and the context is passed to all subsequent steps of the trial.

Selection Phase: A trajectory is sampled using the `select_action` and `sample_outcome` functions, and new nodes are initialised using the heuristic functions and added to the search tree. More specifically, starting with $i = 0$:

Selection Step: Action a_i is selected using `select_action`(s_i), and an outcome s_{i+1} is sampled using `sample_outcome`(s_i, a_i).

Heuristic Step: If $(s_i, a_i) \notin \mathcal{T}$ initialise `node`(s_i, a_i)`.data` using `init_q`, and if $s_{i+1} \notin \mathcal{T}$ initialise `node`(s_{i+1})`.data` using `init_v`. New nodes are added to the tree, $\mathcal{T} \leftarrow \mathcal{T} \cup \{(s_i, a_i), s_{i+1}\}$.

[3] In this thesis, MDPs have a finite horizon H, so $H_{\texttt{THTS++}}$ is somewhat redundant. However, it is useful in practice for environments with a long or infinite horizon, where a finite planning horizon is still required for the tree search.

If $s_{i+1} \in$ `terminal` or $i + 1 = H_{\texttt{THTS++}}$, then proceed to the backup phase.

If `single_node_expansion` and s_{i+1} was just added to the tree, then proceed to the backup phase.

Otherwise, increment i and return to the selection step.

Backup Phase: For $i = h - 1, \ldots, 0$, where h is the final value of i in the selection phase, the visit counts $N(s_i)$ and $N(s_i, a_i)$ are incremented, and the `backup_q` and `backup_v` functions are used to update the values of `node`(s_i, a_i)`.data` and `node`(s_i)`.data` respectively.

2.4.3 Upper Confidence Bound Applied to Trees (UCT)

Upper Confidence Bound Applied to Trees (UCT) [**?** **?**] is a commonly used tree search algorithm, which is based on the Upper Confidence Bound (UCB) algorithm covered in Section **??**. UCT is a good example of a common paradigm in tree search algorithms for sequential decision making, where each node in the tree is tasked with making a single decision in the sequence, and so adapts methods from the MAB literature. More specifically, UCT can be viewed as running UCB on a non-stationary MAB at every decision node, where it is non-stationary because the rewards obtained depend on the decisions that children make. The remainder of this subsection will outline how UCT can be defined using the `THTS++` schema given in Section **??**.

The variables used in UCT consist of sample average value estimates $\bar{V}_{\text{UCT}}(s)$ and $\bar{Q}_{\text{UCT}}(s, a)$, visit counts $N(s)$ and $N(s, a)$. The parameters consist of: b_{UCT}, a bias parameter which controls the amount of exploration UCT will perform and `normalised_Q`, a flag indicating whether the Q-value estimates should be normalised when used in the search policy, which is useful if the Q-values can vary in magnitude throughout the search tree. Typically `single_node_expansion` is set to True for UCT, and this section assumes that this is the case.

```
def run_trial(search_tree: 𝒯,
              search_policy: π_search,
              heuristic_fn: V̂_heuristic,
              heuristic_action_fn: Q̂_heuristic,
              heuristic_policy: π_heuristic,
              single_node_expansion,
              planning_horizon: H_THTS++):
    ctx = sample_context()
    τ_:h = selection_phase(𝒯, π_search, V̂_heuristic, Q̂_heuristic, π_heuristic, ↩
        single_node_expansion, H_THTS++, ctx)
    backup_phase(τ_:h, ctx)

def selection_phase(search_tree: 𝒯,
                    search_policy: π_search,
                    heuristic_fn: V̂_heuristic,
                    heuristic_action_fn: Q̂_heuristic,
                    heuristic_policy: π_heuristic,
                    single_node_expansion,
                    planning_horizon: H_THTS++,
                    ctx):
    i = 0
    while (not single_node_expansion or s_i ∈ 𝒯)
            and (i < H_THTS++)
            and (s_i ∉ terminal):
        # selection step
        a_i ← select_action(s_i, Q̂_heuristic, π_heuristic, ctx)
        r_i ← R(s_i, a_i)
        s_i+1 ← sample_outcome(s_i, a_i, ctx)
        # heuristic step
        if (s_i, a_i) ∉ 𝒯:
            init_q((s_i, a_i), Q̂_heuristic, π_heuristic, ctx)
        if s_i+1 ∉ 𝒯:
            init_v(s_i+1, V̂_heuristic, π_heuristic, ctx)
        𝒯 ← 𝒯 ∪ {(s_i, a_i), s_i+1}
        i += 1
    return (s_0, a_0, r_0, s_1, ..., s_i-1, a_i-1, r_i-1, s_i)

def select_action(s, Q̂_heuristic, π_heuristic, ctx):
    a ~ π_search(·|s, Q̂_heuristic, π_heuristic, ctx)
    return a

def sample_outcome(s, a, ctx):
    s' ~ p(·|s, a)
    return s'

def backup_phase(τ_:h, ctx)
    for i in {h−1, h−2, ..., 1, 0}:
        N(s_i, a_i) ← N(s_i, a_i) + 1
        N(s_i) ← N(s_i) + 1
        node(s_i, a_i).backup_q(node(s_i, a_i).chldrn, τ_i:h, ctx)
        node(s_i).backup_v(node(s_i).chldrn, τ_i:h, V̂_heuristic(s_h), ctx)
```

Listing 2.1: Pseudocode for running a trial in THTS++.

The search policy that UCT follows is given by:

$$\bar{Q}^{\text{norm}}_{\text{UCT}}(s,a) = \begin{cases} \frac{\bar{Q}_{\text{UCT}}(s,a) - \min_{a\in\mathcal{A}} \bar{Q}_{\text{UCT}}(s,a)}{\max_{a\in\mathcal{A}} \bar{Q}_{\text{UCT}}(s,a) - \min_{a\in\mathcal{A}} \bar{Q}_{\text{UCT}}(s,a)} & \text{if } \texttt{normalised_Q}, \\ \bar{Q}_{\text{UCT}}(s,a) & \text{otherwise}, \end{cases} \tag{2.44}$$

$$\pi_{\text{UCT}}(s) = \arg\max_{a\in\mathcal{A}} \bar{Q}^{\text{norm}}_{\text{UCT}}(s,a) + b_{\text{UCT}} \sqrt{\frac{\log(N(s))}{N(s,a)}}, \tag{2.45}$$

In Equation (**??**), when $N(s,a) = 0$ there is a division by zero, which is taken as ∞, and ties are broken uniformly randomly. Note that like UCB (Section **??**), this results in every action being taken once before any action is selected for a second time. The heuristic functions $\hat{Q}_{\text{heuristic}}$ and $\pi_{\text{heuristic}}$ are not used by UCT.

Let $\tau_{:h} \sim \pi_{\text{UCT}}$ be the trajectory sampled in the selection phase of UCT. For any chance nodes (s,a) added to the tree, the `init_q` function initialises the visit count $N(s,a) = 0$ and the Q-value estimate $\bar{Q}_{\text{UCT}}(s,a) = \hat{Q}_{\text{heuristic}}(s,a)$, which will be overwritten by `backup_q` on the first visit (Equation **??**). The `init_v` function initialises the leaf node s_h using the heuristic function and with a single visit:

$$N(s_h) = 1, \tag{2.46}$$

$$\bar{V}_{\text{UCT}}(s_h) = \hat{V}_{\text{heuristic}}(s_h). \tag{2.47}$$

In UCT the backups `backup_v` and `backup_q` update the (sample average) value estimates incrementally. For $t = h-1, \ldots, 0$:

$$\bar{Q}_{\text{UCT}}(s_t,a_t) \leftarrow \frac{N(s_t,a_t)-1}{N(s_t,a_t)} \left(\bar{Q}_{\text{UCT}}(s_t,a_t) + \frac{\hat{V}_{\text{heuristic}}(s_h) + \sum_{i=t}^{h-1} r_i}{N(s_t,a_t)} \right), \tag{2.48}$$

$$\bar{V}_{\text{UCT}}(s_t) \leftarrow \frac{N(s_t)-1}{N(s_t)} \left(\bar{V}_{\text{UCT}}(s_t) + \frac{\hat{V}_{\text{heuristic}}(s_h) + \sum_{i=t}^{h-1} r_i}{N(s_t)} \right). \tag{2.49}$$

There are two common approaches for implementing $\hat{V}_{\text{heuristic}}$ in UCT. The first approach uses a function approximation $\hat{V}_{\text{heuristic}} = \tilde{V}$ [**? ? ?**], where $\tilde{V}$ is often a neural network that aims to approximate the optimal value function V^*. The second approach uses a *rollout*, which completes the trajectory from the selection phase using a *rollout policy* and uses the rewards from the rollout to give a heuristic value estimate [**? ? ? ?**]. If no informative policy is available for π_{rollout}, then a uniformly random policy is often used [**? ?**].

To use a rollout policy π_{rollout} in THTS++, a Monte Carlo estimate $\hat{V}^{\pi_{\text{rollout}}}$ of the value function $V^{\pi_{\text{rollout}}}$ is used for $\hat{V}_{\text{heuristic}}$. Let $\tau_{h:h'} \sim \pi_{\text{rollout}}$ be the rollout trajectory, where $h' = H_{\texttt{THTS++}}$ or $s_{h'} \in \texttt{terminal}$. The Monte Carlo estimate is then the sum of the rewards from the rollout:

$$\hat{V}^{\pi_{\text{rollout}}}(s_h) = \sum_{i=h}^{h'-1} r_i. \tag{2.50}$$

2.4.4 Predictor-UCT (PUCT)

TODO: Maybe move to related work Ch3, Sec 2.4 either is THTS with all the algorithms, or THTS with a couple examples. PUCT, RENTS, TENTS can go in the related work section. Should also define MaxUCT.

AlphaGo and AlphaGo Zero [? ?] popularised a variant of UCT which is given in this section and referred to as *Predictor-UCT* (PUCT). In [?], this variant is ambiguously defined as "PUCT", citing *predictor UCB* [?] as the source, while more closely resembling *polynomial UCB* [?].

PUCT uses similar variables to UCT (Section ??), the sample average value estimates $\bar{V}_{\text{PUCT}}(s)$ and $\bar{Q}_{\text{PUCT}}(s,a)$, visit counts $N(s)$ and $N(s,a)$, the bias parameter b_{PUCT}, and the flag `normalised_Q`.

The PUCT search policy is similar to the UCT search policy (Equation ??), with the following changes:

- The logarithmic confidence interval term in Equation (??) is replaced by a polynomial term [?];
- The confidence interval term is weighted by the *predictor values* $\pi_{\text{heuristic}}(a|s)$, inspired by predictor UCB [?];
- The $N(s,a)$ denominator is modified to $N(s,a)+1$, which allows actions to be selected more than once before all actions have been tried, making the policy better suited to large action spaces such as in Go.

The operation of PUCT is otherwise identical to UCT, except for the changes to the search policy. The PUCT selection policy is given by:

$$\pi_{\text{PUCT}}(s) = \arg\max_{a \in \mathcal{A}} \bar{Q}^{\text{norm}}_{\text{PUCT}}(s,a) + b_{\text{PUCT}} \pi_{\text{heuristic}}(a|s) \frac{\sqrt{N(s)}}{N(s,a)+1}. \tag{2.51}$$

In `THTS++`, the values of $\bar{Q}_{\text{PUCT}}(s,a)$ are initialised using the heuristic action value $\hat{Q}_{\text{heuristic}}(s,a)$. When $\bar{Q}_{\text{PUCT}}(s,a)$ is updated via the backup (Equation (**??**)), the heuristic initialisation is replaced by the sample average estimate.

2.4.5 Maximum Entropy Tree Search

Maximum ENtropy Tree Search (MENTS) [**?**], in contrast to UCT, focuses on the maximum-entropy objective, and uses soft Bellman backups to update its value estimates. In its original presentation, `single_node_expansion` is set to True, and it uses the soft value estimates $\hat{V}_{\text{MENTS}}$ and $\hat{Q}_{\text{MENTS}}$. The MENTS search policy is given by:

$$\pi_{\text{MENTS}}(a|s) = (1 - \lambda(s, \epsilon_{\text{MENTS}}))\rho_{\text{MENTS}}(a|s) + \frac{\lambda(s, \epsilon_{\text{MENTS}})}{|\mathcal{A}|} \tag{2.52}$$

$$\rho_{\text{MENTS}}(a|s) = \exp\left(\frac{1}{\alpha_{\text{MENTS}}}\left(\hat{Q}_{\text{MENTS}}(s,a) - \hat{V}_{\text{MENTS}}(s)\right)\right), \tag{2.53}$$

$$\lambda(s,\epsilon) = \min\left(1, \frac{\epsilon}{\log(e + N(s))}\right). \tag{2.54}$$

where $\epsilon_{\text{MENTS}} \in (0, \infty)$ is an exploration parameter, α_{MENTS} is the temperature parameter used in MENTS for the maximum entropy objective (the coefficient of the entropy term in Equation (**??**)), and λ is a decay function that is also used in later chapters with different arguments.

Let $\tau_{:h} \sim \pi_{\text{MENTS}}$ be the trajectory sampled in the selection phase of MENTS. For any chance nodes (s,a) added to the tree, the `init_q` function initialises the visit count $N(s,a) = 0$ and the Q-value estimate $\hat{Q}_{\text{MENTS}}(s,a) = \hat{Q}_{\text{heuristic}}(s,a)$, which will be overwritten by `backup_q` on the first visit (Equation **??**). The `init_v` function initialises the leaf node s_h using the heuristic function and with a single visit:

$$N(s_h) = 1, \tag{2.55}$$

$$\hat{V}_{\text{MENTS}}(s_h) = \hat{V}_{\text{heuristic}}(s_h). \tag{2.56}$$

The backups `backup_v` and `backup_q` then update the soft value estimates using soft Bellman backups (Equations (**??**) and (**??**)) as follows for $t = h-1, \ldots, 0$:

$$\hat{Q}_{\text{MENTS}}(s_t, a_t) \leftarrow R(s_t, a_t) + \sum_{s' \in \text{Succ}(s_t, a_t)} \left(\frac{N(s')}{N(s_t, a_t)} \hat{V}_{\text{MENTS}}(s') \right), \tag{2.57}$$

$$\hat{V}_{\text{MENTS}}(s_t) \leftarrow \alpha_{\text{MENTS}} \log \sum_{a \in \mathcal{A}} \exp \left(\frac{1}{\alpha_{\text{MENTS}}} \hat{Q}_{\text{MENTS}}(s_t, a) \right). \tag{2.58}$$

In [**?**], the heuristic value function is left as an arbitrary evaluation function, but is set using a function approximation $\hat{V}_{\text{heuristic}} = \tilde{V}$ in experiments. The heuristic action function is set to zero, $\hat{Q}_{\text{heuristic}}(s, a) = 0$, however the authors also suggest that if *policy network* $\tilde{\pi}$ is available, then the heuristic action function can alternatively be set to $\hat{Q}_{\text{heuristic}}(s, a) = \log \tilde{\pi}(a|s)$.

2.5 Convergence of Random Variables

This section introduces the two modes of convergence used in the proofs in this thesis and states existing results that will be used.

Definition 2.5.1. *A sequence of random variables* X_i converges in probability *to a random variable* X*, written as* $X_i \xrightarrow{p} X$*, if for all* $\varepsilon > 0$ *the following holds:*

$$\Pr\left(|X_i - X| > \varepsilon\right) \to 0 \text{ as } i \to \infty. \tag{2.59}$$

Definition 2.5.2. *A sequence of random variables* X_i converges almost surely *to a random variable* X*, written as* $X_i \xrightarrow{as} X$*, if the following holds:*

$$\Pr\left(\lim_{i \to \infty} X_i = X\right) = 1. \tag{2.60}$$

Almost sure convergence is a stronger condition than convergence in probability: almost sure convergence implies convergence in probability, but not vice versa [**?**].

The Dvoretzky-Kiefer-Wolfowitz (DKW) inequality [**?**] bounds the maximum difference between the empirical distribution of samples and the true distribution:

Theorem 2.5.1. *Let* $\{X_i\}_{i=1}^m$ *be random variables drawn from a probability distribution with a cumulative distribution function of* F*. Let the empirical cumulative*

Figure 2.10: The decision-support scenario for multi-objective reinforcement learning [**?**].

distribution function be $F_m(x) = \frac{1}{m}\sum_{i=1}^{m} \mathbb{1}[X_i < x]$. *Then the Dvoretzky-Kiefer-Wolfowitz inequality is:*

$$\Pr\left(\sup_x |F_m(x) - F(x)| > \varepsilon\right) \leq 2\exp\left(-2m\varepsilon^2\right). \tag{2.61}$$

The DKW inequality is used in Chapters **??** and **??** to bound the error when using the empirical visit count ratio $\frac{N(s')}{N(s,a)}$ as an approximation to the transition probability $p(s'|s,a)$.

2.6 Multi-Objective Reinforcement Learning

This thesis follows a utility-based approach to multi-objective reinforcement learning similar to the review of **?**]. This work will specifically consider *linear utility* functions and the *decision support scenario* which is depicted in Figure **??**. In the decision support scenario, an algorithm computes a *solution set* consisting of multiple *possibly optimal* solutions, from which a user then picks their most preferred option to be used. This scenario is useful when a user's preferences over the multiple objectives are unknown or difficult to specify in advance.

This section defines the multi-objective counterparts to various definitions given in Section **??** and **??**. Outside of this section, the prefix "multi-objective" may be dropped where it should be clear from context; however, bold typeface will consistently be used to denote any vector variables or functions. In Section **??**, a multi-objective extension of Value Iteration is given.

To specify problems with multiple objectives, the reward function of an MDP is changed to give a vector of rewards, rather than a scalar reward:

Definition 2.6.1. *A* Multi-Objective Markov Decision Process *(MOMDP) is a tuple* $\boldsymbol{\mathcal{M}} = (\mathcal{S}, s_0, \mathcal{A}, \mathbf{R}, p, H)$, *where* $\mathcal{S}$ *is a set of states,* $s_0 \in \mathcal{S}$ *is an initial state,*

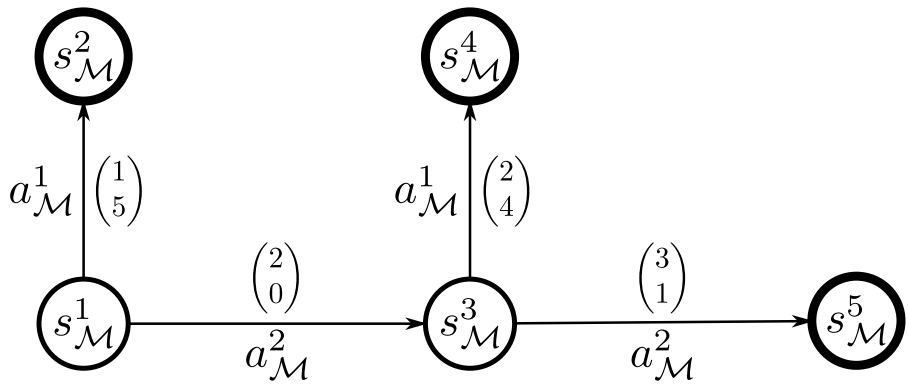


Figure 2.11: An example MOMDP $\boldsymbol{\mathcal{M}}$, where $s_0 = s^1_{\mathcal{M}}$, the finite horizon is $H = 2$ and terminal states are marked with thicker borders. This particular MOMDP is deterministic, so that action nodes are omitted and all transition probabilities are one.

$\mathcal{A}$ is a set of actions, $\mathbf{R}(s, a)$ is a vector reward function $\mathcal{S} \times \mathcal{A} \to \mathbb{R}^D$, where D is the dimension of the rewards and the MOMDP, $p(s'|s, a)$ is a next state transition distribution $\mathcal{S} \times \mathcal{A} \times \mathcal{S} \to [0, 1]$ and $H \in \mathbb{N}$ is a finite-horizon time bound.

Multi-objective trajectories are defined analogously to single-objective trajectories (Definition **??**):

Definition 2.6.2. *A* multi-objective trajectory *τ, is a sequence of state, action and vector rewards, that is induced by a policy π and MOMDP $\boldsymbol{\mathcal{M}}$ pair. Let the trajectory be $\tau = (s_0, a_0, \mathbf{r}_0, s_1, a_1, \mathbf{r}_1, \ldots, s_{H-1}, a_{H-1}, \mathbf{r}_{H-1}, s_H)$, where $a_t \sim \pi(\cdot|s_t)$, $\mathbf{r}_t = \mathbf{R}(s_t, a_t)$ and $s_{t+1} \sim p(\cdot|s_t, a_t)$.*

*The notations used for single-objective trajectories (Definition **??**) will also be used for multi-objective trajectories. Such as, $\tau \sim \pi$ for sampling trajectories using policies, and $\tau_{i:j}$ for truncated trajectories.*

Similarly, multi-objective variants of the (Q-)value of a policy are defined:

Definition 2.6.3. *The* multi-objective value *of a policy π from state s at time t is:*

$$\mathbf{V}^\pi(s; t) = \mathbb{E}_{\tau\sim\pi}\left[\sum_{i=t}^{H-1} \mathbf{r}_i \middle| s_t = s\right]. \tag{2.62}$$

The multi-objective Q-value *of a policy π, from state s, with action a, at time t is:*

$$\mathbf{Q}^\pi(s, a; t) = \mathbf{R}(s, a) + \mathbb{E}_{s'\sim p(\cdot|s,a)}[\mathbf{V}^\pi(s'; t + 1)]. \tag{2.63}$$

In the corresponding point of the single-objective reinforcement learning section (Section **??**), the optimal (Q-)value functions and the objective of single-objective reinforcement learning were defined. However, in a multi-objective setting there is no longer a *total ordering* over values, and so there may be multiple vectors that could be "optimal". For example, consider two vectors $\mathbf{u}, \mathbf{v} \in \mathbb{R}^2$ which are such that $u_1 > v_1$ and $u_2 < v_2$. To resolve this issue, a *utility function* or *scalarisation function* is used to map multi-objective values to scalar values.

Definition 2.6.4. *The* (D-dimensional) Simplex *consists of the set of D-dimensional vectors, whose entries are non-negative and sum to one. More formally, the D dimensional simplex is*

$$\Delta^D = \{\mathbf{w} \in \mathbb{R}^D | w_i \geq 0, \sum_i w_i = 1\}. \tag{2.64}$$

The elements of the D-dimensional Simplex will be referred to as weight vectors *(or just* weights*) in this thesis, as they will be used to specify preferences over the D dimensions of the reward function.*

Definition 2.6.5. *A* utility function *(or* scalarisation function*) $u : \mathbb{R}^D \times \Delta^D \to \mathbb{R}$ is used to map from a multi-objective value $\mathbf{v} \in \mathbb{R}^D$ and a weighting over the objectives $\mathbf{w} \in \Delta^D$ to a scalar value. That is, according to the utility function $u(\cdot; \mathbf{w})$ the multi-objective value $\mathbf{v}$ is mapped to the scalar value $u(\mathbf{v}; \mathbf{w})$.*

Of particular interest in this thesis is the *linear utility function* where the scalar value takes the form of a dot-product:

Definition 2.6.6. *The* linear utility function u_{lin} *is the utility function defined by:*

$$u_{\text{lin}}(\mathbf{v}; \mathbf{w}) = \mathbf{w}^\top \mathbf{v}. \tag{2.65}$$

Equipped with a utility function and a weight vector, any set of multi-objective values can be mapped to scalars and ordered. A policy is *possibly optimal*, or *undominated*, with respect to utility function u if it achieves a maximal utility for some weight $w \in \Delta^D$. In contrast, a policy is *dominated* if for every weight there is another policy that achieves a better utility.

Let Π be the set of all possible policies for a given MOMDP. When constructing a solution set, a subset of Π, it makes sense that it should not contain any dominated policies. This leads to the first notion of a solution set, called an *undominated set*, consisting of all undominated policies:

Definition 2.6.7. *The* undominated set *of policies $U(\Pi; u) \subseteq \Pi$, with respect to a utility function u, is the set of policies for which there is a weight vector $\mathbf{w} \in \Delta^D$ where the utility (scalarised value) is maximised:*

$$U(\Pi; u) = \left\{ \pi \in \Pi \;\middle|\; \exists \mathbf{w} \in \Delta^D . \forall \pi' \in \Pi : u(\mathbf{V}^{\pi}(s_0; 0); \mathbf{w}) \geq u(\mathbf{V}^{\pi'}(s_0; 0); \mathbf{w}) \right\}. \tag{2.66}$$

In particular, the convex hull *of policies $CH(\Pi)$ is the undominated set with respect to the linear utility function u_{lin}. That is $CH(\Pi) = U(\Pi; u_{\text{lin}})$.*

Undominated sets often have an infinite cardinality, and as such are infeasible to compute. However, in undominated sets there are often many redundant policies that obtain the same scalarised values. Instead of computing an undominated set, it is more feasible to compute a *coverage set* that contains at least one policy that maximises the scalarised value given any weight vector $\mathbf{w}$:

Definition 2.6.8. *A set $CS(\Pi; u) \subseteq U(\Pi)$, is a* coverage set *with respect to a utility function u, if for every weight vector $\mathbf{w} \in \Delta^D$, there is a policy $\pi \in CS(\Pi; u)$ that maximises the value of $u(\cdot; \mathbf{w})$. That is, for $CS(\Pi; u)$ to be a coverage set, the following statement must be true:*

$$\begin{aligned} &\forall \mathbf{w} \in \Delta^D .\ \exists \pi \in CS(\Pi; u).\ \forall \pi' \in \Pi : \\ &\quad u(\mathbf{V}^{\pi}(s_0; 0); \mathbf{w}) \geq u(\mathbf{V}^{\pi'}(s_0; 0); \mathbf{w}). \end{aligned} \tag{2.67}$$

In particular, any set $CCS(\Pi)$ is a convex coverage set *if it is a coverage set with respect to the linear utility function u_{lin}.*

A convex coverage set may contain redundant policies and is not necessarily minimal. A minimal convex coverage set is a convex coverage set that contains no redundant policies. That is, for every policy in the minimal convex coverage set, there is some weight for which it uniquely maximises the utility within the set.

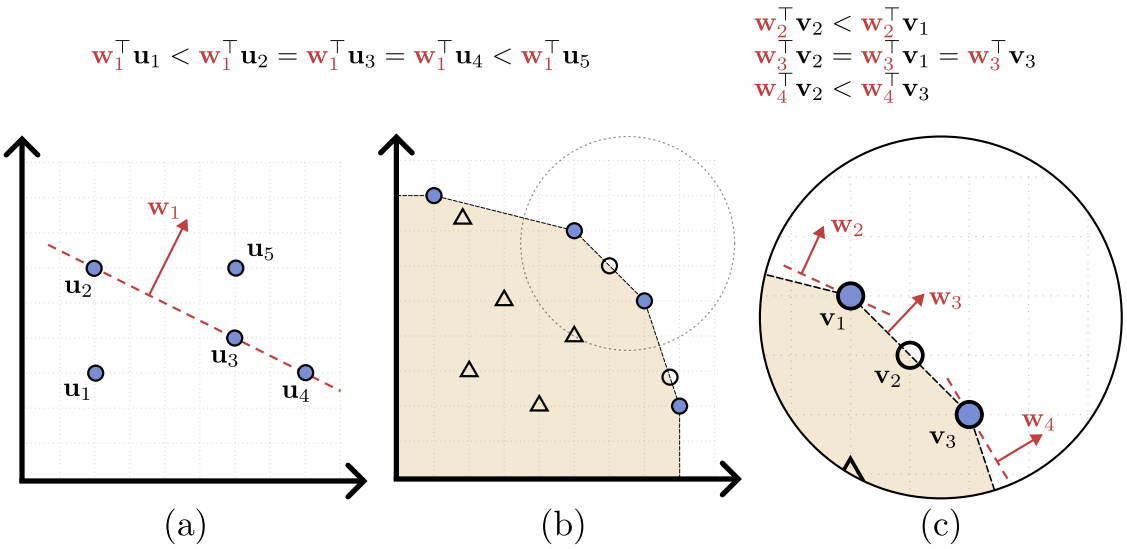


Figure 2.12: The geometry of convex coverage sets, shown with $D = 2$. In each image the points depicted correspond to a value set $\boldsymbol{\mathcal{V}}\mathbf{als}(\Pi')$, for some set of policies Π'. (a) demonstrates that values ($\mathbf{u}_2$,$\mathbf{u}_3$,$\mathbf{u}_4$) obtaining the same utility lie on a hyperplane. Moving in the direction of the weight vector increases the utility ($\mathbf{u}_5$) and likewise moving in the opposite direction decreases utility ($\mathbf{u}_1$); (b) circles correspond to values that are on the convex hull, so form the set $\boldsymbol{\mathcal{V}}\mathbf{als}(CH(\Pi'))$, the blue circles form the minimal convex coverage set, $\boldsymbol{\mathcal{V}}\mathbf{als}(MCCS(\Pi'))$, and triangles are not part of the convex hull. All of the unfilled shapes would be pruned by `cvx_prune` (Equation (**??**)); (c) shows a magnification of (b) with additional labels, it shows that the value $\mathbf{v}_2$ is redundant because one of $\mathbf{v}_1$ or $\mathbf{v}_3$ will achieve an equal or greater utility. Additionally, it shows $\mathbf{w}_2$ and $\mathbf{w}_4$ which are weights that the values $\mathbf{v}_1$ and $\mathbf{v}_3$ uniquely maximise the utility within $\boldsymbol{\mathcal{V}}\mathbf{als}(\Pi')$.

Definition 2.6.9. *A set* $MCCS(\Pi) \subseteq \Pi$ *is a* minimal convex coverage set *if it is a convex coverage set and if the following holds*

$$\forall \pi \in MCCS(\Pi).\ \exists \mathbf{w} \in \Delta^D.\ \forall \pi' \in MCCS(\Pi) - \{\pi\} :$$
$$u_{\text{lin}}(\mathbf{V}^\pi(s_0;0);\mathbf{w}) > u_{\text{lin}}(\mathbf{V}^{\pi'}(s_0;0);\mathbf{w}). \tag{2.68}$$

When computing solution sets, it is often useful to first compute the multi-objective values that could be obtained, and then later use the data structures used by the algorithm to read out the selected policy or solution set (for example, Convex Hull Value Iteration covered in Section **??**). This thesis will refer to the values obtained by a set of policies as a *value set*:

Definition 2.6.10. *The* (multi-objective) value set *with respect to a set of policies* $\Pi' \subseteq \Pi$ *is defined as* $\boldsymbol{\mathcal{V}}\mathbf{als}(\Pi') = \{\mathbf{V}^\pi(s_0;0) | \pi \in \Pi'\}$.

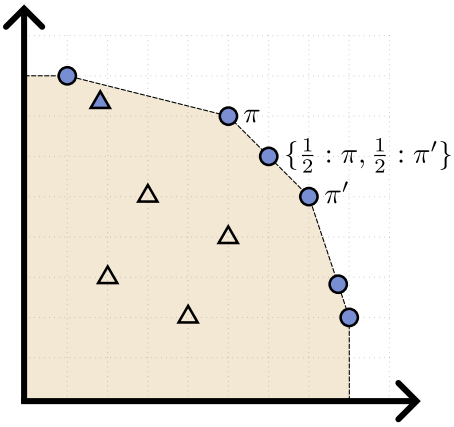


Figure 2.13: An example Pareto front, where the circles correspond to a convex coverage set, and the filled blue points denote the Pareto front. Two points are marked with their corresponding policies π and π', and a mixture policy is shown, denoted $\{\frac{1}{2} : \pi, \frac{1}{2} : \pi'\}$ that follows each of π and π' with probability $\frac{1}{2}$.

Given a value set, the `cvx_prune` operation removes any vectors that are dominated or redundant. That is, for every vector that remains after the pruning operation, there is some weight for which it *uniquely* gives the maximal linear utility. Given a set of vectors $\boldsymbol{\mathcal{V}}$, the `cvx_prune` operation is defined as:

$$\texttt{cvx_prune}(\boldsymbol{\mathcal{V}}) = \{\mathbf{v} \in \boldsymbol{\mathcal{V}} | \exists \mathbf{w} \in \Delta^D . \forall \mathbf{v}' \in \boldsymbol{\mathcal{V}} - \{\mathbf{v}\} . \mathbf{w}^\top \mathbf{v} > \mathbf{w}^\top \mathbf{v}'\}. \tag{2.69}$$

As illustrated in Figure **??**, the `cvx_prune` operation computes the *extreme points* of a *(partial) geometric convex hull.* As such, for some set of policies Π' the `cvx_prune` operation satisfies $\texttt{cvx_prune}(\boldsymbol{\mathcal{V}}\mathbf{als}(\Pi')) = \boldsymbol{\mathcal{V}}\mathbf{als}(MCCS(\Pi'))$, and `cvx_prune` can be implemented using *linear programming* [**?**].

To distinguish between the convex hull of policies, and the geometric convex hulls of values, this thesis will refer to any value set that forms a geometric convex hull as a *convex hull value set* (CHVS).

Definition 2.6.11. *The set of vectors $\boldsymbol{\mathcal{V}} \subseteq \mathbb{R}^D$ is a* convex hull value set *(CHVS) if its points form a partial geometric convex hull. More specifically, $\boldsymbol{\mathcal{V}}$ is a convex hull value set if it satisfies the equation:*

$$\boldsymbol{\mathcal{V}} = \texttt{cvx_prune}(\boldsymbol{\mathcal{V}}). \tag{2.70}$$

Whilst this thesis focuses on the linear utility setting, another commonly used solution set is the *Pareto front*, which is defined using a *strict partial ordering* over vectors, known as *Pareto domination*, as opposed to using utility. An example is shown in Figure **??**.

Definition 2.6.12. *The* Pareto front $PF(\Pi) \subseteq \Pi$ *is defined as:*

$$PF(\Pi) = \{\pi \in \Pi | \nexists \pi' \in \Pi . \mathbf{V}^{\pi'}(s_0; 0) \succ_P \mathbf{V}^{\pi}(s_0; 0)\}, \tag{2.71}$$

where the Pareto domination *relation is defined as:*

$$\mathbf{v} \succ_P \mathbf{v}' \iff (\forall i . v_i \geq v'_i) \wedge (\exists j . v_j > v'_j). \tag{2.72}$$

The Pareto front can be related to the previously defined solution sets if *mixture policies* are allowed, where a mixture policy between π and π' will follow each policy with some probability. Specifically, if Π is closed with respect to mixture policies, then the Pareto front is a convex coverage set.

2.6.1 Convex Hull Value Iteration

Convex Hull Value Iteration (CHVI) [**?**] is a tabular dynamic programming algorithm similar to Value Iteration (Section **??**) that computes optimal convex hull value sets and provides a method for extracting policies for any given weight vector. In CHVI the value estimates of Value Iteration are replaced by CHVSs, which are estimates of the optimal CHVS. These estimates are denoted $\hat{\boldsymbol{\mathcal{V}}}_{\text{CHVI}}(s; t)$ and $\hat{\boldsymbol{\mathcal{Q}}}_{\text{CHVI}}(s, a; t)$.

Firstly, to define a multi-objective version of Value Iteration, arithmetic over sets of vectors needs to be defined. An example of the following arithmetic is given in Figure **??**. Given the vector sets $\boldsymbol{\mathcal{U}}$ and $\boldsymbol{\mathcal{V}}$, define multiplication by a scalar s, addition with a vector $\mathbf{x}$ and addition between sets as follows:

$$\mathbf{x} + s\boldsymbol{\mathcal{V}} = \{\mathbf{x} + s\mathbf{v} | \mathbf{v} \in \boldsymbol{\mathcal{V}}\} \tag{2.73}$$

$$\boldsymbol{\mathcal{U}} + \boldsymbol{\mathcal{V}} = \{\mathbf{u} + \mathbf{v} | \mathbf{u} \in \boldsymbol{\mathcal{U}}, \mathbf{v} \in \boldsymbol{\mathcal{V}}\}. \tag{2.74}$$

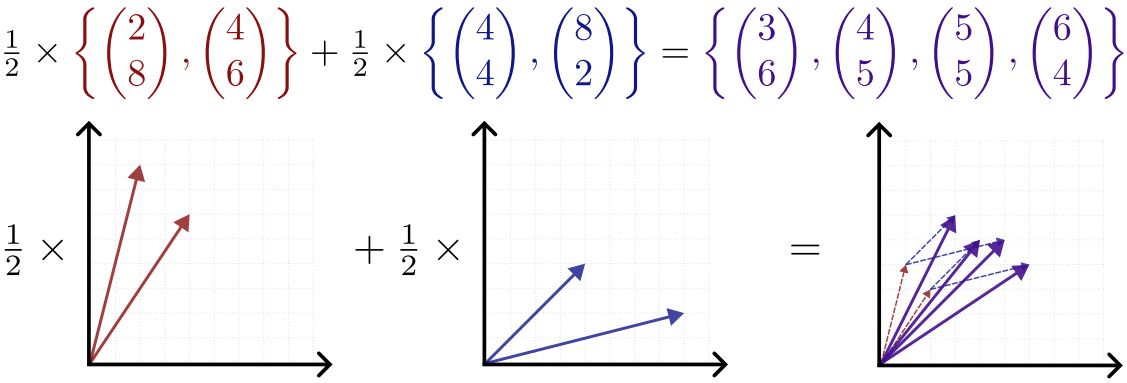


Figure 2.14: An example of arithmetic over vector sets. Two vector sets are shown in red and blue, which are multiplied by $\frac{1}{2}$ using Equation (**??**), and then added using Equation (**??**). On the right the resulting vector set is shown in purple, which are the sum of appropriately scaled red and blue vectors.

Now to define the multi-objective Bellman backups used in CHVI, let $\hat{\boldsymbol{\mathcal{V}}}^0_{\text{CHVI}}(s;t) = \{\mathbf{0}\}$, where $\mathbf{0} = (0, \ldots, 0) \in \mathbb{R}^D$. The CHVI backups are then:

$$\hat{\boldsymbol{\mathcal{V}}}^{k+1}_{\text{CHVI}}(s;t) = \texttt{cvx_prune}\left(\bigcup_{a \in \mathcal{A}} \hat{\boldsymbol{\mathcal{Q}}}^{k+1}_{\text{CHVI}}(s,a;t) \right), \tag{2.75}$$

$$\hat{\boldsymbol{\mathcal{Q}}}^{k+1}_{\text{CHVI}}(s,a;t) = \mathbb{E}_{s' \sim p(\cdot|s,a)}[\mathbf{R}(s,a) + \hat{\boldsymbol{\mathcal{V}}}^{k}_{\text{CHVI}}(s';t+1)]. \tag{2.76}$$

This again parallels the Bellman backups used in single-objective Value Iteration, where the max operation is replaced by the `cvx_prune` operation over the set of achievable values from the current state s.

Once the algorithm has terminated and a weight $\mathbf{w}$ is provided, a policy can be extracted by computing scalar Q-values from the CHVSs [**?**]:

$$Q^{\mathbf{w}}_{\text{CHVI}}(s,a;t) = \max\{\mathbf{w}^\top \mathbf{q} | \mathbf{q} \in \hat{\boldsymbol{\mathcal{Q}}}_{\text{CHVI}}(s,a;t)\}, \tag{2.77}$$

$$\pi_{\text{CHVI}}(s;t,\mathbf{w}) = \arg\max_{a \in \mathcal{A}} Q^{\mathbf{w}}_{\text{CHVI}}(s,a;t). \tag{2.78}$$

Note that if a policy is extracted for every weight vector $\mathbf{w} \in \Delta^D$, the resulting set of policies forms a convex coverage set.

3
Literature Review

Contents

3.1 Multi-Armed Bandits **43**
3.2 Reinforcement Learning **44**
3.3 Trial-Based Heuristic Tree Search and Monte-Carlo Tree Search **46**
3.3.1 Trial Based Heuristic Tree Search 46
3.3.2 Monte-Carlo Tree Search 46
3.3.3 Maximum Entropy Tree Search 47
3.4 Multi-Objective Reinforcement Learning **47**
3.5 Multi-Objective Monte Carlo Tree Search **49**

TODO: should add some boltzmann search stuff here too

TODO: Change chapter title to related work rather than literature review

TODO: currently this is a copy and paste of what I originally wrote for background chapter **??**. Deleted parts which are irrelevant for litreview here (and vice versa for the background section).

TODO: I'm also going to use this as a space to paste papers I should write about as they come up while writing later chapters

3.1 Multi-Armed Bandits

TODO: Maybe dont need to cover this in litrev, but should talk about exploring

bandits, UCT and contextual bandits either in background or in litrev

TODO: linUCB for contextual bandits think this is the linucb paper: https://arxiv.org/pdf/1003.0146

Designing multi-objective multi-armed bandits algorithms: a study - Madalina M. Drugan and Ann Nowe

TODO: MAB book

Talk about Exp3?

3.2 Reinforcement Learning

TODO: Intro should say that look at Sutton and Barto and something else for deep RL, for a more complete overview. Here we will just discuss papers that consider entropy in their work, as thats the most relevant part for this thesis.

TODO: list

- Talk about entropy and some of that work (probably a subsection)

TODO: In the entropy bit talk add this, removed from ch2: Note that there are other forms of entropy, such as relative and Tsallis entropy, which can be used in place of Shannon entropy (TODO cite). For the work considered in this thesis, the other forms of entropy can be used by replacing the definition of $\mathcal{H}$ by the relevant definition.

TODO: talk about some deep learning methods here

TODO: Talk about entropy in (deep) RL methods too. Say that it is primarily introduced primarily for an exploration benifit. A3C "prevent converging to deterministic suboptimal behaviour". While there is some additional. Intro in the soft Q learning provides some alternative motivations, "In some cases, we might actually prefer to learn stochastic behaviors. In this paper, we explore two potential reasons for this: exploration in the presence of multimodal objectives, and compositionality attained via pretraining. Otherbenefits include robustness in the face of uncertain dynamics (Ziebart, 2010), imitation learning (Ziebart et al., 2008), and improved convergence and computational properties (Gu et al., 2016a). Multi-modality also has application in real robot tasks, as demonstrated in (Daniel

et al., 2012). However, in order to learn such policies, we must define an objective that promotes stochasticity.". ANd also talks about adversarital pertobations. So basically: good for meta learning/pretraining/foundation models, good for robustness against uncertain dynamics. BASICALLY, HERE we want to talk about the other reasons why we might want to use maximum entropy.

Entropy widely used in RL literature. Most commonly used for regularization when learning a policy [? ? ?]. This is significantly different to the maximum entropy setting, where entropy is used in the loss function to prevent the policy from degenerating into a deterministic policy. While it is motivated similarly, promoting the policy to remain stochastic and enable exploration.

The significant difference between *entropy regularization* and *maximum entropy reinforcement learning* is that the former is that from some state s_t, regularization only includes the entropy of the policy at s_t, while the latter includes the entropy of the policy at all states $s_t, s_{t+1}, ...$ in the objective/loss function.

Probably write this a bit better, say that entropy regularization dates back to the REINFORCE/MENT paper, but is consistently used to the modern day.

Old entropy stuff from ch4 before rewrite: Loosely, the maximum entropy principle says that given partial knowledge about a probability distribution, the least biased estimate of the probability distribution is the one that maximises the entropy [?]. This principle can naturally be applied to inverse reinforcement learning [? ?], where the goal is to learn the transition/reward functions from a set of expert trajectories.

In contrast, in maximum entropy reinforcement learning (Section ??) entropy is added directly into the objective function to be optimised, rather than using entropy as a constraint to disambiguate between multiple optimal policies. While the maximum entropy objective does yield a unique optimal policy (with respect to the maximum entropy objective), the policy is not guaranteed to align with the standard objective.

3.3 Trial-Based Heuristic Tree Search and Monte-Carlo Tree Search

TODO: Rename this subsection to just Trial Based Search I think makes sense

TODO: talk about UCT, and multiarmed bandits. Specifically that the theoretical lower bound is log T and UCT achieves log T lower bound. Define cumulative regret here.

3.3.1 Trial Based Heuristic Tree Search

TODO: THTS paper, talk about the differences that the paper has to our presentation of `THTS++`

TODO: cut from ch2: Finally we will briefly point out the differences between `THTS++`and the original THTS schema in subsection

TODO: talk about how these methods are still relevant with deep learning because of algorithms that use both, such as alpha zero

3.3.2 Monte-Carlo Tree Search

TODO: list

- Talk about the things that are ambiguous from literature (e.g. people will just say UCT, which originally presented doesn't run in `single_node_expansion`, but often assumed it does)
- Should talk about multi-armed bandits here?

https://inria.hal.science/inria-00164003/document

https://pdf.sciencedirectassets.com/271585/1-s2.0-S0004370211X0005X/1-s2.0-S000437021100052X/main.pdf?X-Amz-Security-Token=IQoJb3JpZ2luX2VjEOP

TODO: removed from ch2, this can be litrev: add polynomial UCT here? and or prioritised UCT from alpha go here?

TODO: removed from ch2: add stuff about regret here, give the O(logn) bound for UCT, and also talk about the papers that

TODO: removed from ch4: H-MCTS algorithm <TODO: cite>which combines UCT and Sequential Halving. BD: H-MCTS is used as an UCT style MCTS algorithm that is also designed using simple regret for comparison. (this was in intro to section 4.4, DENTS results section)

TODO: Should make sure to talk about all of the MCTS algorithms designed around simple regret. Confirm the claim that they all are really just some mix of uniform and UCT

TODO: should talk about MCTS methods that use simple regret, specifically H-MCTS and probably BRUE etc

TODO: Also talk about Trial-Based RTDP here

TODO: introduce RENTS and TENTS outside of the introduction Entropy is widely used in the reinforcement learning literature, commonly introduced to promote exploration and discourage convergence to suboptimal deterministic policies [? ? ? ?]. MENTS (Section ??), RENTS and TENTS (Section ??) are all MCTS algoritms that optimise for maximum entropy objectives. While RENTS and TENTS will be considered as baselines in empirical experiments, MENTS will be used to facilitate and provide discussion around the use of entropy and maximum entropy objective.

TODO: make sure H-MCTS, (BRUE), RENTS and TENTS baselines all introduced in related work section here.

3.3.3 Maximum Entropy Tree Search

TODO: MENTS, RENTS and TENTS

3.4 Multi-Objective Reinforcement Learning

TODO: list

- Should talk about multi-objective and/or contextual multi-armed bandits here?
- Bunch of the work covered in recent MORL survey [?]

- Mention some deep MORL stuff, say that this work (given AlphaZero) is adjacent work

TODO: removed from ch2: comment here or in literature review about there being more types of scalariation function that arent necessarily weighted by a weight, and ESR vs SER stuff

TODO: talk about more of the MORL survey, including some of the other motivating scenarios

TODO: Talk about the better way of doing CHVI? https://www.jmlr.org/papers/volume13/lizotte12a and Efficient reinforcement learning with multiple reward functions for randomized controlled trial analysis. Also by Lizotte. But also say that its approximate for $D > 2$ and so we stick with the slower one? But the CHVS operations can be computed more efficiently in $D = 2$ using this stuff. Did it for the python implementation.

TODO: also need to talk about the MO sequential decision survey. Define Pareto Front. Say that the Pareto Front "Definition 3 If the utility function u is any monotonically increasing function, then the Pareto Front (PF) is the undominated set [Roijers et al., 2013]:"

TODO: need to talk about this because some prior/related work uses PF rather than CH

TODO: cover some inner loop and outer loop things? Roijers computing CCS work? See what MORL survey says about it

TODO: Maybe talk about the *witness algorithm* for Partially Observable MDPs as its very similar to CHVI stuff

Ann Nowe papers:

https://www.jmlr.org/papers/volume15/vanmoffaert14a/vanmoffaert14a.pdf

https://arxiv.org/pdf/2402.07182

https://www.ifaamas.org/Proceedings/aamas2024/pdfs/p1611.pdf

TODO: Go through Roijers and Ann Nowe and Mykel Kochenderfer

TODO: Deep MORL things in MO-gymnasium

TODO: Some of the outer and inner loop things - prism, computing convex coverage sets

3.5 Multi-Objective Monte Carlo Tree Search

TODO: I think this whole section can just go in litrev

TODO: list

- Define the old methods (using the CH object methods, so clear that not doing direct arithmetic)
- Mention that old method could be written using the arithmetic of CHMCTS (but they don't)
- Different flavours copy UCT action selection, but with different variants
- Link back to contributions and front load our results showing that all of the old methods don't explore correctly

TODO: There has been some prior work in multi-objective MCTS which we will outline here

TODO: Write out implementations of prior works using THTS

TODO: define pareto front

TODO: perez algorithms

TODO: https://ieeexplore.ieee.org/document/8107102

TODO: https://www.roboticsproceedings.org/rss15/p72.pdf TODO: https://arxiv.org/abs/211

TODO: https://proceedings.mlr.press/v25/wang12b/wang12b.pdf

TODO: https://ifmas.csc.liv.ac.uk/Proceedings/aamas2021/pdfs/p1530.pdf TODO: https://link.springer.com/article/10.1007/s10458-022-09596-0

TODO: Some stuff we wrote that didnt use in ch2. Might want to talk about it with eval:

Moreover, in the case of MCTS algorithms, by having the user select a preferred policy, it implicitly forces the user to chose a preference over the objectives, as the policy corresponds to a weight vector that it is optimal for. As MCTS algorithms are often used in an online fashion, where planning is interleaved with execution, this implicitly selected weight can be used for any online execution needed, effectively reducing the multi-objective problem into a single-objective problem.

4

Monte Carlo Tree Search With Boltzmann Exploration

Contents

4.1 Introduction and Motivation 53
4.1.1 UCT 54
4.1.2 MENTS 55
4.1.3 Simple Regret and Consistency 57
4.2 Boltzmann Search 58
4.2.1 Boltzmann Tree Search 59
4.2.2 Decaying ENtropy Tree Search 62
4.2.3 Stochastic Search Policies: Prior Knowledge and Computational Efficiency 64
4.3 Theoretical MDPs 69
4.3.1 The D-Chain Problem 70
4.3.2 The Entropy Trap 72
4.4 Empirical Results 74
4.4.1 Environments 75
4.4.2 Evaluation Proceedure 78
4.4.3 Results and Discussion 80
4.5 Theoretical Results 85
4.5.1 Notation 86
4.5.2 Preliminaries 88
4.5.3 Consistency Results 90
4.5.4 Entropy Trap 98
4.6 Summary 102

This chapter considers MCTS algorithms for planning in single-objective environ-

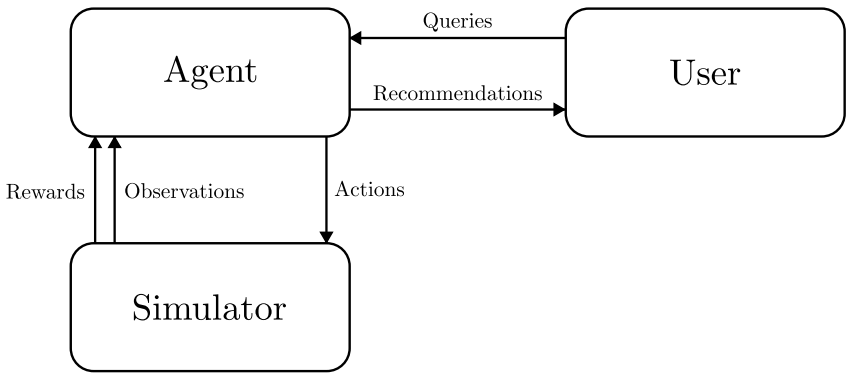


Figure 4.1: Exploration setting in reinforcement learning (edited from Figure **??**), where the agent is only assessed on the recommendations that it provides, and is not penalised for considering poor actions in the simulator. This setting therefore places greater emphasis on exploration than exploitation.

ments, with the optional use of entropy as a secondary objective for exploration. The discussion focuses on the exploration setting for reinforcement learning (see Section **??** and Figure **??**), where the agent is assessed purely on the recommendations it makes, rather than what actions it explored in simulation.

In maximum entropy reinforcement learning (Section **??**), entropy is added directly to the objective function rather than used as a constraint to remove ambiguity between multiple optimal policies, as in inverse reinforcement learning [**? ?**]. Whilst the maximum entropy objective yields a unique optimal policy, it is not guaranteed to align with the standard objective. This chapter demonstrates that the temperature parameter may have to be set to a prohibitively small value for the objectives to align, which in practice can negate the benefit of entropy-based exploration.

This motivates the investigation of **Q1.1 - Entropy**in this chapter, and lays the foundations for **Q1.2 - Multi-Objective Exploration** in Chapter **??**.

The remainder of this chapter is organised as follows:

- Section **??** discusses limitations of existing MCTS algorithms in the exploration setting of reinforcement learning. *Simple regret* is defined in the context of sequential decision making, to theoretically analyse the performance of MCTS algorithms in the exploration setting.

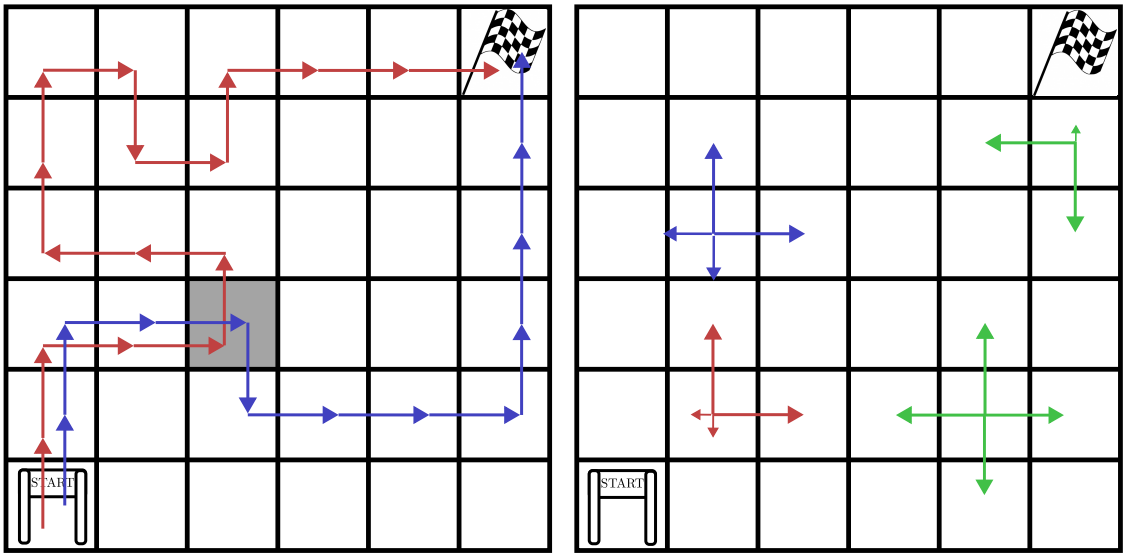
- Section **??** defines the *Boltzmann Tree Search* (BTS) and *Decaying ENtropy Tree Search* (DENTS) algorithms.
- Section **??** discusses some properties of using a stochastic search policy in MCTS, and examines how the *Alias Method* can be used to address **Q2.1 - Complexity**.
- Section **??** introduces two theoretical MDPs, which are used to empirically demonstrate the limitations of the existing MCTS algorithms.
- Section **??** presents experimental results on grid world environments and the game of Go, and gives an empirical answer to **Q1.1 - Entropy**.
- Section **??** presents the main theoretical analysis and proofs. Convergence properties of MENTS, BTS and DENTS are proven to provide a theoretical answer to **Q1.1 - Entropy**.

4.1 Introduction and Motivation

This section considers a grid world shortest path problem, which is used to discuss the behaviour of UCT and MENTS in the exploration setting, and highlight some limitations of these algorithms.

In this grid world, the agent starts at the origin $(0, 0)$, and must reach the goal state at $(6, 6)$. Each timestep, the agent may move deterministically in any cardinal direction, $\{\texttt{UP}, \texttt{RIGHT}, \texttt{DOWN}, \texttt{LEFT}\}$, as long as it remains in the grid. Each action incurs a cost of 1 (or a reward of -1), so the cumulative cost of a path is equal to its length. The time horizon is chosen to be 36 (six times the width of the grid) so that the horizon is long enough for exploration. The optimal policy will always select from the actions `UP` and `RIGHT`. The problem, with example paths, is illustrated in Figure **??**.

While UCT and MENTS can easily solve this problem given enough trials and appropriately set parameters, it is used with a limited number of trials to illustrate the behaviour of the algorithms in the exploration setting. In Section **??**, MDPs are



(a) Some possible paths from start to finish.

(b) Optimal policy with the maximum entropy objective for varying temperatures.

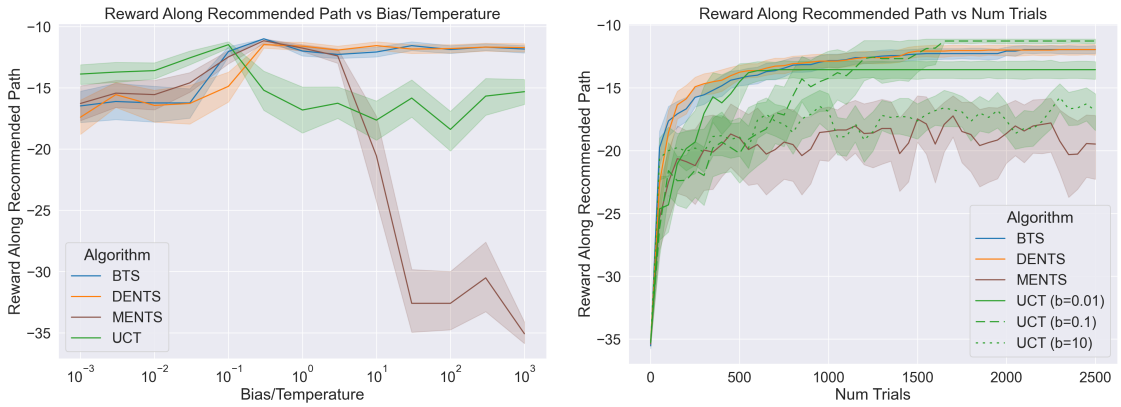
Figure 4.2: An example grid world shortest path problem. (a) Shows two paths that UCT may have explored, where the blue path is shorter, but the red path takes one of the optimal actions at the (shaded) state $(2, 2)$. (b) Illustrates optimal probability distributions at four states in the maximum entropy objective, with varying temperatures. In red, the temperature is low and in blue the temperature is larger. In green, the temperature is large so that gathering entropy "reward" outweighs reaching the goal.

constructed adversarially to expose limitations of UCT and MENTS, and Section **??** considers more complex grid worlds — including terminal states, sparse rewards and stochastic transition distributions — that are harder to solve in practice.

4.1.1 UCT

The UCT algorithm (Section **??**) is designed in the traditional reinforcement learning setting described in Section **??**, where the UCT agent aims to minimise the cumulative regret. Thus UCT makes a trade-off between exploration and exploitation during its trials, and frequently chooses the same action to exploit, which can result in it getting stuck in local optima, particularly when rewards are sparse or not informative.

A *sparse reward* is where the reward signal is infrequent, that is, most actions give a reward of zero. In the shortest path example, the reward is dense but uninformative, as the immediate cost of taking any action is the same.



(a) Reward of best path, after 2500 trials, for a range of bias/temperature parameters.

(b) Reward of best path relative to the number of trials. ($\alpha = 1$ and $b_{\text{PUCT}} = 0.01, 0.1, 1$).

Figure 4.3: Results on the example grid world problem for a variety of bias and temperature parameters. For comparison, the Boltzmann Tree Search (BTS) and Decaying ENtropy Tree Search (DENTS) algorithms defined in Section **??** are also shown. (a) Each algorithm is run for 2500 trials for a range of bias and temperature parameters, and (b) shows the evolution of the reward of the recommended best path as a function of the number of trials. In (a) and (b) each data point is averaged over 25 repeated experiments.

Suppose that UCT has been run for many trials, and the best paths found by UCT so far are shown in red and blue in Figure **??**. The two paths coincide until the shaded state at $(2, 2)$, where UCT has the Q-value estimates $\bar{Q}_{\text{UCT}}((2, 2), \texttt{DOWN}) \approx -8$ and $\bar{Q}_{\text{UCT}}((2, 2), \texttt{UP}) \approx -12$. When a UCT trial reaches the shaded state at $(2, 2)$, it will frequently select the action `DOWN` over `UP` to exploit the best path it has found so far, rather than exploring the optimal actions `UP` and `RIGHT`.

In theory, UCT selects every action infinitely often (that is, $N(s, a) \to \infty$ for all reachable s, a) and will converge to the optimal values and policy. In practice, UCT shifts its focus from exploration to exploitation throughout the search, which is determined by the confidence intervals and the bias parameter b_{UCT} from Equation (**??**). This is illustrated in Figure **??**, where for small bias parameters UCT quickly exploits a suboptimal solution and for large bias parameters UCT is still exploring and its value estimates have not yet converged.

4.1.2 MENTS

One can argue that when using the maximum entropy objective, it is possible to recover the standard objective by setting the temperature parameter to zero, or an

infinitesimally small value. However, in practice, setting α to a tiny value will nullify the exploration advantages of using entropy. Considering the other extreme, when a very large temperature is used, entropy becomes the dominant objective in the maximum entropy objective, and the optimal soft policy will be (almost) uniform.

The optimal soft policy for a large temperature may be different from the optimal policy for the standard objective. In cases such as this, it can be said that the maximum entropy objective is *misaligned* with the standard objective.

Definition 4.1.1. *The policy induced by the optimal soft Q-values is* $\psi^*_{\text{sft}}(s) = \arg\max_{a'} Q^*_{\text{sft}}(s, a')$. *The maximum entropy objective is* misaligned *when the policy* $\psi^*_{\text{sft}}(s) \neq \pi^*(s)$, *where* π^* *is the optimal standard policy (Definition* **??***). The* misalignment threshold *is the smallest temperature parameter at which the maximum entropy objective is misaligned.*

In Figure **??**, the optimal soft policy for the maximum entropy objective is shown for varying temperature parameters. For small temperature values ($\alpha < 1$, illustrated in red), the optimal soft policy is close to the optimal standard policy, and for large temperature values ($\alpha > 100$, illustrated in green) the optimal soft policy is nearly uniform, except at states adjacent to the goal, where the agent will avoid reaching the goal to obtain more entropy "reward".

When using the maximum entropy objective, the temperature parameter often needs to be carefully tuned, large enough to gain an exploration benefit, but small enough to avoid misalignment issues. In this example problem, the misalignment threshold is around $\alpha = 10$, as can be seen in Figure **??**. Moreover, because the entropy of a policy depends on the MDP, the temperature typically needs to be tuned for each MDP. In Section **??**, an MDP is constructed where the misalignment threshold is prohibitively small for the maximum entropy objective to be effective, and Section **??** considers more complex grid worlds where the misalignment threshold is low enough to significantly affect performance.

Parameters: An MDP $\mathcal{M}$.

- For each round $m = 1, 2, \ldots$:
  1. the agent produces a search policy π^m to follow;
  2. the environment samples a trajectory $\tau \sim \pi^m$ (including rewards $r_t = R(s_t, a_t)$ for each s_t, a_t pair in τ);
  3. the agent produces a recommendation policy ψ^m;
  4. if the environment sends a stop signal, then the game ends, otherwise the next round starts.

Figure 4.4: The procedure of an exploration planning problem for MDPs, where ψ^m is the recommendation policy the agent produces after m trajectories are sampled using the exploration policy π^m.

4.1.3 Simple Regret and Consistency

This section generalises the concept of simple regret (Definition **??**) to MDPs and introduces the concept of consistency for MCTS algorithms. *Simple regret* is used to motivate and analyse the algorithms introduced in this chapter. *Consistency* refers to the correctness of an MCTS algorithm in the limit — informally, a consistent algorithm will eventually find the optimal policy given enough trials. The term originates from the MCTS literature, to describe whether modifications to UCT preserve its asymptotic behaviour [**?** **?** **?**].

The *simple regret* of a policy is the difference between the value of the policy and optimal value of the policy (Equation (**??**)). Whilst *cumulative regret* is used to analyse algorithms in the standard reinforcement learning setting, simple regret is more appropriate for the exploration setting, where the agent is assessed only on its recommendations and not penalised for exploring poor actions in simulation.

Definition 4.1.2. *The* simple regret *of a policy ψ at state $s \in \mathcal{S}$ is the difference between the value of the policy and the optimal value at that state:*

$$\texttt{sim_regr}(s, \psi) = V^*(s) - V^{\psi}(s). \tag{4.1}$$

TODO: make sure simple regret is in the table of notation

By definition, the optimal policy achieves a simple regret of zero. In this thesis an MCTS algorithm is considered *consistent* if its expected simple regret tends to zero. Informally, a consistent algorithm will eventually output an optimal policy if run indefinitely, and is expected to improve its recommendation with more trials.

Definition 4.1.3. *An agent, that produces recommendation policies* $\psi^1, \psi^2, \ldots$ *is said to be* consistent *if* $\mathbb{E}[\texttt{sim_regr}(s, \psi^m)] \to 0$ *as* $m \to \infty$.

Note that the simple regret (Equation (**??**)) is a random variable, as it depends on the recommendation policy ψ, which itself depends on the random trajectories that are sampled. Hence the expected simple regret is used in Definition **??** to define consistency, and is the main value of interest in the theoretical analysis in Section **??**.

?] show that upper bounds on cumulative regret lead to both upper and lower bounds on simple regret. Returning to the discussion around UCT and MENTS, using this result, UCT is consistent with respect to Definition **??** because its upper bound on cumulative regret [**?**] leads to an upper bound on simple regret that tends to zero. However, MENTS is only consistent for a sufficiently small temperature parameter that depends on the MDP (TODO: ref section and/or theorem later in chapter).

4.2 Boltzmann Search

This section defines the new algorithms Boltzmann Tree Search (BTS) and Decaying ENtropy Tree Search (DENTS). Following the discussion from Section **??**, the design aims of these algorithms are as follows:

- to be as simple to define and implement as UCT and MENTS[1];
- to be able to explore effectively when rewards are sparse or uninformative;
- to be able to exploit when necessary, in large or stochastic domains;
- to be consistent using parameters that do not depend on the MDP.

[1] i.e. each decision node operates on a multi-armed bandit problem

In Section **??** BTS is defined, which uses a Boltzmann search policy, similar to MENTS, but using the standard objective and Bellman backups rather than the soft Bellman backups used in MENTS. Section **??** defines DENTS which re-introduces entropy as an exploration bonus while maintaining consistency. Section **??** discusses the use of stochastic search policies in MCTS further, highlighting that mixture policies can be used to incorporate prior knowledge from $\pi_{\text{heuristic}}$ (Section **??**) and how the *Alias Method* can be used to arrive at a more efficient MCTS algorithm addressing **Q2.1 - Complexity** (Section **??**).

4.2.1 Boltzmann Tree Search

Algorithm Definition The *Boltzmann Tree Search* (BTS) algorithm uses the variables $\hat{V}_{\text{BTS}}$ and $\hat{Q}_{\text{BTS}}$ for value estimates, $N(s)$ and $N(s,a)$ for visit counts, and the function $\alpha_{\text{BTS}}(x) > 0$ for the search temperature. The temperature used at each decision node s is computed with respect to the visit count $N(s)$ as $\alpha_{\text{BTS}}(N(s))$.

The BTS search policy uses the flag `normalised_Q` to optionally normalise Q-values, and incorporates prior knowledge from $\hat{Q}_{\text{heuristic}}$ weighted by $N_{\text{BTS}}^{\hat{Q}_{\text{heuristic}}}$ pseudo-visits:

$$\hat{Q}_{\text{BTS}}^{\text{bias}}(s,a) = \frac{N(s,a)\hat{Q}_{\text{BTS}}(s,a) + N_{\text{BTS}}^{\hat{Q}_{\text{heuristic}}}\hat{Q}_{\text{heuristic}}(s,a)}{N(s,a) + N_{\text{BTS}}^{\hat{Q}_{\text{heuristic}}}}, \tag{4.2}$$

$$\hat{Q}_{\text{BTS}}^{\text{norm}}(s,a) = \begin{cases} \frac{\hat{Q}_{\text{BTS}}^{\text{bias}}(s,a) - \min_{a\in\mathcal{A}} \hat{Q}_{\text{BTS}}^{\text{bias}}(s,a)}{\max_{a\in\mathcal{A}} \hat{Q}_{\text{BTS}}^{\text{bias}}(s,a) - \min_{a\in\mathcal{A}} \hat{Q}_{\text{BTS}}^{\text{bias}}(s,a)} & \text{if } \texttt{normalised_Q}, \\ \hat{Q}_{\text{BTS}}^{\text{bias}}(s,a) & \text{otherwise.} \end{cases} \tag{4.3}$$

The search policy of BTS is similar to MENTS (Section **??**), but using a variable temperature and (the normalised and biased) Bellman value estimates rather than soft value estimates.

$$\pi_{\text{BTS}}(a|s) = (1 - \lambda(s, \epsilon_{\text{BTS}}))\rho_{\text{BTS}}(a|s) + \frac{\lambda(s, \epsilon_{\text{BTS}})}{|\mathcal{A}|}, \tag{4.4}$$

$$\rho_{\text{BTS}}(a|s) \propto \exp\left(\frac{\hat{Q}_{\text{BTS}}^{\text{norm}}(s,a)}{\alpha_{\text{BTS}}(N(s))}\right), \tag{4.5}$$

$$\lambda(s,\epsilon) = \min\left(1, \frac{\epsilon}{\log(e + N(s))}\right), \tag{4.6}$$

where $\epsilon_{\text{BTS}} \in [0, \infty)$ is an exploration parameter.

For any new decision nodes s added to the tree, the `init_v` function initialises the visit count to $N(s) = 0$ and the value estimate to $\hat{V}_{\text{BTS}}(s) = \hat{V}_{\text{heuristic}}(s)$. For any new chance nodes (s, a) added to the tree, the `init_q` function initialises the visit count to $N(s, a) = 0$ and the Q-value estimate to $\hat{Q}_{\text{BTS}}(s, a) = \hat{Q}_{\text{heuristic}}(s, a)$.

BTS also allows the value estimates to be biased by $\hat{V}_{\text{heuristic}}$ with a weighting of $N_{\text{BTS}}^{\hat{V}_{\text{heuristic}}}$ pseudo-visits. When `single_node_expansion` is True, $N_{\text{BTS}}^{\hat{V}_{\text{heuristic}}}$ is typically set to one, weighting the heuristic value estimate as a single pseudo-trial. When `single_node_expansion` is False, $N_{\text{BTS}}^{\hat{V}_{\text{heuristic}}}$ is set to zero, as the trajectory already captures the rollout information.

Let $\tau_{:h} = (s_0, a_0, r_0, \ldots, s_{h-1}, a_{h-1}, r_{h-1}, s_h) \sim \pi_{\text{BTS}}$ be the trajectory sampled during the selection phase. The `backup_v` and `backup_q` functions then update the value estimates for $t = h - 1, \ldots, 0$:

$$\hat{Q}_{\text{BTS}}(s_t, a_t) \leftarrow R(s_t, a_t) + \sum_{s' \in \text{Succ}(s_t, a_t)} \left(\frac{N(s')}{N(s_t, a_t)} \hat{V}_{\text{BTS}}(s') \right), \tag{4.7}$$

$$\hat{V}_{\text{BTS}}(s_t) \leftarrow \frac{N_{\text{BTS}}^{\hat{V}_{\text{heuristic}}} \hat{V}_{\text{heuristic}}(s_t) + N(s_t) \max_{a \in \mathcal{A}} \hat{Q}_{\text{BTS}}(s_t, a)}{N(s_t) + N_{\text{BTS}}^{\hat{V}_{\text{heuristic}}}}. \tag{4.8}$$

When BTS needs to recommend a policy, it can use its Q-value estimates, or the visit counts:

$$\psi_{\text{BTS}}(s) = \arg\max_{a \in \mathcal{A}} \hat{Q}_{\text{BTS}}(s, a), \tag{4.9}$$

$$\texttt{mv}_{\text{BTS}}(s) = \arg\max_{a \in \mathcal{A}} N(s, a). \tag{4.10}$$

Average Returns The BTS search policy can still be used with average returns. In *Boltzmann Tree Search with Average Returns* (AR-BTS) the Bellman value estimates are replaced with average returns $\bar{V}_{\text{AR-BTS}}$ and $\bar{Q}_{\text{AR-BTS}}$. Given a trajectory $\tau = (s_0, a_0, r_0, \ldots, s_{h-1}, a_{h-1}, r_{h-1}, s_h)$ the average returns are updated for $t = h - 1, \ldots, 0$:

$$\bar{Q}_{\text{AR-BTS}}(s_t, a_t) \leftarrow \frac{N(s_t, a_t) - 1}{N(s_t, a_t)} \left(\bar{Q}_{\text{AR-BTS}}(s_t, a_t) + \frac{\hat{V}_{\text{h}}(s_h) + \sum_{i=t}^{h-1} r_i}{N(s_t, a_t) - 1} \right), \tag{4.11}$$

$$\bar{V}_{\text{AR-BTS}}(s_t) \leftarrow \frac{N(s_t) - 1}{N(s_t)} \left(\bar{V}_{\text{AR-BTS}}(s_t) + \frac{\hat{V}_{\text{h}}(s_h) + \sum_{i=t}^{h-1} r_i}{N(s_t) - 1} \right), \tag{4.12}$$

where $\hat{V}_{\text{h}}$ is a shorthand for $\hat{V}_{\text{heuristic}}$.

The corresponding definitions of $\bar{Q}^{\text{norm}}_{\text{AR-BTS}}$, $\pi_{\text{AR-BTS}}$, $\psi_{\text{AR-BTS}}$, $\texttt{mv}_{\text{BTS}}$ and so on are similar to the definitions for BTS and are omitted for brevity. The complete equations for all algorithms can be found in Appendix **??** TODO: double check this ref.

Summary of Theoretical Results In Section **??** it is shown that the BTS recommendation policy will eventually converge to the optimal policy.

Theorem 4.2.1. *For any MDP $\mathcal{M}$, if $\epsilon_{\text{BTS}} > 0$, the expected simple regret of ψ_{BTS} tends to zero: $\mathbb{E}[\texttt{sim_regr}(s_0, \psi_{\text{BTS}})] \to 0$.*

Additionally, if the temperature function is allowed to tend to zero, then the search policy will also converge to the optimal policy, and the most visited recommendation policy will also converge to the optimal policy as a result.

Theorem 4.2.2. *For any MDP $\mathcal{M}$, if $\epsilon_{\text{BTS}} > 0$ and $\alpha_{\text{BTS}}(x) \to 0$, then $\pi_{\text{BTS}} \xrightarrow{p} \pi^*$ and $\mathbb{E}[\texttt{sim_regr}(s_0, \texttt{mv}_{\text{BTS}})] \to 0$.*

Conversely, if the temperature has a strictly positive lower bound (and does not converge to zero), then the BTS recommendation policy will converge to the optimal policy exponentially fast. However, note that neither the search policy nor the most-visited recommendation policy is guaranteed to converge to the optimal policy in this case.

Theorem 4.2.3. *For any MDP $\mathcal{M}$, if there exists $\alpha_{\text{BTS}}(x) \geq L > 0$, then there exist constants $C, k > 0$ such that $\mathbb{E}[\texttt{sim_regr}(s_0, \psi_{\text{BTS}})] \leq C \exp(-kn)$ after running n trials of BTS.*

For AR-BTS, the temperature function always needs to be restricted to functions that tend to zero. Intuitively, the average return estimates converge only if the search policy converges, and the search policy converges only if the temperature tends to zero.

Theorem 4.2.4. *For any MDP $\mathcal{M}$, if $\epsilon_{\text{AR-BTS}} > 0$ and $\alpha_{\text{AR-BTS}}(x) \to 0$, then $\pi_{\text{AR-BTS}} \xrightarrow{p} \pi^*$, $\mathbb{E}[\texttt{sim_regr}(s_0, \psi_{\text{AR-BTS}})] \to 0$ and $\mathbb{E}[\texttt{sim_regr}(s_0, \texttt{mv}_{\text{AR-BTS}})] \to 0$.*

4.2.2 Decaying ENtropy Tree Search

Algorithm Definition *Decaying ENtropy Tree Search* (DENTS) extends the BTS algorithm by adding *entropy estimates* as an exploration bonus. DENTS uses Bellman value estimates $\hat{V}_{\text{DNTS}}$ and $\hat{Q}_{\text{DNTS}}$ similarly to BTS, but also uses entropy value estimates, $\bar{\mathcal{H}}_{V,\text{DNTS}}$ and $\bar{\mathcal{H}}_{Q,\text{DNTS}}$, which are Monte Carlo estimates of the entropy of the search policy rooted from the relevant node. By keeping a separate estimate for entropy, DENTS is able to use entropy in its search policy, and discard it for recommendations.

DENTS uses parameters that correspond to the BTS parameters: `normalised_Q`, $N_{\text{DNTS}}^{\hat{Q}_{\text{heuristic}}}$, $N_{\text{DNTS}}^{\hat{V}_{\text{heuristic}}}$, $\alpha_{\text{DNTS}}(x) > 0$ and ϵ_{DNTS}. Additionally, the entropy terms are weighted by a non-negative function $\beta_{\text{DNTS}}(x) \geq 0$ during the selection phase. Typically, β_{DNTS} is chosen such that $\beta_{\text{DNTS}}(x) \to 0$ as $x \to \infty$, so that the entropy weighting diminishes over time, allowing deeper parts of the tree to be explored[2].

The Q-values used during search are computed as follows:

$$\hat{Q}_{\text{DNTS}}^{\text{sft}}(s,a) = \hat{Q}_{\text{DNTS}}(s,a) + \beta_{\text{DNTS}}(N(s))\bar{\mathcal{H}}_{Q,\text{DNTS}}(s,a), \tag{4.13}$$

$$\hat{Q}_{\text{DNTS}}^{\text{sft,bias}}(s,a) = \frac{N(s,a)\hat{Q}_{\text{DNTS}}^{\text{sft}}(s,a) + N_{\text{DNTS}}^{\hat{Q}_{\text{heuristic}}}\hat{Q}_{\text{heuristic}}(s,a)}{N(s,a) + N_{\text{DNTS}}^{\hat{Q}_{\text{heuristic}}}}, \tag{4.14}$$

$$\hat{Q}_{\text{DNTS}}^{\text{sft,norm}}(s,a) = \begin{cases} \frac{\hat{Q}_{\text{DNTS}}^{\text{sft,bias}}(s,a) - \min_{a\in\mathcal{A}} \hat{Q}_{\text{DNTS}}^{\text{sft,bias}}(s,a)}{\max_{a\in\mathcal{A}} \hat{Q}_{\text{DNTS}}^{\text{sft,bias}}(s,a) - \min_{a\in\mathcal{A}} \hat{Q}_{\text{DNTS}}^{\text{sft,bias}}(s,a)} & \text{if } \texttt{normalised_Q}, \\ \hat{Q}_{\text{DNTS}}^{\text{sft,bias}}(s,a) & \text{otherwise.} \end{cases} \tag{4.15}$$

The DENTS search policy is similar to BTS:

$$\pi_{\text{DNTS}}(a|s) = (1 - \lambda(s, \epsilon_{\text{DNTS}}))\rho_{\text{DNTS}}(a|s) + \frac{\lambda(s, \epsilon_{\text{DNTS}})}{|\mathcal{A}|}, \tag{4.16}$$

$$\rho_{\text{DNTS}}(a|s) \propto \exp\left(\frac{\hat{Q}_{\text{DNTS}}^{\text{sft,norm}}(s,a)}{\alpha_{\text{DNTS}}(N(s))}\right), \tag{4.17}$$

$$\lambda(s,\epsilon) = \min\left(1, \frac{\epsilon}{\log(e + N(s))}\right). \tag{4.18}$$

[2]In practice, it is also advisable to choose β_{DNTS} such that $\frac{\beta_{\text{DNTS}}(x)}{\alpha_{\text{DNTS}}(x)} \to 0$, so that the entropy weighting decays faster than the temperature. Otherwise, even as the temperature vanishes, the entropy bonus may still dominate the Q-values in the Boltzmann exponent, skewing the search policy away from the optimal action.

The `init_v` and `init_q` functions are also similar to BTS, setting: $N(s) = 0$, $N(s,a) = 0$, $\hat{V}_{\text{DNTS}}(s) = \hat{V}_{\text{heuristic}}(s)$ and $\hat{Q}_{\text{DNTS}}(s,a) = \hat{Q}_{\text{heuristic}}(s,a)$. Additionally, the entropy value estimates are initialised to zero: $\bar{\mathcal{H}}_{V,\text{DNTS}}(s) = 0$ and $\bar{\mathcal{H}}_{Q,\text{DNTS}}(s,a) = 0$.

Given a trajectory $\tau = (s_0, a_0, r_0, \ldots, s_{h-1}, a_{h-1}, r_{h-1}, s_h) \sim \pi_{\text{DNTS}}$, the `backup_q` and `backup_v` functions update the entropy value estimates for $t = h-1, \ldots, 0$:

$$\bar{\mathcal{H}}_{Q,\text{DNTS}}(s_t, a_t) \leftarrow \sum_{s' \in \text{Succ}(s_t,a_t)} \frac{N(s')}{N(s_t,a_t)} \bar{\mathcal{H}}_{V,\text{DNTS}}(s'), \tag{4.19}$$

$$\bar{\mathcal{H}}_{V,\text{DNTS}}(s_t) \leftarrow \mathcal{H}(\pi_{\text{DNTS}}(\cdot|s_t)) + \sum_{a \in \mathcal{A}} \pi_{\text{DNTS}}(a|s_t) \bar{\mathcal{H}}_{Q,\text{DNTS}}(s_t, a), \tag{4.20}$$

where $\mathcal{H}$ is the Shannon entropy function. The Bellman value estimates are updated similarly to BTS, for $t = h-1, \ldots, 0$:

$$\hat{Q}_{\text{DNTS}}(s_t, a_t) \leftarrow R(s_t, a_t) + \sum_{s' \in \text{Succ}(s_t,a_t)} \left(\frac{N(s')}{N(s_t,a_t)} \hat{V}_{\text{DNTS}}(s') \right), \tag{4.21}$$

$$\hat{V}_{\text{DNTS}}(s_t) \leftarrow \frac{N_{\text{DNTS}}^{\hat{V}_{\text{heuristic}}} \hat{V}_{\text{heuristic}}(s_t) + N(s_t) \max_{a \in \mathcal{A}} \hat{Q}_{\text{DNTS}}(s_t, a)}{N(s_t) + N_{\text{DNTS}}^{\hat{V}_{\text{heuristic}}}}. \tag{4.22}$$

The recommendation policies for DENTS are also defined identically to BTS:

$$\psi_{\text{DNTS}}(s) = \arg\max_{a \in \mathcal{A}} \hat{Q}_{\text{DNTS}}(s,a), \tag{4.23}$$

$$\texttt{mv}_{\text{DNTS}}(s) = \arg\max_{a \in \mathcal{A}} N(s,a), \tag{4.24}$$

Average Returns In *Decaying ENtropy Tree Search with Average Returns* (AR-DENTS) the Bellman value estimates are replaced with average returns $\bar{V}_{\text{AR-DNTS}}$ and $\bar{Q}_{\text{AR-DNTS}}$. For brevity and to avoid repetition, the equations are omitted but can be found in Appendix **??** TODO: double check this ref.

Summary of Theoretical Results Corresponding theoretical results hold for DENTS and AR-DENTS to those stated for BTS. It is additionally required that $\beta_{\text{DNTS}} \to 0$, so that the entropy bonus vanishes and the search policy is not skewed away from the optimal action. Proofs of the theorems below are given in Section **??**.

Theorem 4.2.5. *For any MDP $\mathcal{M}$, if $\epsilon_{\text{DNTS}} > 0$, the expected simple regret of ψ_{DNTS} tends to zero: $\mathbb{E}[\texttt{sim_regr}(s_0, \psi_{\text{DNTS}})] \to 0$.*

Theorem 4.2.6. *For any MDP $\mathcal{M}$, if $\epsilon_{\text{DNTS}} > 0$, $\alpha_{\text{DNTS}}(x) \to 0$ and $\beta_{\text{DNTS}}(x) \to 0$, then $\pi_{\text{DNTS}} \xrightarrow{p} \pi^*$ and $\mathbb{E}[\texttt{sim_regr}(s_0, \texttt{mv}_{\text{DNTS}})] \to 0$.*

Theorem 4.2.7. *For any MDP $\mathcal{M}$, if there exists $\alpha_{\text{DNTS}}(x) \geq L > 0$ and $U \geq \beta_{\text{DNTS}}(x)$, then there exist constants $C, k > 0$ such that $\mathbb{E}[\texttt{sim_regr}(s_0, \psi_{\text{DNTS}})] \leq C\exp(-kn)$ after running n trials of DENTS.*

Theorem 4.2.8. *For any MDP $\mathcal{M}$, if $\epsilon_{\text{AR-DNTS}} > 0$, $\alpha_{\text{AR-DNTS}}(x) \to 0$ and $\beta_{\text{AR-DNTS}}(x) \to 0$, then $\pi_{\text{AR-DNTS}} \xrightarrow{p} \pi^*$, $\mathbb{E}[\texttt{sim_regr}(s_0, \psi_{\text{AR-DNTS}})] \to 0$ and $\mathbb{E}[\texttt{sim_regr}(s_0, \texttt{mv}_{\text{AR-DNTS}})] \to 0$.*

4.2.3 Stochastic Search Policies: Prior Knowledge and Computational Efficiency

This subsection discusses two properties of using a stochastic search policy in MCTS. First, the prior policy $\pi_{\text{heuristic}}$ can be incorporated into the search policy as a mixture policy, allowing prior knowledge to guide the selection phase. Second, the *Alias Method* [**? ?**] is used to derive a more efficient MCTS algorithm by trading off policy accuracy for computational speed, providing an answer to **Q2.1 - Complexity**.

4.2.3.1 Incorporating Prior Knowledge Through Mixture Policies

In MENTS, BTS and DENTS, a uniform distribution is mixed with the Boltzmann distribution to form the search policy, using a logarithmically decaying weight that gradually transitions towards the Boltzmann distribution as the number of visits increases. The prior policy $\pi_{\text{heuristic}}$ can be incorporated in the same way: by mixing it into the algorithm's search policy π_{ALG} with a decaying weight, prior knowledge guides exploration early in the search while leaving the asymptotic behaviour unchanged.

The mixture policy π_{mix} is defined by:

$$\pi_{\text{mix}}(a|s) = (1 - \lambda(s, \epsilon_{\text{mix}}))\pi_{\text{ALG}}(a|s) + \lambda(s, \epsilon_{\text{mix}})\pi_{\text{heuristic}}(a|s), \tag{4.25}$$

$$\lambda(s, \epsilon) = \min\left(1, \frac{\epsilon}{\log(e + N(s))}\right). \tag{4.26}$$

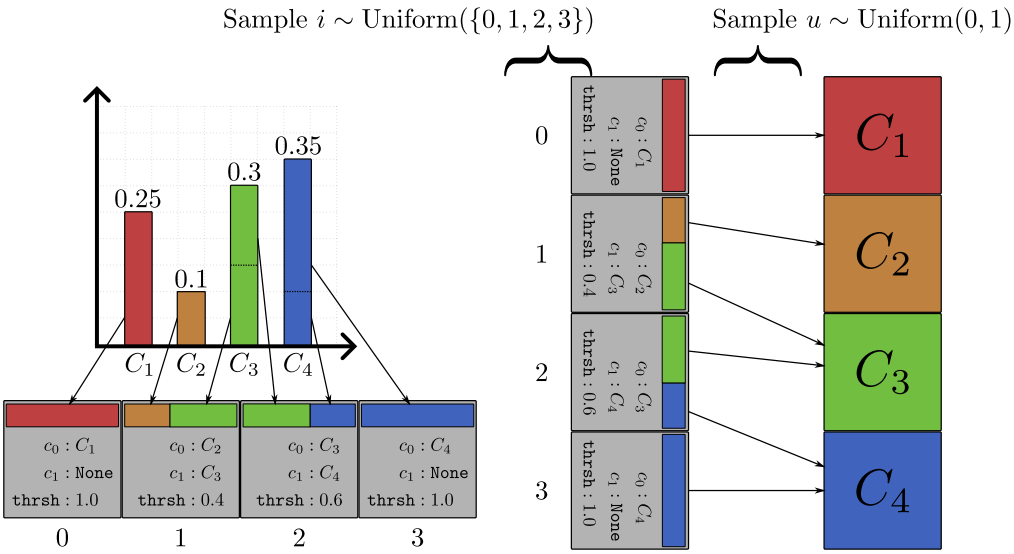


Figure 4.5: Example of building and sampling from an alias table, on a distribution with four categories. Left: shows how the probability mass is split into the four entries of the table. Right: shows how two random variables can be used to sample from the table in $O(1)$ time, where i is used to index into the table, and u is used to compare with `thrsh` and decide which of the two arrows from the entry to follow.

where $\epsilon_{\text{mix}} \in [0, \infty)$ controls the weight given to the prior policy in the mixture. The weight given to $\pi_{\text{heuristic}}$ decays with the visit count, eventually reaching zero, so that the behaviour of the algorithm is unchanged in the limit.

4.2.3.2 The Alias Method

This subsection briefly covers the *Alias Method* [**?** **?**], which is used to sample from a categorical distribution in constant time, with a linear cost for preprocessing.

Let $f : \{C_1, \ldots, C_m\} \to [0, 1]$ be the probability mass function of a categorical distribution with m categories. Suppose a sample $c \sim f$ is required.

The naive method to sample from f takes $O(m)$ time, where a continuous uniform random value is sampled, $u \sim$ Uniform$(0, 1)$, and f is linearly scanned until a probability mass of u has been passed.

The Alias Method, however, uses an $O(m)$ pre-processing step to construct an *alias table*, which is a table consisting of m entries, each accounting for $\frac{1}{m}$ of the probability mass of f. Each entry takes the form $(c_0, c_1, \texttt{thrsh})$, where $c_0, c_1 \in \{C_1, \ldots, C_m\}$ are categories and $\texttt{thrsh} \in [0, 1]$ gives the ratio of the $\frac{1}{m}$

probability mass to assign to c_0 and c_1. That is, a mass of $\frac{\texttt{thrsh}}{m}$ is assigned to c_0 and $\frac{1-\texttt{thrsh}}{m}$ is assigned to c_1.

Sampling from the alias table can be performed in constant time by sampling two random numbers. The first number is a random integer index into the table, and the second number is a random continuous uniform value between 0 and 1. If the entry $(c_0, c_1, \texttt{thrsh})$ is sampled and the second number is less than or equal to `thrsh`, then c_0 is returned, otherwise c_1 is returned. Figure **??** illustrates an example of constructing an alias table and sampling from it.

?] proves that an alias table can always be constructed from an arbitrary categorical distribution, such that the probability of sampling any category from the alias table is identical to the probability of sampling it from the original probability mass function.

4.2.3.3 A More Efficient MCTS

The discussion in this section assumes that `single_node_expansion` is true, and that `transposition_table` is false. Additionally, the runtime complexities of `init_q` and `sample_outcome` are assumed to be $O(1)$[3]. In this case, the runtime of one MCTS trial consists of the following time costs:

Selection Phase: The `select_action` function is called at up to $H_{\texttt{THTS++}}$ many decision nodes.

Heuristic Step: The `init_v` function is called once to initialise the value of the new decision node.

Backup Phase: The `backup_v` and `backup_q` functions are called at up to $H_{\texttt{THTS++}}$ many decision nodes and chance nodes.

Hence, the worst case complexity of an MCTS trial in `THTS++` is:

$$O(H_{\texttt{THTS++}} \cdot (O(\texttt{select_action}) + O(\texttt{backup_v}) + O(\texttt{backup_q})) + O(\texttt{init_v})). \tag{4.27}$$

[3]This is typically the case when using a simulator. If given an arbitrary MDP, the Alias Method can be used with some preprocessing cost to sample outcomes in $O(1)$ time.

When a rollout is used in `init_v`, its complexity is $O(H_{\texttt{THTS++}} \cdot A)$, where $A = |\mathcal{A}|$ is the number of actions. Unfortunately this is the dominating term in the runtime complexity of an MCTS trial. However, when using function approximation in `init_v`, i.e. $\hat{V}_{\text{heuristic}} = \tilde{V}$, and $\pi_{\text{heuristic}} = \tilde{\pi}$, the runtime complexity of `init_v` is $O(A)$, as $\pi_{\text{heuristic}}$ is evaluated for each action in $\mathcal{A}$ and the results cached. The rest of this subsection will assume that `init_v` runs in $O(A)$ time using function approximation.

Otherwise, `select_action` is typically the dominating factor in Equation (**??**), taking $O(A)$ time. For example in UCT (Section **??**), the `select_action` function must compute a UCB score for each action in $\mathcal{A}$, taking $O(A)$ time.

However, if one is using a stochastic search policy and willing to use a slightly outdated search policy during selection, the Alias Method can be used as follows:

- In `select_action`, actions are sampled from an alias table stored in the decision node;
- In `init_v`, an initial alias table is constructed, with an additional $O(A)$ preprocessing cost, maintaining the $O(A)$ time complexity;
- In `backup_v`, the alias table is updated every A visits to the decision node, with an additional $O(A)$ time cost every A visits, giving only an amortised $O(1)$ time complexity on top of any backup equations used in `backup_v`.

The only remaining significant time costs are in `backup_v` and `backup_q`.

Average Return Backups. It is well known that the incremental updates for computing the average returns used in UCT (Equations (**??**) and (**??**)), AR-BTS and AR-DENTS have a constant $O(1)$ complexity.

Bellman `backup_v`. In BTS and DENTS, the `backup_v` function (Equations (**??**) and (**??**)) computes a maximum over actions in $\mathcal{A}$. Because the Q-value of only one action a_t is changed between the selection and backup phases, the maximum can

be computed more efficiently using a max heap. `backup_v` updates the Q-value[4] corresponding to a_t in the max heap, in $O(\log(A))$ time, and then the maximum value can be read from the top of the heap.

Bellman `backup_q`. In the algorithms considered in this thesis so far, the Q-value backups are all either Bellman or average return backups. Bellman Q-value backups can be computed incrementally in $O(1)$ time. Consider the general form of the Bellman update at chance node (s_t, a_t), using the value estimates $\hat{V}$ and $\hat{Q}$:

$$\hat{Q}(s_t, a_t) \leftarrow R(s_t, a_t) + \sum_{s' \in \mathrm{Succ}(s_t, a_t)} \left(\frac{N(s')}{N(s_t, a_t)} \hat{V}(s') \right). \tag{4.28}$$

Because the value of only one child node, s_{t+1}, is changed between the selection and backup phases, Equation (**??**) can be computed incrementally in $O(1)$ time as:

$$\hat{Q}(s_t, a_t) \leftarrow \begin{cases} R(s_t, a_t) + \hat{V}(s_{t+1}) & \text{if } N(s_t, a_t) = 1, \\ R(s_t, a_t) + S(s_t, a_t) + \frac{N(s_{t+1})}{N(s_t, a_t)} \hat{V}(s_{t+1}) & \text{otherwise,} \end{cases} \tag{4.29}$$

$$S(s_t, a_t) = \frac{N^{\mathrm{prev}}(s_t, a_t)}{N(s_t, a_t)} \left(Q(s_t, a_t) - R(s_t, a_t) - \frac{N^{\mathrm{prev}}(s_{t+1})}{N^{\mathrm{prev}}(s_t, a_t)} \hat{V}^{\mathrm{prev}}(s_{t+1}) \right), \tag{4.30}$$

where S computes the corrected average of $\hat{V}(s')$ for $s' \in \mathrm{Succ}(s_t, a_t) - \{s_{t+1}\}$, $\hat{V}^{\mathrm{prev}}(s_{t+1})$ is the previous value of $\hat{V}(s_{t+1})$, and $N^{\mathrm{prev}}(s_{t+1}) = N(s_{t+1}) - 1$, $N^{\mathrm{prev}}(s_t, a_t) = N(s_t, a_t) - 1$ are the previous values of $N(s_{t+1})$, $N(s_t, a_t)$.

MENTS `backup_v`. Using the same observation as in the previous paragraph, the `backup_v` function in MENTS (Equation (**??**)) can be computed in $O(1)$ time. However, this is difficult to implement in a numerically stable way, because the *correction term*[5] involves products of exponentials.

DENTS Entropy `backup_v`. When the Alias Method is used with DENTS, the search policy π_{DNTS} is updated every A visits. On trials where the search policy is not updated, the same observation can be used again, correcting the value estimates using only the updated value of the child node whose value changed.

[4]Note that in an arbitrary MDP, the Q-value may decrease due to stochasticity in the MDP. If the MDP is deterministic, computing the Q-values will only increase, and the maximum can be computed in $O(1)$ time.

[5]e.g. $S(s_t, a_t)$ in Equation (**??**)

Algorithm	`select_action`	`backup_v`	`init_v`	Trial Complexity
UCT	$O(A)$	$O(1)$	$O(1)$	$O(H_{\texttt{THTS++}} \cdot A)$
PUCT	$O(A)$	$O(1)$	$O(A)$	$O(H_{\texttt{THTS++}} \cdot A)$
MENTS	$O(A)$	$O(1)$	$O(A)$	$O(H_{\texttt{THTS++}} \cdot A)$
MENTS+AM	$O(1)$	$O(1)$	$O(A)$	$O(H_{\texttt{THTS++}} + A)$
BTS	$O(A)$	$O(\log(A))$	$O(A)$	$O(H_{\texttt{THTS++}} \cdot A)$
BTS+AM	$O(1)$	$O(\log(A))$	$O(A)$	$O(H_{\texttt{THTS++}} \cdot \log(A) + A)$
AR-BTS	$O(A)$	$O(1)$	$O(A)$	$O(H_{\texttt{THTS++}} \cdot A)$
AR-BTS+AM	$O(1)$	$O(1)$	$O(A)$	$O(H_{\texttt{THTS++}} + A)$
DENTS	$O(A)$	$O(A)$	$O(A)$	$O(H_{\texttt{THTS++}} \cdot A)$
DENTS+AM	$O(1)$	$O(\log(A))$	$O(A)$	$O(H_{\texttt{THTS++}} \cdot \log(A) + A)$
AR-DENTS	$O(A)$	$O(1)$	$O(A)$	$O(H_{\texttt{THTS++}} \cdot A)$
AR-DENTS+AM	$O(1)$	$O(1)$	$O(A)$	$O(H_{\texttt{THTS++}} + A)$

Table 4.1: Summary of complexities for different MCTS algorithms. Complexities for each of the significant components are shown for each algorithm. AM is short for Alias Method. `backup_q` can be implemented in $O(1)$ time for all algorithms in this table so is omitted. Finally, UCT does not use $\pi_{\text{heuristic}}$, so it can implement `init_v` in $O(1)$ by only computing the function approximation $\hat{V}_{\text{heuristic}} = \tilde{V}$.

When the Alias Method is not used, the search policy π_{DNTS} may change on each trial, so Equation (**??**) needs to be recomputed, taking $O(A)$ time.

Summary of Complexities. The complexities of the different components of an MCTS trial are summarised in Table **??**. To answer **Q2.1 - Complexity**in the affirmative, when the Alias Method is used with the most efficient implementations of `backup_v` and `backup_q`, the complexity of running a trial of MENTS can be improved to $O(H_{\texttt{THTS++}} + A)$. The complexity of running a trial of BTS and DENTS can be improved to $O(H_{\texttt{THTS++}} \log(A) + A)$, and the complexity of running a trial of AR-BTS and AR-DENTS can be improved to $O(H_{\texttt{THTS++}} + A)$.

4.3 Theoretical MDPs

This section builds on the discussion from Section **??** to construct adversarial MDPs that expose the limitations of UCT and the maximum entropy objective.

?] introduce the *D-chain problem*, which is a theoretical MDP for which UCT requires exponentially many trials to find the optimal policy. MENTS is able to solve the problem in a small number of trials due to its maximum entropy

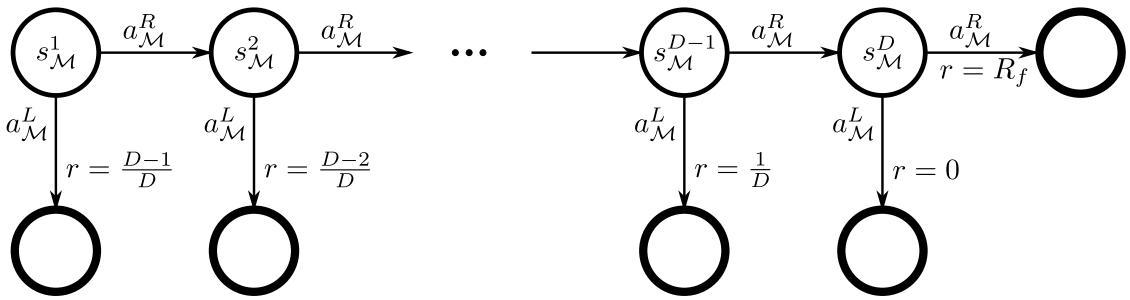


Figure 4.6: An illustration of the *(modified) D-chain problem*, where $s^1_{\mathcal{M}}$ is the starting state and transitions are deterministic. The action $a^L_{\mathcal{M}}$ from state $s^i_{\mathcal{M}}$ results in an immediate reward of $\frac{D-i}{D}$ and the trial ends. Taking action $a^R_{\mathcal{M}}$ to the end of the chain results in a reward of R_f. Where $R_f = 1$ in the original D-chain, and $R_f = \frac{1}{2}$ in the modified D-chain.

objective. However, Section **??** demonstrates that changing a single reward in the MDP can shift the misalignment threshold to a prohibitively small value, rendering the maximum entropy objective ineffective.

In Section **??**, the D-chain problem is extended to include an *entropy trap*, to create an MDP for which both UCT and MENTS cannot solve in any feasible time.

4.3.1 The D-Chain Problem

The *D-chain problem* consists of a chain of D states with two possible actions and is depicted in Figure **??**. If the action $a^L_{\mathcal{M}}$ is taken from state $s^i_{\mathcal{M}}$, then an immediate reward of $R(s^i_{\mathcal{M}}, a^L_{\mathcal{M}}) = \frac{D-i}{D}$ is collected and the trial ends. If the action $a^R_{\mathcal{M}}$ is taken the agent continues along the chain without any reward, until the end of the chain is reached where the agent receives a reward of $R(s^D_{\mathcal{M}}, a^R_{\mathcal{M}}) = 1$. By inspection, the optimal policy is $\pi^*(s) = a^R_{\mathcal{M}}$, to follow the chain to the end and collect the maximum reward of one.

To intuitively understand why UCT will struggle on this MDP, the reward function can be decomposed into a dense and sparse component:

$$R_{\text{dense}}(s^i_{\mathcal{M}}, a^L_{\mathcal{M}}) = \frac{D-i}{D}, \tag{4.31}$$

$$R_{\text{dense}}(s^i_{\mathcal{M}}, a^R_{\mathcal{M}}) = 0, \tag{4.32}$$

$$R_{\text{sparse}}(s^i_{\mathcal{M}}, a) = \mathbb{1}[i = D] \cdot \mathbb{1}[a = a^R_{\mathcal{M}}], \tag{4.33}$$

$$R(s, a) = R_{\text{dense}}(s, a) + R_{\text{sparse}}(s, a). \tag{4.34}$$

The two components of the rewards can be seen to encourage the agent to perform two different behaviours. Intuitively, R_{dense} encourages the agent to leave the chain as soon as possible, while R_{sparse} rewards the agent for staying on the chain until the end. Since UCT balances exploration and exploitation, it will frequently select $a^L_{\mathcal{M}}$ to exploit the immediate reward of $\frac{D-1}{D}$, rather than exploring to find the reward of one.

Compounding this, once a trial of UCT reaches the unit reward for the first time, the value estimates of $\bar{V}_{\text{UCT}}(s^i_{\mathcal{M}})$ still need many more trials to converge and recommend the optimal policy. This phenomenon can be seen in Figure **??**, where UCT never recommends the optimal policy, while MaxUCT (TODO: add ref to ch2/3) that uses the same search policy with Bellman backups will recommend the optimal policy once one trial reaches the unit reward at the end of the chain.

?] formally prove that UCT will suffer a regret of $\Omega((\exp \circ \exp \circ \ldots \circ \exp)(1))$, where $\circ$ denotes functional composition, and there are D many composed exponential functions.

From the perspective of practical single-objective reinforcement learning, this issue may appear to be one of poor reward shaping. However, rewards with conflicting objectives similar to this naturally arise in the context of multi-objective reinforcement learning. An example of this will be seen in the *Deep Sea Treasure* problem in Chapter **??**, where the scalarised reward combines a dense time cost with a sparse treasure reward, yielding a structure similar to Equation (**??**).

Introducing entropy as an exploration bonus naturally encourages the agent to explore along the chain, because once the agent leaves the chain it can no longer collect a bonus entropy reward. As can be seen in Figure **??**, the D-chain problem is an example of an MDP where the standard and maximum-entropy objectives are well aligned.

To illustrate that in maximum entropy methods the temperature parameter requires careful per-MDP tuning, consider the *modified D-chain problem*, where the sparse reward at the end of the chain is replaced with $R(s^D_{\mathcal{M}}, a^R_{\mathcal{M}}) = \frac{1}{2}$. In this case the optimal policy takes the immediate reward of $R(s^1_{\mathcal{M}}, a^L_{\mathcal{M}}) = \frac{D-1}{D}$ from the

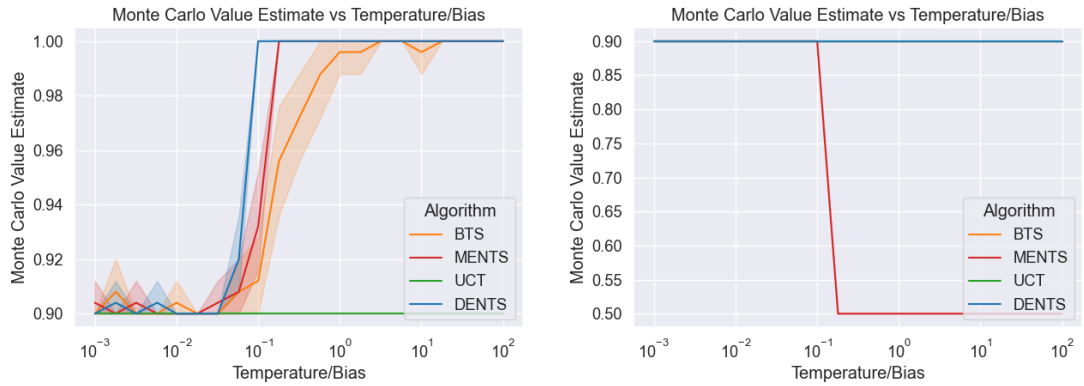


Figure 4.7: Results on the D-chain problem (left), and the modified D-chain problem (right), for a variety of parameters and with $D = 10$. Each algorithm was run for 5000 trials, and results are averaged over 25 repeated experiments. For BTS the temperature function is set to a constant value, and in DENTS the temperature function and entropy weighting function are set to the same constant value. TODO: update figures to have y-axis 0 to 1

initial state, and as illustrated in Figure **??**, the maximum entropy objective is misaligned with the standard objective for a temperature $\alpha > \frac{1}{5}$.

4.3.2 The Entropy Trap

In the *D-chain with Entropy Trap problem* (or *Entropy Trap problem* for short), an additional chain of states is appended to the end of the D-chain, designed to exploit the misalignment between the maximum entropy and standard objectives. At the end of the D-chain, the agent must choose between collecting the sparse reward of one, or entering the *entropy chain* where no reward is given but the agent can accumulate entropy "reward". The Entropy Trap problem is depicted in Figure **??**.

At each step of the entropy chain the agent still has the option of the two actions $a^L_{\mathcal{M}}$ and $a^R_{\mathcal{M}}$. The optimal maximum entropy policy is uniform, taking either action with probability $\frac{1}{2}$ with an entropy of $\log(2)$ per step. If there are E steps in the entropy chain and a temperature of α is used in the maximum entropy objective, then the maximum entropy bonus that can be collected from the chain is $\alpha E \log(2)$. Therefore, $\frac{1}{E\log(2)}$ is an upper bound on the misalignment threshold for the entropy trap, as if $\alpha > \frac{1}{E\log(2)}$, a maximum entropy agent will select the entropy chain over the reward of one at the end of the D-chain.

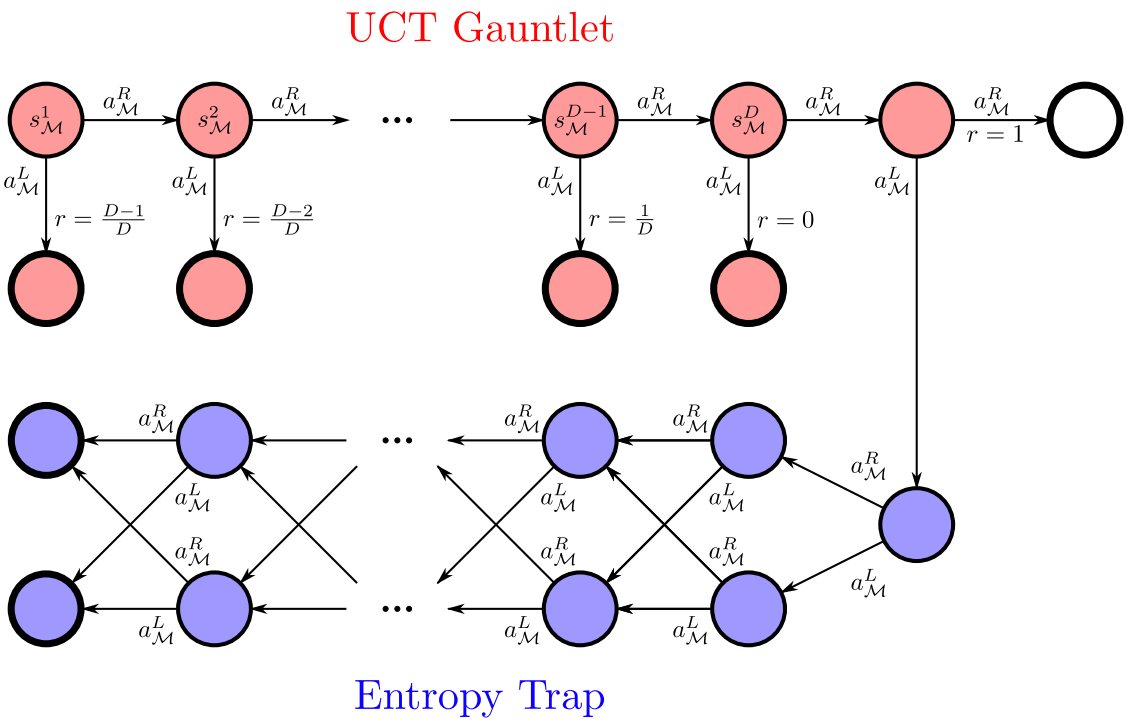


Figure 4.8: An illustration of the *Entropy Trap problem*. In red, the original D-chain problem (Figure **??**) is labelled as the UCT Gauntlet, and the additional states added for the Entropy Trap are in blue. If a reward is not specified for a transition it is zero. TODO: rephrase labels to D-Chain and Entropy Chain

The entropy trap problem can therefore be made arbitrarily difficult for UCT by increasing D, while the misalignment threshold can be made arbitrarily small by increasing E. Theorem **??** uses the case when $D = E$ to show that MENTS must either suffer from misalignment or require an exponential number of trials with respect to the size of the state space.

Theorem 4.3.1. *For any value of α_{MENTS}, either the maximum entropy objective is misaligned, or MENTS requires an exponential number of trials to recommend the optimal policy.*

More precisely, after n trials of MENTS: $\Pr(\psi_{\text{MENTS}} \neq \pi^*) > \frac{1}{2}$ *if* $n \leq 2^{D-1}$*; and either* $\mathbb{E}\left[\texttt{sim_regr}(s_0, \psi_{\text{MENTS}})\right] \not\to 0$ *or there exist constants* $c, k > 0$ *such that* $\mathbb{E}\left[\texttt{sim_regr}(s_0, \psi_{\text{MENTS}})\right] \geq c\left(1 - \frac{n}{k^{|\mathcal{S}|}}\right)$.

Results on the Entropy Trap problem are given in Figure **??** for $(D, E) = (10, 10)$ and $(D, E) = (15, 25)$. In the $(D, E) = (15, 25)$ case, 500,000 trials were run per

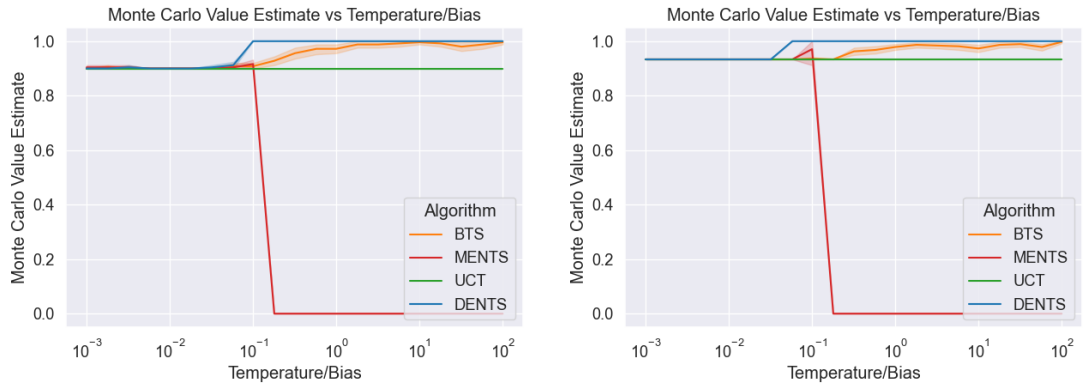


Figure 4.9: Results on the Entropy Trap problem, with $(D, E) = (10, 10)$ (left) and $(D, E) = (15, 25)$ (right). Each algorithm was run with a variety of temperature and bias parameters, averaged over 25 repeated experiments. For BTS the temperature function is set to a constant, and in DENTS the temperature and entropy weighting functions are set to the same constant. All other exploration parameters ϵ and the heuristic value function $\hat{V}_{\text{heuristic}}$ are set to zero. In the $(D, E) = (10, 10)$ case each algorithm was run for 5000 trials: MENTS produces the optimal policy for a small range of temperatures, and BTS and MaxUCT do so for sufficiently large temperature and bias parameters. In the $(D, E) = (15, 25)$ case each algorithm was run for 500,000 trials: at the misalignment threshold MENTS recommendations drop to zero due to the entropy trap, while DENTS recommends the optimal policy using its Bellman value estimates.

parameter setting. The misalignment threshold is bounded above by $\frac{1}{25\log(2)} \approx 0.06$, which is prohibitively small for effective exploration. Below this threshold, no more than 1000 of the 500,000 trials reached the third state of the D-chain, and typically fewer than 100. While BTS and MaxUCT are able to produce the optimal policy given a sufficiently large temperature/bias parameter, increasing D further would eventually prevent them from doing so within a reasonable number of trials. DENTS, by contrast, is able to recommend the optimal policy across a wider range of settings since its Bellman value estimates are unaffected by the entropy chain. For BTS and DENTS, each of α_{BTS}, α_{DNTS} and β_{DNTS} is set to a constant value. While this leads to the majority of trials in DENTS entering the entropy chain, this can be resolved by using a function that tends to zero for β_{DNTS}.

4.4 Empirical Results

TODO: Reference appendix for hyperparameters used. State somewhere that all of the generic edits to the selection function (normalisation, search bias and backup

bias), and reference the BTS equations for those. Actually, make the appendix called additional details on Ch4 empirical results, and there give the entire algorithm used give the full details with full backups and biasing terms for UCT, MENTS etc

TODO: also back reference RTDP, and how BTS with single node expansion turned off is essentially that. Difference is that temperature is decayed but not to zero in RTDP and BTS adds the epsilon exploration term

TODO: also make a comment about how just use H-MCTS as a simple regret baseline, but the simplex regret MCTS methods loosly boil down to a mixture of UCT and uniform combinations.

TODO: Wasn't too careful about changing from active voice to passive voice here in this entire section

TODO: Get some better results using hyperparam optimise //// want a dense env where the (not entropy) temp decay fn gets optimised to soemthing that decays a lot /// want a sparse env where the temp decay fn gets optimised to something flat (or basically flat)

This section provides empirical results on gridworld environments and on the game of Go. It begins by describing the environments, the evaluation setup, and is followed by the empirical results and discussion. The main algorithms discussed in this chapter, BTS and DENTS will be evaluated, using UCT, MENTS, RENTS, TENTS and H-MCTS as baselines TODO: ref litrev or cite?.

4.4.1 Environments

4.4.1.1 (Deterministic) Frozen Lake

The *(Deterministic) Frozen Lake* TODO: remove deterministic is a grid world environment with one goal state. The agent can move in any cardinal direction at each time step, and walking into a wall leaves the agent in the same location. TODO: Add is slippery description. Trap states exist where the agent falls into a hole and the trial ends. TODO: remove next sentence and add paragraph below. If the agent arrives at the goal state after t timesteps, then a sparse reward of

0.99^t is received. TODO: Also add dense reward -1, remove the determinsitic bit, and describe the is slippery option too.

TODO: Two rewards considered. Dense uninformative reward of -1 everywhere, and when falling in a hole the agent gets stuck and recieves a total return of -H. Second reward is sparse, where agents gets a reward of one for reaching the goal state. When is slippery is not on, a discount factor is used, so that shorter paths gain more reward than longer ones. This can effectively be implemented by returning gamma to the t in the deterministic case when agent reaches goal after t timesteps.

TODO: Extra credit: would like to have experiments which vary the proportion of the sparse reward. So have FL(lambda), where lambda specifies the ratio between the dense and sparse rewards. Then investigate what happens as vary lambda. So this is the grid world experiments run again,

Figure **??** gives the specific maps used for Frozen Lake in these experiments. In the maps, `S` denotes the starting location of the agent, `F` denotes spaces the agent can move to, `H` denote holes that end the agents trial and `G` is the goal location.

An 8x8 map (Figure **??**) is used in Section **??** to demonstrate how the different Boltzmann search and entropy based algorithms perform with a variety of temperature parameters. The 8x12 map (Figure **??**) is used for an environment that is sufficiently large map to test and compare algorithms. Each of these maps was randomly generated, with each location having a probability of 1/5 of being a hole, and the maps were checked to have a viable path from the starting location to the goal location. TODO: with the exception of maps XXX, YYY, which were not randomly generated for these experiments, and chosen to correspond to the default maps for that size in Gymnasium (TODO: cite)

4.4.1.2 Sailing Problem

The *Sailing Problem* is another grid world environment with one goal state, at the opposite corner to the starting location of the agent. This problem has often been used to evaluate MCTS algorithms [**?** **?** **?** **?**].

```
SFFFFFHF
FFFFFFFF
FHFHFFFF
FFFFFFHH
FFFHFFFF
FHHHFFFF
FFFFFHFF
FFFFFFFG
```

(a) 8x8 Frozen Lake.

```
SFHFFFHFFFFF
FFFFFFFHFFFF
HFFFFFHFFFFF
FHFFHFFFFFFF
HHFFFFFFFFFF
FHFFFFHFFFFF
FHFFFHHFHFFF
FFFFFFFFFHHG
```

(b) 8x12 Frozen Lake.

Figure 4.10: Maps used for the Frozen Lake environment. `S` is the starting location for the agent, `F` represents floor that the agent can move too, `H` are holes that end the agents trial and `G` is the goal location. TODO: Add 4x4, 5x5 or 4x6, 6x6 or 4x8 maps for stoch envs TODO: change to images of 4x4 and 8x8 frozen lake from running gym, just take screenshots

```
FFFFFG
FFFFFF
FFFFFF
FFFFFF
FFFFFF
SFFFFF
```

(a) 6x6 Sailing Problem map. TODO: Replace with blue grid, boat and buoy clip art if time.

$$\begin{pmatrix} 0.4 & 0.3 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.3 \\ 0.4 & 0.3 & 0.3 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 \\ 0.0 & 0.4 & 0.3 & 0.3 & 0.0 & 0.0 & 0.0 & 0.0 \\ 0.0 & 0.0 & 0.4 & 0.3 & 0.3 & 0.0 & 0.0 & 0.0 \\ 0.0 & 0.0 & 0.0 & 0.4 & 0.2 & 0.4 & 0.0 & 0.0 \\ 0.0 & 0.0 & 0.0 & 0.0 & 0.3 & 0.3 & 0.4 & 0.0 \\ 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.3 & 0.3 & 0.4 \\ 0.4 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.3 & 0.3 \end{pmatrix}$$

(b) Wind transition probabilities.

Figure 4.11: The map and wind transition probabilities used for the 6x6 Sailing Problem. The same notation is used for the map as Figure **??** For the wind transition probabilities, the $(i, j)th$ element of the matrix denotes the probability that the wind changes from direction i to direction j, where 0 denotes North/up, 1 denotes North-East/up-right, and so on. TODO: Update caption.

The agent has 8 different actions, where it can travel in any cardinal or semi-cardinal direction. In each state of the MDP, the wind is blowing in a given direction and will stochastically change after every transition. The agent cannot sail directly into the wind. Actions and wind directions can take values in $\{0, 1, \ldots, 7\}$, with a value of 0 representing North/up, 2 representing East/right, 4 representing South/down and 6 representing West/right. The remaining numbers represent the inter-cardinal directions. The cost of each action depends on the *tack*, the angle between the direction of the agent's travel and the wind. TODO: The agent incurs a (dense) cost of (thetawind - theta)/45 + 1 for traveling at angle theta

when wind is in direction thetawind.

Figure **??** gives the map and wind transition probabilities for the Sailing problem, using the same map notation as for Frozen Lake.

4.4.1.3 Go

For a more challenging domain, a round-robin tournament was run on the game of Go, which has widely motivated the development of MCTS methods [**? ? ?**]. *Area scoring* is used to score the games, with a *komi* (a score handicap for black) of 7.5.

The open-source library KataGo [**?**] was used to provide a value network $\tilde{V}$ and prior policy network $\tilde{\pi}$ for the algorithms.

PUCT algorithm [**?**] was used as a baseline algorithm, as described in Alpha Go Zero [**?**], using prioritised UCB [**?**] to utilise the policy neural network. Each algorithm was limited to 5 seconds of compute time per move, allowed to use 32 search threads per move, and had access to 80 Intel Xeon E5-2698V4 CPUs clocked at 2.2GHz, and a single Nvidia V100 GPU on a shared compute cluster.

The methods described in this chapter can be adapted for two-player games by using a discount factor in the MDP. The value of a policy π with a discount factor γ is defined as:

$$V^{\pi}(s;t) = \mathbb{E}_{\tau\sim\pi}\left[\sum_{i=t}^{H-1}\gamma^{i-t}r_i \middle| s_t = s\right]. \tag{4.35}$$

TODO: should define this in ch2? It's only really relevant here, and kind of in Frozen lake If the two players are using the policies π_0 and π_1, and π is the policy that interleves π_0 at even timesteps with π_1 at odd timesteps, then using a discount factor of $\gamma = -1$ is sufficient to handle two-player games. TODO: assuming that the reward function always gives rewards from the perspective of the player taking the current action.

4.4.2 Evaluation Proceedure

Consider an algorithm with search tree $\mathcal{T}$, which provides a partial recommendation policy ψ_{alg}. The recommendation policy is made complete by using a uniformly

random policy for states and actions that are outside of the tree $\mathcal{T}$ as follows:

$$\psi(a|s) = \begin{cases} 1 & \text{if } s \in \mathcal{T} \text{ and } a = \psi_{\text{alg}}(s), \\ 0 & \text{if } s \in \mathcal{T} \text{ and } a \neq \psi_{\text{alg}}(s), \\ \frac{1}{|\mathcal{A}|} & \text{otherwise.} \end{cases} \tag{4.36}$$

Using the completed policy ψ, a monte carlo value estimate of V^{ψ} is computed by sampling a number of trajectories from ψ and averaging the returns. TODO: Kind of want a better argument for this line of eval here. Although this proceedure is evaluating the algorithms in an *offline planning* setting, it still indicates how the algorithms perform in an *online* setting when planning in simulation is interleaved with letting the agent act in the real environment. TODO: either scrap the evaluation question, or do something more to justify this.But should say something like "this evaluates the policy, and the online performance depends on the policies produced online, therefore correlated"

All experiments in this chapter were run with `single_node_expansion` turned off and did not use the Alias method as described in Section **??**, apart from the experiments on Go, which used `single_node_expansion`and the Alias method.

4.4.2.1 Gridworld Hyperparameter Selection

To select hyperparameters for the algorithms in the gridworld environments, the BayesOpt package [**?**] was used. Full details on the hyperparameter tuning and the selected parameters are given in Appendix **??** TODO: double check this ref.

These experiments were run with `single_node_expansion` turned off and did not use the Alias method (Section **??**).

4.4.2.2 Go Hyperparameter Selection

To tune hyperparemters for algorithms used in the Go experiments, a methodical sequence of round robin tournaments were run on a 9×9 board, where in each tournament one parameter was tuned at a time. Full details of these tournaments and the selected hyperparamters are given in Appendix **??** TODO: double check this ref.

Of note, both Bellman value estimates and average returns were considered for the Go round robin. In the hyperparamter tuning the two versions of each

algorithm were tuned seperately and then played against each other. In each case the average returns algorithms decisively beat the Bellman versions. This could have been because of average returns helping to average out noisy neural network outputs, and the ability of the AR algorithms to run more trials (as described in Section **??**) in the given search time.

These experiments were run with `single_node_expansion` turned on and made use of the Alias method (Section **??**). Additionally, the value network $\tilde{V}$ was used for the value initialisation function $\hat{V}_{\text{heuristic}}$, and prior policy network $\tilde{\pi}$ was mixed into the search policy as described in Section **??**.

4.4.3 Results and Discussion

TODO: Wanted to show the breadth first search if Qinit incorrectly set then get breadth first search in costs case. If can estimate, for example know Rmin and can get Qinit to H times Rmin then can do that. Another zero knowledge Qinit function can use is Qinit equals min a Q(s,a) where for (s,a) in tree, if no such (s,a) then zero (its uniform policy then anyway).

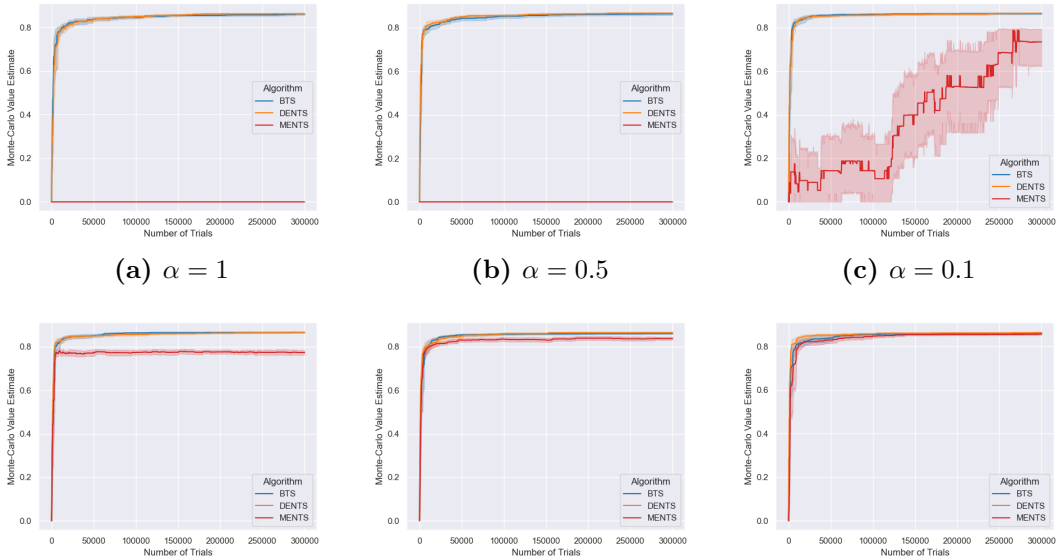
TODO: Talk about how exploitation is useful in two cases in the exploration setting. When have stochastic env, need to exploit to get better value estimates. When env is very large, need exploitation to explore deeper parts of env. Also exploiting helpful when reward is dense and/or informative

TODO: Should also eval AR-BTS etc on these envs? Might make the arguments/results nicer and more complete

This subsection first demonstrates the parameter sensitivity of maximum entropy methods on the Frozen Lake environment. Then results on Frozen Lake and the Sailing problem are given with optimised parameters, and finally results from the Go round robin.

4.4.3.1 Parameter sensitivity in Frozen Lake

TODO: this subsubsection currently just a c and p from neurips



(a) $\alpha = 1$ **(b)** $\alpha = 0.5$ **(c)** $\alpha = 0.1$

(d) $\alpha = 0.05$ **(e)** $\alpha = 0.01$ **(f)** $\alpha = 0.005$

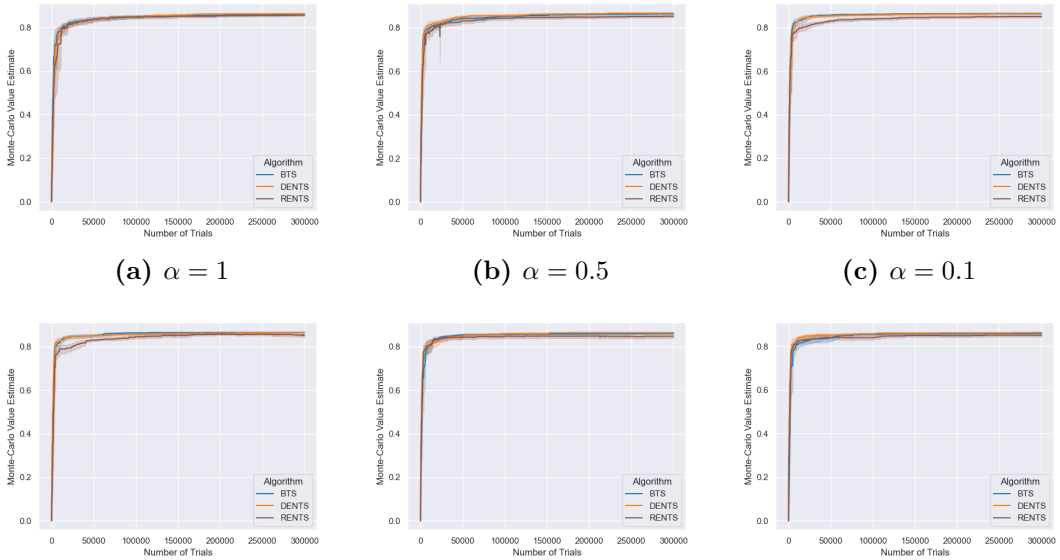
Figure 4.12: MENTS with a variety of temperatures on an 8x8 Frozen Lake environment. BTS and DENTS are included for reference. TODO: Update fig?

Further discussion on sensitivity to the temperature parameter in MENTS [?], RENTS and TENTS [?] is now given. Results on the 8x8 Frozen Lake environment (Figure ??) are given using a variety of temperatures to demonstrate how each algorithm performs with different temperatures in that domain.

TODO: clean up hyperparam setting

In Figures ?? and ?? MENTS and TENTS take the scenic route to the goal state for medium temperatures, where the reward for reaching the goal is still significant, but they can obtain more entropy reward by wondering around the gridworld for a while first. For higher temperatures they completely ignore the goal state, opting to rather maximise policy entropy. Interestingly, RENTS in Figure ?? fared better than MENTS and TENTS and never really ignored the goal state for all of the temperatures considered.

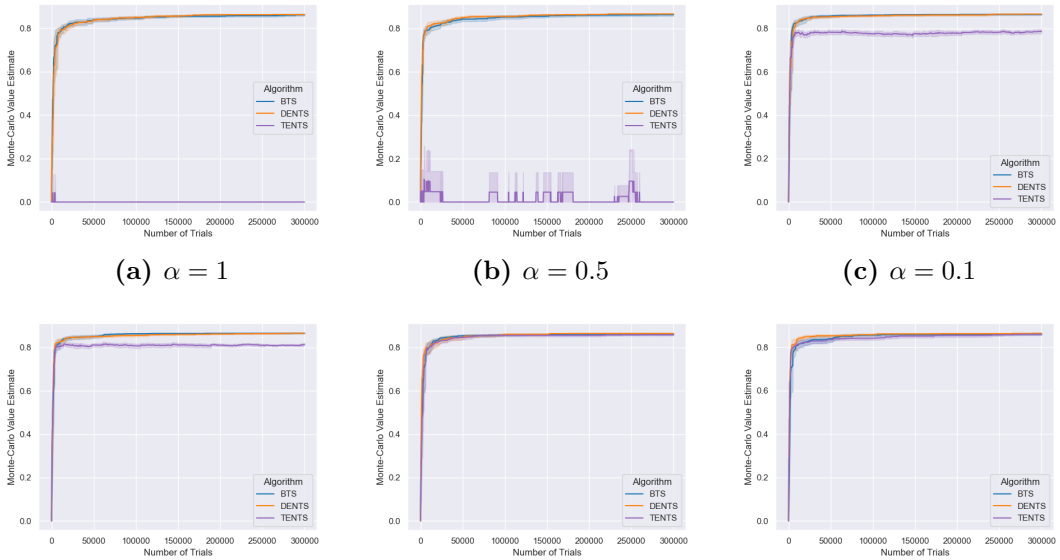
In contrast, both BTS and DENTS were relativly agnostic to the temperature parameter in this environment (with $\epsilon = 1$) and were always able to find the goal state. Curves for BTS and DENTS are included in all of Figures ??, ?? and ?? for reference and as a comparison for MENTS, RENTS and TENTS.



(a) $\alpha = 1$ **(b)** $\alpha = 0.5$ **(c)** $\alpha = 0.1$

(d) $\alpha = 0.05$ **(e)** $\alpha = 0.01$ **(f)** $\alpha = 0.005$

Figure 4.13: RENTS with a variety of temperatures on an 8x8 Frozen Lake environment. BTS and DENTS are included for reference. TODO: Update fig?



(a) $\alpha = 1$ **(b)** $\alpha = 0.5$ **(c)** $\alpha = 0.1$

(d) $\alpha = 0.05$ **(e)** $\alpha = 0.01$ **(f)** $\alpha = 0.005$

Figure 4.14: TENTS with a variety of temperatures on an 8x8 Frozen Lake environment. BTS and DENTS are included for reference. TODO: Update fig?

4.4.3.2 Gridworld Results

TODO: this subsubsection currently just a c and p from neurips

The 8x12 Frozen Lake environment (Figure **??**) and a 6x6 Sailing Problem (Figure **??**) were used to evaluate the algorithms introduced in this chapter against the baseline algorithms.

Each algorithm is run 25 times on each environment and evaluated every 250 trials using 250 trajectories. A horizon of 100 was used for Frozen Lake and 50 for the Sailing Problem. Curves of these evaluations are given in Figure **??**.

In Frozen Lake (Figure **??**), entropy proved to be a useful exploration bonus for the *sparse reward*. Values in UCT and BTS remain at zero until a trial successfully reaches the goal. However, entropy guides agents to avoid trap states, where the entropy is zero. DENTS was able to perform similarly to MENTS, and BTS was able to improve its policy over time more than UCT.

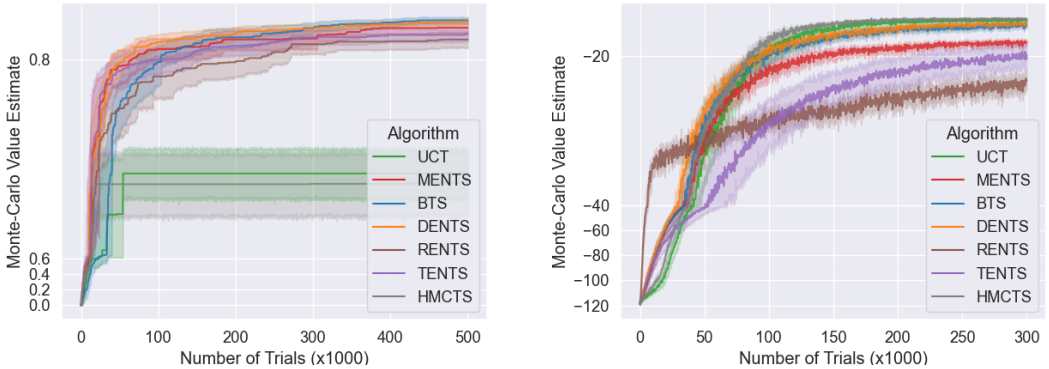
In the Sailing Problem (Figure **??**) UCT performs well due to the dense reward. TODO: also mention how exploiting is useful to get better value estimates in stochastic environments. BTS and DENTS also manage to keep up with UCT. MENTS and TENTS appear to be slightly hindered by entropy in this environment. The relative entropy encourages RENTS to pick the same actions over time, so it tends to pick a direction and stick with it regardless of cost.

Finally, BTS and DENTS were able to perform well in both domains with a sparse and dense reward structure, whereas the existing methods performed better on one than the other, hence making BTS and DENTS good candidates for a general purpose MCTS algorithm.

TODO: Say here that the initial wind directions for Sailing we set to North, and South-East.

4.4.3.3 Go Results

Results of the round-robin are summarised in Table **??**, where each algorithm played 50 games as black and 50 as white. BTS was able to run the most trials per move and beat all of the other algorithms other than DENTS which it drew. The Alias



(a) 8x12 Frozen Lake. **(b)** 6x6 Sailing Problem.

Figure 4.15: Results for gridworld environments. TODO: update fig? TODO: add more caption?

method allowed the Boltzmann search algorithms to run significantly more trials per move than PUCT. BTS and DENTS were able to beat PUCT with results of 57-43 and 58-42 respectively. Using entropy did not seem to have much benefit in these experiments, as can be witnessed by MENTS only beating TENTS, and DENTS drawing 50-50 with BTS. This is likely because the additional exploration provided by entropy is vastly outweighed by utilising the information contained in the neural networks $\tilde{V}$ and $\tilde{\pi}$. Interestingly RENTS had the best performance out of the prior works, losing 43-57 to PUCT, and the use of relative entropy appears to take advantage of a heuristic for Go that the RAVE [**?**] algorithm used: the value of a move is typically unaffected by other moves on the board.

To validate the strength of the `THTS++` PUCT agent, it was played directly against KataGo [**?**], with a budget of 1600 trials per move. Interestingly, the `THTS++` PUCT agent won 61-39 in 9x9 Go, and lost 35-65 in 19x19 Go, suggesting that our PUCT agent is strong enough to provide a meaningful comparison for our other general purpose algorithms. Finally, note that the neural networks from KataGo were not fine-tuned, so the Boltzmann search algorithms directly used the networks that were trained for use in PUCT (with average returns).

Black \White	PUCT	AR-M	AR-R	AR-T	AR-B	AR-D	Trials/move
PUCT	-	33-17	27-23	42-8	17-33	15-35	1054
AR-MENTS	12-48	-	13-37	38-12	10-40	12-38	4851
AR-RENTS	20-30	24-26	-	39-11	18-32	14-36	3672
AR-TENTS	8-42	11-39	9-41	-	6-44	10-40	5206
AR-BTS	25-25	35-15	31-19	34-16	-	15-35	5375
AR-DENTS	23-27	36-14	29-21	36-14	15-35	-	4677

Table 4.2: Results for the Go round-robin tournament. The first column gives the agent playing as black. The final column gives the average trials run per move across the entire round-robin. In the top row, we abbreviate the algorithm names for space.

4.5 Theoretical Results

This section provides the theoretical analysis and proofs for this chapter. It is structured as follows:

1. First, Section **??** defines additional notation so that *MCTS processes* can be reasoned about with respect to the number of trials run;

2. Second, Section **??** establishes preliminary results, lemmas and definitions used in subsequent proofs;

3. Third, Section **??** proves the consistency results for (AR-)BTS and (AR-)DENTS from Section **??** (Theorems **??**, **??**, **??**, **??**, **??**, **??**);

4. Finally, Section **??** proves Theorem **??**, showing that the maximum entropy objective is either misaligned or requires an exponential number of trials in the Entropy Trap problem.

Some of the particularly verbose arguments are left to Appendix **??** TODO: double check this ref so that they do not interrupt the flow of the main arguments. In particular, Theorems **??** and **??**, which show exponential simple regret bounds on BTS and DENTS, are left to Appendix **??** TODO: double check this ref.

The following assumptions are made for this section, which do not affect the asymptotic behaviour of the algorithms but simplify the proofs and presentation:

- the unique state assumption (Section **??**) remains, `single_node_expansion` is set to false and `transposition_table` is set to false;

- heuristic values are set to zero, $\hat{V}_{\text{heuristic}}(s) = 0$ and $\hat{Q}_{\text{heuristic}}(s, a) = 0$ for all $s \in \mathcal{S}$ and $a \in \mathcal{A}$;
- the length of all trajectories and the planning horizon $H_{\texttt{THTS++}}$ are equal to the horizon H of the MDP;
- the pseudo-visit biasing terms $N_{\text{BTS}}^{\hat{V}_{\text{heuristic}}}$, $N_{\text{BTS}}^{\hat{Q}_{\text{heuristic}}}$, $N_{\text{DNTS}}^{\hat{V}_{\text{heuristic}}}$, $N_{\text{DNTS}}^{\hat{Q}_{\text{heuristic}}}$, $N_{\text{AR-BTS}}^{\hat{V}_{\text{heuristic}}}$, $N_{\text{AR-BTS}}^{\hat{Q}_{\text{heuristic}}}$, $N_{\text{AR-DNTS}}^{\hat{V}_{\text{heuristic}}}$, $N_{\text{AR-DNTS}}^{\hat{Q}_{\text{heuristic}}}$, from Section **??** are all set to zero.

Finally, note that mathematically (AR-)BTS is a special case of (AR-)DENTS, with $\beta_{\text{DNTS}}(x) = 0$ and $\beta_{\text{AR-DNTS}}(x) = 0$. Results will be proven about (AR-)DENTS, and the corresponding results for BTS follow as a direct consequence.

4.5.1 Notation

Because of the unique state assumption (Section **??**), any state $s_t \in \tau = (s_0, a_0, r_0, \ldots, s_{H-1}, a_{H-1}, r_{H-1}, s$ may only appear at timestep t in any trajectory. Throughout this chapter, s_t and a_t are used to denote an arbitrary state and action at an arbitrary timestep t.

When making arguments that apply to multiple algorithms, the general policies π and ψ are used. When making arguments about specific algorithms, subscripts are used, e.g. π_{BTS}.

To reason about MCTS processes over multiple trials, the existing notation is extended with superscripts to track both the number of trials run and the number of times each node has been visited.

Most variables are denoted with a superscript n to denote that it is the value used on the nth trial of MCTS. For example:

- If an algorithm uses a search policy π and outputs a recommendation policy ψ, then the search policy followed on the nth trial is denoted π^n, and the recommendation policy after n trials is denoted ψ^n.
- When it is necessary to reason about multiple trajectories, the superscript notation will also be used. The trajectory sampled in the nth trial is $\tau^n = (s_0^n, a_0^n, r_0^n, \ldots, s_{H-1}^n, a_{H-1}^n, r_{H-1}^n, s_H^n)$. If it is sampled from a search policy π^n, then $\tau^n \sim \pi^n$.

- The search tree after n trials is denoted $\mathcal{T}^n$, the initial search tree is $\mathcal{T}^0 = \{s_0\}$, and $\mathcal{T}^n = \mathcal{T}^{n-1} \cup \{s \in \tau^n\} \cup \{(s, a) \in \tau^n\}$.

Let $T^n(s_t)$ and $T^n(s_t, a_t)$ denote the sets of trial indices on which s_t was visited and action a_t was selected respectively, in the first n trials, where t is the timestep at which the node appears in the trajectory:

$$T^n(s_t) = \{i | 1 \leq i \leq n, s_t^i = s_t\} \tag{4.37}$$

$$T^n(s_t, a_t) = \{i | 1 \leq i \leq n, s_t^i = s_t, a_t^i = a_t\}. \tag{4.38}$$

The visit counts after n trials are defined as follows, with $s_t \in \mathcal{S}$ and $a_t \in \mathcal{A}$:

$$N^n(s_t) = \sum_{i=1}^{n} \mathbb{1}[s_t^i = s_t] = |T^n(s_t)|, \tag{4.39}$$

$$N^n(s_t, a_t) = \sum_{i \in T^n(s_t)} \mathbb{1}[a_t^i = a_t] = |T^n(s_t, a_t)|. \tag{4.40}$$

Under the unique state assumption, the visit counts can also be expressed in the following equivalent forms, which are more convenient for the proofs:

$$N^n(s_t, a_t) = \sum_{i \in T^n(s_t)} \mathbb{1}[a_t^i = a_t], \tag{4.41}$$

$$N^n(s_{t+1}) = \sum_{i \in T^n(s_t, a_t)} \mathbb{1}[s_{t+1}^i = s_{t+1}] = |T^n(s_t, a_t)|. \tag{4.42}$$

The key observation is that $N^n(s_t, a_t)$ can be computed as a sum over the $N^n(s_t)$ trials on which s_t was visited, and $N^n(s_{t+1})$ as a sum over the $N^n(s_t, a_t)$ trials on which a_t was selected from s_t.

Value estimates are the one exception to the superscript notation to denote the number of trials. Superscripts for value estimates denote the number of times that the value has been backed up. To differentiate this counting with respect to backups, instead of with respect to trials, a superscript with brackets will be used. For example, the backups for DENTS value estimates would be written as follows. Let $\tau^n \sim \pi^n_{\text{DNTS}}$ be a trajectory sampled from the DENTS search policy on the nth trial. Let $\tau^n = (s_0, a_0, r_0, \ldots, s_{h-1}, a_{h-1}, r_{h-1}, s_h)$. The backups

for $t = h-1, \ldots, 0$ are then:

$$\hat{Q}_{\text{DNTS}}^{(N^n(s_t,a_t))}(s_t,a_t) = R(s_t,a_t) + \sum_{s' \in \text{Succ}(s_t,a_t)} \left(\frac{N^n(s')}{N^n(s_t,a_t)} \hat{V}_{\text{DNTS}}^{(N^n(s'))}(s') \right), \tag{4.43}$$

$$\hat{V}_{\text{DNTS}}^{(N^n(s_t))}(s_t) = \max_{a \in \mathcal{A}} \hat{Q}_{\text{DNTS}}^{(N^n(s_t,a))}(s_t,a), \tag{4.44}$$

$$\bar{\mathcal{H}}_{Q,\text{DNTS}}^{(N^n(s_t,a_t))}(s_t,a_t) = \sum_{s' \in \text{Succ}(s_t,a_t)} \frac{N^n(s')}{N^n(s_t,a_t)} \bar{\mathcal{H}}_{V,\text{DNTS}}^{(N^n(s'))}(s'), \tag{4.45}$$

$$\bar{\mathcal{H}}_{V,\text{DNTS}}^{(N^n(s_t))}(s_t) = \mathcal{H}(\pi_{\text{DNTS}}^n(\cdot|s_t)) + \sum_{a \in \mathcal{A}} \pi_{\text{DNTS}}^n(a_t|s_t) \bar{\mathcal{H}}_{Q,\text{DNTS}}^{(N^n(s_t,a_t))}(s_t,a_t). \tag{4.46}$$

For reference, all of the algorithms considered in this Section are written out in full with this additional notation in Appendix **??** TODO: double check this ref. TODO: make sure handle initv and initq properly in the appendix

4.5.2 Preliminaries

This subsection introduces definitions and preliminary lemmas that are used in the proofs of Sections **??** and **??**.

The proofs in this section make use of the *immediate simple regret*, which measures the regret incurred by taking a single action step rather than following the full policy. Working with immediate simple regret simplifies the proofs, since bounds and convergence results transfer between the two notions via Lemma **??**.

Definition 4.5.1. *The* immediate simple regret *is defined as:*

$$\texttt{imm_regr}(s_t, \psi^n) = V^*(s_t) - \mathbb{E}_{a_t \sim \psi^n(s_t)}[Q^*(s_t,a_t)]. \tag{4.47}$$

The following lemma shows that asymptotic bounds and convergence results transfer between immediate and simple regret, so it suffices to prove results about immediate simple regret.

Lemma 4.5.1. $\mathbb{E}[\texttt{sim_regr}(s_t,\psi^n)] = O(f(n))$ *for all* $s_t \in \mathcal{S}_0$ *if and only if* $\mathbb{E}[\texttt{imm_regr}(s_t,\psi^n)] = O(f(n))$ *for all* $s_t \in \mathcal{S}_0$. *Consequently,* $\mathbb{E}[\texttt{sim_regr}(s_t,\psi^n)] \to 0$ *if and only if* $\mathbb{E}[\texttt{imm_regr}(s_t,\psi^n)] \to 0$.

Proof. Firstly, the simple regret can be written recursively in terms of the immediate simple regret:

$$
\begin{aligned}
\texttt{sim_regr}(s_t, \psi^n) =& V^*(s_t) - V^{\psi^n}(s_t) && (4.48)\\
=& V^*(s_t) - \mathbb{E}_{a_t \sim \psi^n(s_t)}[Q^{\psi^n}(s_t, a_t)] && (4.49)\\
=& V^*(s_t) - \mathbb{E}_{a_t \sim \psi^n(s_t)}[R(s_t, a_t) + \mathbb{E}_{s_{t+1} \sim \Pr(\cdot|s_t,a_t)}[V^{\psi^n}(s_{t+1})]] && (4.50)\\
=& V^*(s_t) - \mathbb{E}_{a_t \sim \psi^n(s_t)}\Big[Q^*(s_t, a_t) - \mathbb{E}_{s_{t+1} \sim \Pr(\cdot|s_t,a_t)}[V^*(s_{t+1})]\\
&+ \mathbb{E}_{s_{t+1} \sim \Pr(\cdot|s_t,a_t)}[V^{\psi^n}(s_{t+1})]\Big] && (4.51)\\
=& V^*(s_t) - \mathbb{E}_{a_t \sim \psi^n(s_t)}[Q^*(s_t, a_t)]\\
&+ \mathbb{E}_{a_t \sim \psi^n(s_t), s_{t+1} \sim \Pr(\cdot|s_t,a_t)}[V^*(s_{t+1}) - V^{\psi^n}(s_{t+1})] && (4.52)\\
=& \texttt{imm_regr}(s_t, \psi^n)\\
&+ \mathbb{E}_{a_t \sim \psi^n(s_t), s_{t+1} \sim \Pr(\cdot|s_t,a_t)}[\texttt{sim_regr}(s_{t+1}, \psi^n)], && (4.53)
\end{aligned}
$$

where line (**??**) uses $R(s_t, a_t) = Q^*(s_t, a_t) - \mathbb{E}_{s_{t+1} \sim \Pr(\cdot|s_t,a_t)}[V^*(s_{t+1})]$, a rearrangement of the Bellman optimality equation for $Q^*(s_t, a_t)$.

Immediately, this shows that $\mathbb{E}[\texttt{imm_regr}(s_t, \psi^n)] \leq \mathbb{E}[\texttt{sim_regr}(s_t, \psi^n)]$ and hence if $\mathbb{E}[\texttt{sim_regr}(s_t, \psi^n)] = O(f(n))$ then $\mathbb{E}[\texttt{imm_regr}(s_t, \psi^n)] = O(f(n))$.

If $\mathbb{E}[\texttt{imm_regr}(s_t, \psi^n)] = O(f(n))$ then by induction over $t = H, \ldots, 0$, using the relation in Equation (**??**):

$$
\begin{aligned}
\mathbb{E}[\texttt{sim_regr}(s_t, \psi^n)] &= \mathbb{E}\Big[O(f(n)) + \mathbb{E}_{a_t \sim \psi^n(s_t), s_{t+1} \sim \Pr(\cdot|s_t,a_t)}[O(f(n))]\Big] && (4.54)\\
&= O(f(n)). && (4.55)
\end{aligned}
$$

The base case is $t = H$: since s_H is a terminal state $\texttt{sim_regr}(s_H, \psi^n) = \texttt{imm_regr}(s_H, \psi^n) = 0$. □

The DKW inequality (Theorem **??**) is used to show that the empirical transition probabilities converge in probability to the true transition probabilities.

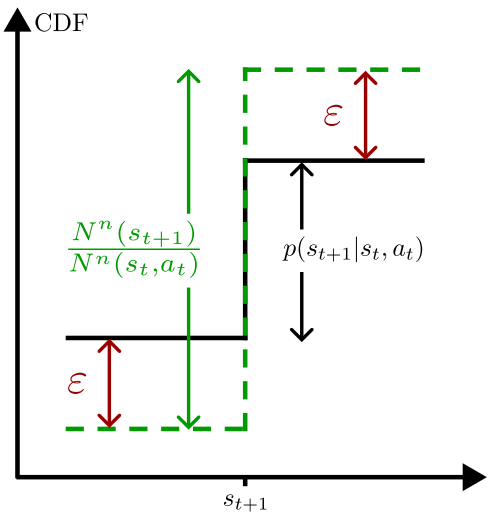


Figure 4.16: Bounding the empirical transition probabilities to the true transition probabilities. The true cdf is shown as a solid black line, the empirical cdf is shown as a dashed green line, and a worst case error of $\varepsilon/2$, using Theorem **??**, is shown in red. The probability mass of $p(s_{t+1}|s_t, a_t)$ and empirical probability mass of $\frac{N^n(s_{t+1})}{N^n(s_t,a_t)}$ is also indicated to demonstrate how the constructed distribution gives Corollary **??**.

Corollary 4.5.1. *For all* $(s_t, a_t) \in \mathcal{S} \times \mathcal{A}$ *and for all* $\varepsilon > 0$*:*

$$\Pr\left(\max_{s_{t+1}\in \mathrm{Succ}(s_t,a_t)} \left|\frac{N^n(s_{t+1})}{N^n(s_t, a_t)} - p(s_{t+1}|s_t, a_t)\right| > 2\varepsilon\right) \leq 2\exp\left(-2\varepsilon^2 N^n(s_t, a_t)\right), \tag{4.56}$$

as $N^n(s_t, a_t) \to \infty$*. Consequently* $\frac{N^n(s_{t+1})}{N^n(s_t,a_t)} \xrightarrow{p} p(s_{t+1}|s_t, a_t)$ *as* $N^n(s_t, a_t) \to \infty$*.*

Proof outline. Consider the cumulative distribution function of the successor state distribution $p(s_{t+1}|s_t, a_t)$, using some arbitrary ordering over the successor states in $\mathrm{Succ}(s_t, a_t)$. The result follows by applying Theorem **??** with $m = N^n(s_t, a_t)$.

To see why the bound involves 2ε rather than ε, consider Figure **??**. Applying the DKW inequality bounds the error in the cumulative distribution function by ε at every point. Since the probability mass of any single successor state corresponds to the difference between two CDF values, the errors at either side may accumulate, giving a worst-case error of 2ε in the empirical probability of any individual successor state. □

4.5.3 Consistency Results

This subsection proves the consistency results for (AR-)DENTS. The proof of Theorem **??** is given in full, as it uses all of the preliminary lemmas. The proofs of

the remaining theorems are given as outlines, highlighting only where they differ.

Theorems **??**, **??** and **??** for (AR-)BTS follow directly from Theorems **??**, **??** and **??** respectively, by setting $\beta_{\text{DNTS}}(x) = 0$ and $\beta_{\text{AR-DNTS}}(x) = 0$.

Theorem ??. *For any MDP $\mathcal{M}$, if $\epsilon_{\text{AR-DNTS}} > 0$, $\alpha_{\text{AR-DNTS}}(x) \to 0$ and $\beta_{\text{AR-DNTS}}(x) \to 0$, then $\pi_{\text{AR-DNTS}} \xrightarrow{p} \pi^*$, $\mathbb{E}[\texttt{sim_regr}(s_0, \psi_{\text{AR-DNTS}})] \to 0$ and $\mathbb{E}[\texttt{sim_regr}(s_0, \texttt{mv}_{\text{AR-DNTS}})] \to 0$.*

Proof. Firstly by Corollary **??** (of Lemma **??**), with the assumption that $\epsilon_{\text{AR-DNTS}} > 0$, all visit counts $N^n(s_t)$ and $N^n(s_t, a_t)$ will tend to infinity almost surely as $n \to \infty$.

Then the (Q-)value estimates and search policy will converge almost surely to their optimal counterparts. This follows by induction on the timestep t. For the base case $t = H$ at the planning horizon, $\hat{V}^{(N^n(s_H))}_{\text{AR-DNTS}}(s_H) = V^*(s_H) = 0$ by definition. The inductive steps proceed as follows, for each $s_t \in \mathcal{S}_0$ and $a_t \in \mathcal{A}$:

1. Assume that $\hat{V}^{(N^n(s_{t+1}))}_{\text{AR-DNTS}}(s_{t+1}) \xrightarrow{p} V^*(s_{t+1})$;
2. By Lemma **??**, it follows that $\hat{Q}^{(N^n(s_t,a_t))}_{\text{AR-DNTS}}(s_t, a_t) \xrightarrow{p} Q^*(s_t, a_t)$ and consequently $\psi^n_{\text{AR-DNTS}}(a_t|s_t) \xrightarrow{p} \pi^*(a_t|s_t)$;
3. Letting $\dot{Q}^n(s_t, a_t) = \hat{Q}^{(N^n(s_t,a_t))}_{\text{AR-DNTS}}(s_t, a_t) + \beta_{\text{AR-DNTS}}(N^n(s_t))\bar{\mathcal{H}}^{(N^n(s_t,a_t))}_{Q,\text{AR-DNTS}}(s_t, a_t)$, the assumption that $\beta_{\text{AR-DNTS}}(x) \to 0$ ensures that $\dot{Q}^n(s_t, a_t) \xrightarrow{p} Q^*(s_t, a_t)$. Then using Lemma **??**, with the assumption that $\alpha_{\text{AR-DNTS}}(x) \to 0$, it follows that $\pi^n_{\text{AR-DNTS}}(a_t|s_t) \xrightarrow{p} \pi^*(a_t|s_t)$;
4. Then by Lemma **??** it follows that $\frac{N^n(s_t,a_t)}{N^n(s_t)} \xrightarrow{p} \pi^*(a_t|s_t)$ and consequently $\texttt{mv}^n_{\text{AR-DNTS}}(a_t|s_t) \xrightarrow{p} \pi^*(a_t|s_t)$;
5. By Lemma **??**, $\hat{V}^{(N^n(s_t))}_{\text{AR-DNTS}}(s_t) \xrightarrow{p} V^*(s_t)$.

The immediate regret (Definition **??**) then satisfies:

$$\mathbb{E}\left[\texttt{imm_regr}(s_t, \psi^n_{\text{AR-DNTS}})\right] = \sum_{a \in \mathcal{A}} \left(V^*(s_t) - Q^*(s_t, a)\right) \psi^n_{\text{AR-DNTS}}(a|s_t) \tag{4.57}$$

$$\xrightarrow{p} \sum_{a \in \mathcal{A}} \left(V^*(s_t) - Q^*(s_t, a)\right) \pi^*(a|s_t) \tag{4.58}$$

$$= \mathbb{E}\left[\texttt{imm_regr}(s_t, \pi^*)\right] \tag{4.59}$$

$$= 0, \tag{4.60}$$

and similarly

$$\mathbb{E}\left[\texttt{imm_regr}(s_t, \texttt{mv}^n_{\text{AR-DNTS}})\right] \to 0. \tag{4.61}$$

The result follows by Lemma **??** to lift the convergence of the immediate regret to convergence of the simple regret. □

The proof of Theorem **??** proceeds similarly, but since only the recommendation policy ψ_{DNTS} is required to converge, steps **??** and **??** are unnecessary.

Theorem ??. *For any MDP $\mathcal{M}$, if $\epsilon_{\text{DNTS}} > 0$, the expected simple regret of ψ_{DNTS} tends to zero:* $\mathbb{E}[\texttt{sim_regr}(s_0, \psi_{\text{DNTS}})] \to 0$.

Proof outline. Proof proceeds similarly to the proof of Theorem **??**, where the induction steps **??** and **??** are unnecessary, and hence the conditions $\alpha_{\text{DNTS}}(x) \to 0$ and $\beta_{\text{DNTS}}(x) \to 0$ are not required. Induction step **??** uses Lemma **??** instead of Lemma **??**. □

The proof of Theorem **??** is nearly identical to that of Theorem **??**, the only difference being that Lemma **??** is used in place of Lemma **??**.

Theorem ??. *For any MDP $\mathcal{M}$, if $\epsilon_{\text{DNTS}} > 0$, $\alpha_{\text{DNTS}}(x) \to 0$ and $\beta_{\text{DNTS}}(x) \to 0$, then $\pi_{\text{DNTS}} \xrightarrow{p} \pi^*$ and* $\mathbb{E}[\texttt{sim_regr}(s_0, \texttt{mv}_{\text{DNTS}})] \to 0$.

Proof outline. Proof proceeds similarly to the proof of Theorem **??**. □

The following lemma considers a bandit setting at a single decision node, and shows that the logarithmically decaying exploration term ensures every action is selected infinitely often almost surely.

Lemma 4.5.2. *Let ρ^m be an arbitrary policy and consider the following search policy used on the mth round of a bandit problem:*

$$\pi^m(a|s) = (1 - \lambda(m))\rho^m(a) + \lambda(m)\frac{1}{|\mathcal{A}|}, \tag{4.62}$$

$$\lambda(m) = \min(1, \epsilon/\log(e + m)), \tag{4.63}$$

where $\epsilon > 0$, $s \in \mathcal{S}$ and $a \in \mathcal{A}$. Let $a^m \sim \pi^m$ be the mth independently sampled action, and let $M^m(a)$ be the number of times action a was selected in the first m rounds, i.e. $M^m(a) = \sum_{i=1}^m \mathbb{1}[a^i = a]$. Then $M^m(a) \overset{as}{\to} \infty$ as $m \to \infty$ for all $a \in \mathcal{A}$.

Proof. Suppose that the result does not hold and there exists some $b \in \mathcal{A}$ such that:

$$\Pr\left(M^m(b) \to \infty\right) < 1. \tag{4.64}$$

If $M^m(b) \not\to \infty$, then it is logically equivalent to say that there exists some round $\ell \in \mathbb{N}$ from which b is never sampled again (with $\ell \geq \exp(\epsilon + 1)$). That is:

$$\Pr\left(\bigcap_{m=\ell}^{\infty} a^m \neq b\right) = \Pr\left(M^m(b) \not\to \infty\right) > 0. \tag{4.65}$$

However, from Equation (**??**) action b is sampled with probability at least:

$$\Pr\left(a^m = b\right) \geq \frac{\lambda(m)}{|\mathcal{A}|} = \frac{\epsilon}{|\mathcal{A}|\log(e + m)}. \tag{4.66}$$

So the probability of never sampling action b again after the first $\ell - 1$ rounds is at most:

$$\Pr\left(\bigcap_{m=\ell}^{\infty} a^m \neq b\right) = \lim_{k\to\infty} \prod_{m=\ell}^{k} \Pr(a^m \neq b) \tag{4.67}$$

$$\leq \lim_{k\to\infty} \prod_{m=\ell}^{k} \left(1 - \frac{\epsilon}{|\mathcal{A}|\log(e + k)}\right) \tag{4.68}$$

$$\leq \lim_{k\to\infty} \left(1 - \frac{\epsilon}{|\mathcal{A}|\log(e + k)}\right)^{k-\ell} \tag{4.69}$$

$$= 0, \tag{4.70}$$

which is in contradiction to inequality (**??**) that was assumed.

Inequality (**??**) holds because the factors in the product are increasing with respect to m. To see why the final equality (**??**) holds, consider the simpler function $f(x) = \left(1 - \frac{1}{\log(x)}\right)^x$. Taking logarithms and applying L'Hôpital's rule multiple times gives $\lim_{x\to\infty} \log f(x) = -\infty$, so $\lim_{x\to\infty} f(x) = 0$.

□

As a consequence of Lemma **??**, every reachable state (and state-action pair) will be visited infinitely often in any MCTS process using a search policy of the form given in Equation (**??**).

The following corollary lifts Lemma **??** to the full tree search setting, where the search policy is indexed by the number of trials n, but the lemma is applied at each node with respect to its visit count $m = N^n(s_t)$.

Corollary 4.5.2. *For all $s_t \in \mathcal{S}_0$ and $a_t \in \mathcal{A}$, any MCTS process using a search policy of the following form on the nth trial:*

$$\pi^n(a_t|s_t) = (1 - \lambda(s_t, \epsilon))\rho^n(a_t|s_t) + \lambda(s_t, \epsilon)\frac{1}{|\mathcal{A}|}, \tag{4.71}$$

$$\lambda(s_t, \epsilon) = \min(1, \epsilon/\log(e + N^n(s_t))), \tag{4.72}$$

where ρ^n is an arbitrary policy and $\epsilon > 0$. All reachable states and state-action pairs will be visited infinitely often. That is $N^n(s_t) \overset{as}{\to} \infty$ and $N^n(s_t, a_t) \overset{as}{\to} \infty$ as $n \to \infty$.

Proof outline. The result follows by induction over Lemma **??**. At $t = 0$ the result holds directly from the lemma with $m = n$. For the inductive step, if $N^n(s_t, a_t) \overset{as}{\to} \infty$ then $N^n(s_{t+1}) \overset{as}{\to} \infty$ since s_{t+1} is sampled from $p(\cdot|s_t, a_t)$ with positive probability, then $N^n(s_{t+1}, a_{t+1}) \overset{as}{\to} \infty$ by applying Lemma **??** with $m = N^n(s_{t+1})$. □

The following lemma shows that if the value estimates converge to the optimal values, then the Q-value estimates computed via a sample-weighted average of child node values also converge to the optimal Q-values.

Lemma 4.5.3. *For all $s_t \in \mathcal{S}_0$, $a_t \in \mathcal{A}$, if $N^n(s_t, a_t) \to \infty$ and $\dot{V}^{(N^n(s'))}(s') \xrightarrow{p} V^*(s')$ for all $s' \in \mathrm{Succ}(s_t, a_t)$, then the Q-values computed by:*

$$\dot{Q}^{(N^n(s_t,a_t))}(s_t, a_t) = R(s_t, a_t) + \sum_{s' \in \mathrm{Succ}(s_t,a_t)} \frac{N^n(s')}{N^n(s_t, a_t)} \dot{V}^{(N^n(s'))}(s'), \tag{4.73}$$

converge to the optimal Q-values: $\dot{Q}^{(N^n(s_t,a_t))}(s_t, a_t) \xrightarrow{p} Q^*(s_t, a_t)$.

Proof outline. By Corollary **??**, $\frac{N^n(s')}{N^n(s_t,a_t)} \xrightarrow{p} p(s'|s_t, a_t)$ as $N^n(s_t, a_t) \to \infty$. Then:

$$\begin{aligned}
\dot{Q}^{(N^n(s_t,a_t))}(s_t, a_t) &= R(s_t, a_t) + \sum_{s' \in \mathrm{Succ}(s_t,a_t)} \frac{N^n(s')}{N^n(s_t, a_t)} \dot{V}^{(N^n(s'))}(s') && (4.74)\\
&\xrightarrow{p} R(s_t, a_t) + \sum_{s' \in \mathrm{Succ}(s_t,a_t)} p(s'|s_t, a_t) V^*(s') && (4.75)\\
&= Q^*(s_t, a_t). && (4.76)
\end{aligned}$$

□

For Boltzmann search policies, if the Q-value estimates converge to the optimal Q-values and the temperature tends to zero, then the search policy converges to the optimal policy.

Lemma 4.5.4. *If a search policy of the following form is used in MCTS:*

$$\pi^n(a_t|s_t) = (1 - \lambda(s_t, \epsilon))\rho^n(a_t|s_t) + \lambda(s_t, \epsilon)\frac{1}{|\mathcal{A}|}, \tag{4.77}$$

$$\rho^n \propto \exp\left(\frac{\dot{Q}^n(s_t, a_t)}{\alpha(N^n(s_t))}\right), \tag{4.78}$$

$$\lambda(s_t, \epsilon) = \min(1, \epsilon/\log(e + N^n(s_t))), \tag{4.79}$$

where $\epsilon > 0$, $\alpha(x) \to 0$ and $\dot{Q}^n(s_t, a_t) \xrightarrow{p} Q^*(s_t, a_t)$ as $n \to \infty$ for all $s_t \in \mathcal{S}_0$ and $a_t \in \mathcal{A}$. Then $\pi^n(a_t|s_t) \xrightarrow{p} \pi^*(a_t|s_t)$ as $n \to \infty$.

Proof outline. From Corollary **??**, $N^n(s_t) \xrightarrow{as} \infty$ as $n \to \infty$, and hence $\lambda(s_t, \epsilon) \xrightarrow{as} 0$, so $\pi^n(a_t|s_t) - \rho^n(a_t|s_t) \xrightarrow{as} 0$.

It is known that a Boltzmann distribution converges to the one-hot distribution over the argmax as the temperature tends to zero: if $\alpha(N^n(s_t)) \to 0$ and $\dot{Q}^n(s_t, a_t) \xrightarrow{p} Q^*(s_t, a_t)$, then $\rho^n(a_t|s_t) \xrightarrow{p} \mathbb{1}[a_t = \arg\max_{a \in \mathcal{A}} Q^*(s_t, a)] = \pi^*(a_t|s_t)$.

Combining these two results gives $\pi^n(a_t|s_t) \xrightarrow{p} \pi^*(a_t|s_t)$. □

The following lemma shows that if the search policy converges to a limiting policy under which one action is strictly preferred at every state, then the most-visited recommendation policy converges to that uniquely preferred action.

Lemma 4.5.5. *Suppose $\pi^n \xrightarrow{as} \pi_{\text{lim}}$ as $n \to \infty$, where π_{lim} is a policy maximised by some action a_t^*. That is, $\pi_{\text{lim}}(a_t^*|s_t) > \pi_{\text{lim}}(a_t|s_t)$ for all $a_t \in \mathcal{A} - \{a_t^*\}$ and $s_t \in \mathcal{S}_0$. Then the empirical action frequencies converge almost surely to the limiting policy:*

$$\frac{N^n(s_t, a_t)}{N^n(s_t)} \xrightarrow{as} \pi_{\text{lim}}(a_t|s_t). \tag{4.80}$$

Consequently, since π_{lim} has a unique maximising action $a_t^* = \arg\max_{a \in \mathcal{A}} \pi_{\text{lim}}(a|s_t)$ at every state, the most-visited recommendation policy converges to always recommending a_t^*.

Proof.

$$\frac{N^n(s_t, a_t)}{N^n(s_t)} = \sum_{i \in T^n(s_t)} \frac{\mathbb{1}[a_t^i = a_t]}{N^n(s_t)} \tag{4.81}$$

$$\xrightarrow{as} \lim_{n \to \infty} \sum_{i \in T^n(s_t)} \frac{\pi^i(a_t|s_t)}{N^n(s_t)} \tag{4.82}$$

$$= \pi_{\text{lim}}(a_t|s_t), \tag{4.83}$$

where line (**??**) uses Equation (**??**). The almost sure convergence follows from Kolmogorov's criterion[6] [**?**]. The final equality uses Corollary **??** and Stolz-Cesàro's

[6] A form of the Strong Law of Large Numbers for independent but *not* necessarily identically distributed random variables.

theorem: since $\pi^n(a_t|s_t) \xrightarrow{as} \pi_{\text{lim}}(a_t|s_t)$ by assumption, the running average converges to $\pi_{\text{lim}}(a_t|s_t)$.

Since $\frac{N^n(s_t,a_t)}{N^n(s_t)} \xrightarrow{as} \pi_{\text{lim}}(a_t|s_t)$ and π_{lim} has a unique maximising action a_t^* at every state, it follows that $\arg\max_{a\in\mathcal{A}} N^n(s_t,a) \xrightarrow{as} a_t^*$, so the most-visited recommendation policy (e.g. Equation (**??**)) converges to always recommending a_t^*. □

The next two lemmas show that the state value estimates converge to the optimal values, for DENTS and AR-DENTS respectively, given that the Q-value estimates have already converged.

For DENTS, the state value is computed as the maximum over Q-values, so convergence follows directly.

Lemma 4.5.6. *If $N^n(s_t) \xrightarrow{as} \infty$ and $\hat{Q}_{\text{DNTS}}^{(N^n(s_t,a_t))}(s_t,a_t) \xrightarrow{p} Q^*(s_t,a_t)$ for all $s_t \in \mathcal{S}_0$ and $a_t \in \mathcal{A}$, then $\hat{V}_{\text{DNTS}}^{(N^n(s_t))}(s_t) \xrightarrow{p} V^*(s_t)$.*

Proof outline. From the definition of the DENTS state value backup (Equation (**??**)), it follows that:

$$\hat{V}_{\text{DNTS}}^{(N^n(s_t))}(s_t) = \max_{a\in\mathcal{A}} \hat{Q}_{\text{DNTS}}^{(N^n(s_t,a))}(s_t,a) \tag{4.84}$$

$$\xrightarrow{p} \max_{a\in\mathcal{A}} Q^*(s_t,a) \tag{4.85}$$

$$= V^*(s_t). \tag{4.86}$$

□

For AR-DENTS, the state value is computed as a visit-count weighted average of Q-values, so convergence additionally requires the search policy to converge.

Lemma 4.5.7. *If $N^n(s_t) \xrightarrow{as} \infty$, ${\pi_{\text{AR-DNTS}}}^n(a_t|s_t) \xrightarrow{p} \pi^*(a_t|s_t)$ and $\hat{Q}_{\text{AR-DNTS}}^{(N^n(s_t,a_t))}(s_t,a_t) \xrightarrow{p} Q^*(s_t,a_t)$ for all $s_t \in \mathcal{S}_0$ and $a_t \in \mathcal{A}$, then $\hat{V}_{\text{AR-DNTS}}^{(N^n(s_t))}(s_t) \xrightarrow{p} V^*(s_t)$.*

Proof outline. An equivalent equation, computed using child Q-values and visit counts, for the AR-DENTS state value backup is:

$$\hat{V}^{(N^n(s_t))}_{\text{AR-DNTS}}(s_t) = \sum_{a \in \mathcal{A}} \frac{N^n(s_t, a)}{N^n(s_t)} \hat{Q}^{(N^n(s_t,a))}_{\text{AR-DNTS}}(s_t, a). \tag{4.87}$$

By Lemma **??**, $\frac{N^n(s_t,a)}{N^n(s_t)} \xrightarrow{p} \pi^*(a|s_t)$ as $N^n(s_t) \to \infty$. It follows that:

$$\hat{V}^{(N^n(s_t))}_{\text{AR-DNTS}}(s_t) \xrightarrow{p} \sum_{a \in \mathcal{A}} \pi^*(a|s_t) Q^*(s_t, a) \tag{4.88}$$

$$= V^*(s_t). \tag{4.89}$$

□

4.5.4 Entropy Trap

This subsection provides the proof of Theorem **??**, that informally says that for any temperature either the maximum entropy objective is misaligned, or MENTS requires an exponential number of trials to recommend the optimal policy. The core of the argument reduces to bounding the *hitting time* of the state $s^E_{\mathcal{M}}$ — the number of trials required for MENTS to first reach the end of the D-chain portion of the Entropy Trap MDP. For reference, the Entropy Trap MDP is repeated in Figure **??**, with additional state labels added for the proof.

Theorem ??. *For any value of α_{MENTS}, either the maximum entropy objective is misaligned, or MENTS requires an exponential number of trials to recommend the optimal policy.*

More precisely, after n trials of MENTS: $\Pr(\psi_{\text{MENTS}} \neq \pi^*) > \frac{1}{2}$ *if* $n \leq 2^{D-1}$*; and either* $\mathbb{E}\left[\texttt{sim_regr}(s_0, \psi_{\text{MENTS}})\right] \not\to 0$ *or there exist constants* $c, k > 0$ *such that* $\mathbb{E}\left[\texttt{sim_regr}(s_0, \psi_{\text{MENTS}})\right] \geq c\left(1 - \frac{n}{k^{|\mathcal{S}|}}\right)$.

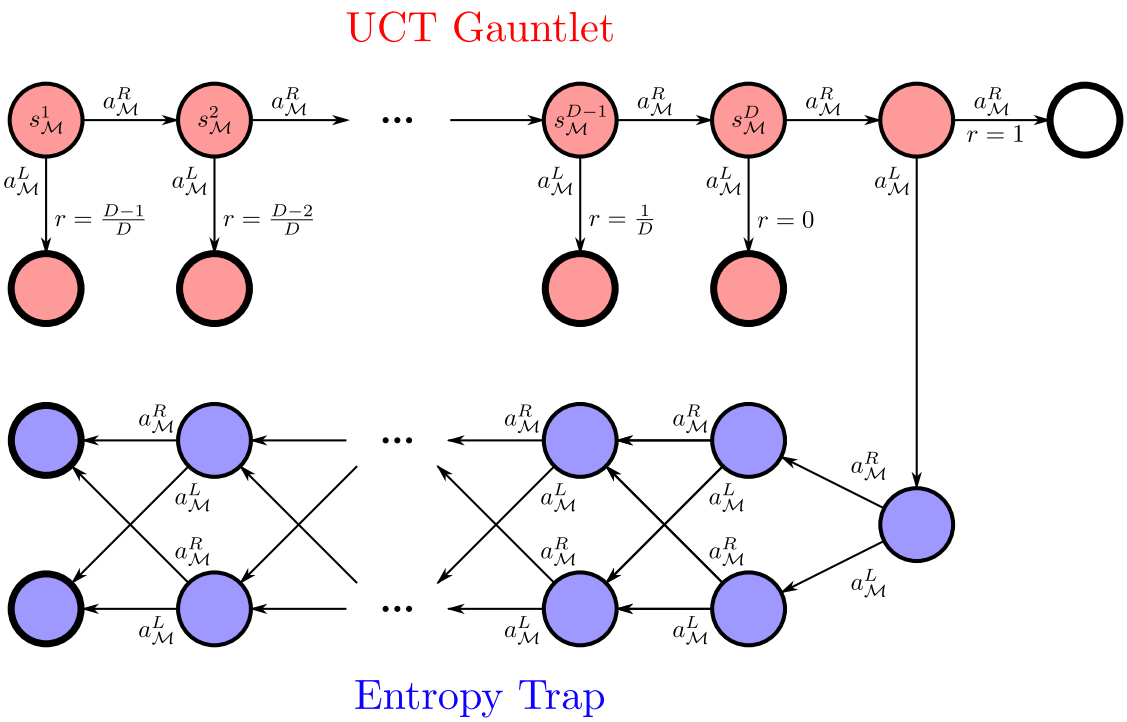


Figure 4.17: (Repeated). An illustration of the *Entropy Trap problem*, with state labels added for reference in the proof below. TODO: Update figure to include an additional state s to the E and s to the F

Proof outline. Proof is by construction. Consider the Entropy Trap MDP defined in Figure **??**, which is parameterised by D the length of the D-chain, and K, half the number of states in the entropy trap. In this proof, the case where $K = D$ is considered.

Note that for clarity, the full superscript notation established in Section **??** is dropped in this proof outline to avoid confusion with the Entropy Trap state notation $s^i_{\mathcal{M}}$.

Two cases need to be considered: when the temperature is sufficiently high for the maximum entropy objective to be misaligned; and, when the temperature is lower than this threshold.

Case 1: $\alpha_{\text{MENTS}} > \frac{1}{\log(2)D}$ **(Maximum entropy objective is misaligned)**

When $\alpha_{\text{MENTS}} > \frac{1}{\log(2)D}$, the optimal soft value of the state $s^F_{\mathcal{M}}$ is greater than one, and so action $a^L_{\mathcal{M}}$ is preferred over $a^R_{\mathcal{M}}$ at state $s^E_{\mathcal{M}}$. Let ϕ be a uniform policy.

Then:

$$Q^*_{\text{sft}}(s^E_{\mathcal{M}}, a^L_{\mathcal{M}}) = V^*_{\text{sft}}(s^F_{\mathcal{M}}) \tag{4.90}$$
$$= 0 + \alpha_{\text{MENTS}} \cdot \mathcal{H}(\phi) \tag{4.91}$$
$$= \alpha_{\text{MENTS}} \cdot \log(2)D \tag{4.92}$$
$$> 1 \tag{4.93}$$
$$= Q^*_{\text{sft}}(s^E_{\mathcal{M}}, a^R_{\mathcal{M}}). \tag{4.94}$$

Hence the maximum entropy objective is misaligned: $\mathbb{E}\left[\texttt{sim_regr}(s^1_{\mathcal{M}}, \psi_{\text{MENTS}})\right] \to 1 > 0$.

Case 2: $\alpha_{\text{MENTS}} \leq \frac{1}{\log(2)D}$

In this case, the temperature is sufficiently low that the MENTS search policy is nearly greedy. Hence MENTS will have a small probability of ever having a trial hitting state $s^E_{\mathcal{M}}$. For now, assume that $\hat{V}_{\text{MENTS}}(s^E_{\mathcal{M}}) = 0$, and $N(s^E_{\mathcal{M}}) = 0$.

First consider the composed and simplified soft backup on the D-chain states for $1 \leq i \leq D$:

$$\hat{V}_{\text{MENTS}}(s^i_{\mathcal{M}}) \leftarrow \alpha \log\left(\frac{1}{\alpha}\left(\frac{D-i}{D} + \hat{V}_{\text{MENTS}}(s^{i+1}_{\mathcal{M}})\right)\right) \tag{4.95}$$
$$\leq \max\left(\frac{D-i}{D}, \hat{V}_{\text{MENTS}}(s^{i+1}_{\mathcal{M}})\right) + \alpha_{\text{MENTS}} \log(2) \tag{4.96}$$
$$\leq \max\left(\frac{D-i}{D}, \hat{V}_{\text{MENTS}}(s^{i+1}_{\mathcal{M}})\right) + \frac{1}{D} \tag{4.97}$$

where inequality (**??**) uses the property of log-sum-exp that $\alpha \log \sum_{j=1}^{\ell} \exp(x_i/\alpha) \leq \max_j(x_j) + \alpha \log(\ell)$.

Then, by induction on $i = D, D-1, \ldots, 1$, it holds that $\hat{V}_{\text{MENTS}}(s^i_{\mathcal{M}}) \leq \frac{D-(i-1)}{D}$:

$$\hat{V}_{\text{MENTS}}(s^i_{\mathcal{M}}) \leq \max\left(\frac{D-i}{D}, \hat{V}_{\text{MENTS}}(s^{i+1}_{\mathcal{M}})\right) + \frac{1}{D} \tag{4.98}$$
$$= \frac{D-i}{D} + \frac{1}{D} \tag{4.99}$$
$$= \frac{D-(i-1)}{D}. \tag{4.100}$$

Hence, it must be the case that $\hat{Q}_{\text{MENTS}}(s^i_{\mathcal{M}}, a^L_{\mathcal{M}}) > \hat{Q}_{\text{MENTS}}(s^i_{\mathcal{M}}, a^R_{\mathcal{M}})$ and $\pi_{\text{MENTS}}(a^R_{\mathcal{M}}|s^i_{\mathcal{M}}) \leq \frac{1}{2}$ for all $1 \leq i \leq D$.

Let $Y(n)$ be the event that the state $s^E_{\mathcal{M}}$ is visited in the nth trial. Additionally, let $Z(n)$ be the event that $s^E_{\mathcal{M}}$ has not been visited on any of the first n trials. That is, $Z(n) = \bigcap_{j=1}^{n} \neg Y(j)$. Additionally note that $Z(n)$ implies that $\hat{V}_{\text{MENTS}}(s^E_{\mathcal{M}}) = 0$ and so:

$$\Pr(Y(n)|Z(n-1)) = \Pr(Y(n)|\hat{V}_{\text{MENTS}}(s^E_{\mathcal{M}}) = 0) \tag{4.101}$$

$$\leq 2^{-D}. \tag{4.102}$$

Hence, the probability that $s^E_{\mathcal{M}}$ is not visited in the first n trials can be bounded below inductively:

$$\Pr(Z(n)) = \Pr\left(\neg Y(n) \cap Z(n-1)\right) \tag{4.103}$$

$$= \Pr\left(\neg Y(n)|Z(n-1)\right) \Pr(Z(n-1)) \tag{4.104}$$

$$\geq (1 - 2^{-D}) \Pr(Z(n-1)) \tag{4.105}$$

$$\geq \dots \tag{4.106}$$

$$\geq (1 - 2^{-D})^n \Pr(Z(0)) \tag{4.107}$$

$$\geq (1 - n2^{-D}), \tag{4.108}$$

where the last line uses Bernoulli's inequality and $\Pr(Z(0)) = 1$. $Z(0)$ is trivially true, as no trials have been run yet.

From the argument above, $Z(n)$ implies that $\hat{Q}_{\text{MENTS}}(s^1_{\mathcal{M}}, a^R_{\mathcal{M}}) \leq \frac{D-1}{D} = \hat{Q}_{\text{MENTS}}(s^1_{\mathcal{M}}, a^L_{\mathcal{M}})$ and so $\psi_{\text{MENTS}}(s^1_{\mathcal{M}}) = a^L_{\mathcal{M}} \neq a^R_{\mathcal{M}} = \pi^*(s^1_{\mathcal{M}})$. Hence, $\Pr(\psi_{\text{MENTS}} \neq \pi^*) = \Pr(Z(n)) \geq (1 - n2^{-D})$, which is greater than $\frac{1}{2}$ if $n \leq 2^{D-1}$.

For the second part of the result, a simple regret of at least $\frac{D-1}{D}$ is incurred with probability at least $\Pr(Z(n))$. Hence, the expected simple regret can be bounded below as follows:

$$\mathbb{E}\left[\texttt{sim_regr}(s^1_{\mathcal{M}}, \psi_{\text{MENTS}})\right] \geq \left(1 - \frac{D-1}{D}\right) \Pr(Z(n)) \tag{4.109}$$

$$\geq \frac{1}{D}\left(1 - n2^{-D}\right). \tag{4.110}$$

Then $|\mathcal{S}| = 4D + 3 \leq 5D$ for $D \geq 3$ and so substituting $D \geq \frac{|\mathcal{S}|}{5}$ gives the result with $c = \frac{1}{D}$ and $k = 2^{1/5}$:

$$\mathbb{E}\left[\texttt{sim_regr}(s^1_{\mathcal{M}}, \psi_{\text{MENTS}})\right] \geq \frac{1}{D}\left(1 - n2^{-D}\right) \tag{4.111}$$

$$\geq \frac{1}{D}\left(1 - \frac{n}{\left(2^{\frac{1}{5}}\right)^{|\mathcal{S}|}}\right). \tag{4.112}$$

□

4.6 Summary

TODO: This chapter introduced BTS and DENTS, established their consistency properties, and demonstrated empirically that DENTS can solve MDPs where UCT and MENTS fail. Chapter **??** extends these ideas to the multi-objective setting.

- What questions were answered
- Copy the intro and reword
- Investigated and demonstrated that entropy can be misaligned
- Demonstrated that can use it as an auxilary bonus objective
- Entropy is revisited in Chapter 6
- Now the focus shifts to the true multi-objective setting

5

Convex Hull Monte Carlo Tree Search

Contents

5.1 Introduction 106
5.1.1 Contextual Tree Search 108
5.2 Contextual Zooming for Trees 111
5.3 Convex Hull Monte Carlo Tree Search 112
5.3.1 CH-CZT 114
5.3.2 CH-UCT 114
5.3.3 CH-BTS 116
5.3.4 CH-DENTS 116
5.4 Results 116
5.4.1 Evaluation Proceedure 117
5.4.2 Environments 119
5.4.3 Results and Discussion 119
5.5 Theory 119

New Ch5 plan:

- Intro, introduce and motivate contextual tree search, forward ref to
- Algos, contextual zooming (motivate by parallel to UCT)
- Algos, convex hull backups
- Algos, why no entropy
- Optimizations, solved labelling (ref LRTDP)

- Optimizations, pruning the number of points in the convex hull
- Empirical results
- Theory, contextual regret, non-contextual policies garunteed to have a linear contextual regret
- Theory, reproduce the CHMCTS results, and say that garunteed to eventually produce optimal (lemmas are actions selected inf often)
- No theory for CZT directly, makes a heuristic adaptation from the base work, and the proofs are very complex in themselves (would add like 50 pages of maths)
- Conclusion, wrap up, motivate move to simplex maps (convex hull slow) and weight space (entropy)

should have a QHull reference when talking about the implementation

say that we prune using that, unless it is a degenerate case where we fall back to using the linear programming method

for theory would just like to have convergence, use ch4 proofs and UCT picks actions inf often as lemma and use CHVI proofs

Allude to that boltzmann probably gives exp contextual simple regret, and PUCT gives similar contextual cumulative regret, and the lower bound from exploring bandits means simple regret of UCT methods has a lower bound

also more critical to look into litrev for what results about MORL there are, shimons paper probably helpful

want to cite if someone has done that linear scalarisation iff making local decisions result

TODO: Move into correct place and update notation appropriately for the chapter. Add pre-amble saying about how its an algorithm for CMAB, but only introduce here because used in one place only

?] introduce the Contextual Zooming (CZ) algorithm to solve CMAB problems. CZ achieves a contextual regret of $O(n^{\frac{1+c}{2+c}} \log(T))$ over the first n rounds, using the

doubling trick [?], where c is the *(Lebesgue) covering dimension* of the similarity space $\mathcal{W} \times \mathcal{X}$. Within this thesis, the covering dimension coincides with the standard notion of dimension: $\mathcal{W} \times \mathcal{X}$ has covering dimension c, if it embeds into $\mathbb{R}^c$.

The CZ algorithm uses a set of balls in the similarity space, called *active balls*. Let A^m denote the set of active balls at the start of the mth round. A ball with centre (w, x) and radius r is defined as $\{(w', x') \in \mathcal{W} \times \mathcal{X} \mid d((w, x), (w', x')) < r\}$.

Each round, CZ applies two rules: a *selection rule*, which selects a ball B^m from A^m; and an *activation rule* which optionally adds a ball with smaller radius to A^m.

Given a context w^m, the selection rule proceeds as follows: let $N^m(B)$ be the number of times that a ball $B \in A^m$ has been selected in the first m rounds, and $\bar{y}^m(B)$ the average reward received over the first m rounds when ball B was selected.

$$N^m(B) = \sum_{i=1}^{m} \mathbb{1}[B = B^m], \tag{5.1}$$

$$\bar{y}^m(B) = \frac{1}{N^m(B)} \sum_{i=1}^{m} y^m \mathbb{1}[B = B^m]. \tag{5.2}$$

Let $r(B)$ denote the radius of ball B and let $\texttt{dom}^m(B)$ be the domain of ball B on the mth round, which is the subset of B that excludes all balls of smaller radius:

$$\texttt{dom}^m(B) = B - \left(\bigcup_{B' \in A^m : r(B') < r(B)} B' \right). \tag{5.3}$$

The *confidence radius* $\texttt{conf}^m(B)$ of ball B on the mth round is:

$$\texttt{conf}^m(B) = 4\sqrt{\frac{\log T}{1 + N^{m-1}(B)}}. \tag{5.4}$$

Let $\texttt{relevant}^m(w, A^m)$ denote the set of balls that are relevant for context w on the mth round, which is the set of balls that contain an arm x such that (w, x) is in the domain of the ball:

$$\texttt{relevant}^m(w, A^m) = \{B \in A^m | \exists x \in \mathcal{X}.(w, x) \in \texttt{dom}^m(B)\}. \tag{5.5}$$

The ball B^m selected on the mth round with context w^m is chosen as follows:

$$I^m(B) = \bar{y}^{m-1}(B) + r(B) + \texttt{conf}^m(B), \tag{5.6}$$

$$B^m = \underset{B \in \texttt{relevant}^m(w^m, A^m)}{\arg\max} \; r(B) + \min_{B' \in A^m} \left(I^m(B') + d(B, B') \right), \tag{5.7}$$

where $d(B, B')$ is the distance between the centres of balls B, B' according to the metric d.

Once a ball has been selected, the arm sampled by CZ, x^m_{CZ}, is sampled randomly from the domain of ball B^m (from the set $\{x \in \mathcal{X} | (w^m, x) \in \texttt{dom}^m(B^m)\}$).

The activation rule is applied when the confidence radius of the selected ball B^m is smaller than its radius, i.e. if $\texttt{conf}^{m+1}(B^m) \leq r(B^m) < \texttt{conf}^m(B^m)$. In this case, a new ball B' is added to the set of active balls, with radius $r(B') = \frac{1}{2}r(B^m)$ and centre (w^m, x^m). If a new ball is added, then $A^{m+1} = A^m \cup \{B'\}$ and otherwise $A^{m+1} = A^m$.

BD: list, brain dumping it out.

This chapter...

- Sec 5.1, introduction. Starting with existing MOMCTS methods discussed in Ch3 TODO: ref. These algorithms come in two flavours. One where a point estimate is maintained, and one where a convex hull is maintained. This section demonstrates why point estimates are not sufficient, and
- Sec 5.2, convex hull monte carlo tree search
- Sec 5.3, contextual zooming for trees
- Sec 5.5, results
- Sec 5.6, theoretical results

TODO: link back to questions and how we're answering them here

5.1 Introduction

TODO: list

- high level overview of CHMCTS work
- discuss how CHMCTS answers the research questions from introduction chapter
- moving into multi-objective land now

- Comment about CHVI and prior MOMCTS work motivating this

First prior multi-objective MCTS (MOMCTS) algorithms are discussed. Specifically,

BD: Point estimates insufficient can be copies from CHMCTS paper

BD: In prior methods that use a convex hull TODO: ref BD: , the environment is assumed to be deterministic, and backups can be seen to be a special case of Convex Hull backups from Chapter 2 TODO: ref. BD: Therefore, for the remainder of this chapter, most algorithms will use convex hull backups, similar to Convex Hull Value Iteration (todo ref), as in the deteminstic MOMDP case it reduces to the original algorithms, and extends them for stochastic MOMDPs. As such, this chapter, will predominantly focus on how perform action selection in MOMCTS (exploration?)

BD: this paper (below) talks about the need for making consistent action selections. Because the aim is to discover for all weights w an optimal policy. Given a w, actions should be taken consistently to try and maximise the objective.

BD: The hypervolume is the area under the convex hull. The optimal convex hull is the unique convex hull that maximised the hypervolume. The premise of some of the prior work is to maximise the hypervolume at every node. Or the premise is to select from the "optimal actions" that lie on the convex hull. However, reducing the problem back to discovering the optimal policies from the root node, this objective is a bit excessive. The optimal convex hull is not needed at non root nodes. TODO: wrote a bit about this in confirmation in a wordy way, copy that? BD: Consider a 2d problem, with two actions, which correspond to optimising each of the objectives. Once a step has been taken in the MOMDP, the agent has in some sence committed to a weighting over the objectives, and it does not make sense for the agent to then select actions to optimise the other objective, as if the agent wanted to optimise the other objective it would have made a different first step. This has also been observed by XXX et al, who do YYY to solve this issue. In this work, a different solution is considered, by turning the problem into a contextual one.

TODO: want details in contextual tree search, but want to link to it from here. The detail above really should go below in sec 5.1.1

BD: This example, and an example from XXX et al, are given in subsection TODO ref, to concretely demonstrate these issues.

TODO: Demonstrate the need for consistent action selection according to objectives. This paper talks about the same issue: https://jmlr.org/papers/volume15/vanmoffaert14a/vanmoffa so should read, cite and discuss. They essentially use some policy tracking

5.1.1 Contextual Tree Search

TODO: Talk about how values can be extracted from the convex hull objects given a context, and how this can be used to solve this consistent action problem. Refer back to section 2, and say that for decision support, when a policy is chosen, it implies a weight vector, which can be used to extract the policies.

TODO: use this to highlight that the intuition, of maximising the hypervolume or trying to find an optimal convex hull at every node is unnecessary. And leads to suboptimal behaviour and exploration.

TODO: Use the simple example of reward in dim 1 for action 1 and reward in dim 2 for action 2

TODO: Also make a simple 3 step example demonstrating what the vanmoffaert paper does, where there is a joint state on the two optimal paths. Which is an example of just picking actions from even the optimal convex hull at each step does not mean that you are following an optimal policy.

BD: So the main things to highlight with these two examples. One, that taking the objective to compute each local convex hull (the full convex hulls at not root node) is not necessary, and leads to suboptimal exploration. Two, that even if you are given all of the optimal convex hulls at every state, that some form of context or history is required to follow a globally optimal policy.

TODO: Argue that in the case of decision support, if a context/weight vector can be obtained from the selection of a policy by the user, then the weight vector is a natural choice (and sufficient) for the context. As once given a context, the optimal choice is defined at every step. (And reference the theory section (and quote the result here).)

TODO: Front load results on the 2 action tree search. Demonstrate that the prior works dont do well on it, and show that ours does.

TODO: Introduce contextual (simple) regret

TODO: Below is brain dumping this section on 22 feb. Didn't look at CHMCTS paper at all when writing it.

This subsection discusses the use of *contextual tree search* for planning in multi-objective settings. BD: And highlights why any multi-objective tree search algorithm needs to be contextual to run effectively through an example MOMDP.

TODO: Example MOMDP where the rewards are a circle. Have an image of the rewards, have an example of the 2 action 2D, and 3 action 2D envioronments. Also want to cite the EUM point generation that we used to generate points uniformly on a hypersphere, which allows the environment to be generate for an arbitrary (num actions, dimension) pair

In previous work TODO: refs back to litrev, the rough intuition used is at every node in the tree that a full convex hull is desirable, and maps action selection in a variety of ways to try and achieve this.

Using this example MOMDP TODO: reffering to the 3d one?, where there are M actions, it is quite clear from inspection what policies form the Convex Hull. $\phi^i(s) = a_{\mathcal{M}(i)}$ is the optimal policy for the BD: weight vector $\mathbf{w}^i = \mathbf{R}(\cdot, a_{\mathcal{M}(i)})$ TODO: probably want to fix up notation. The convex hull is from inspection $\{\phi^i | 1 \leq i \leq M\}$.

BD: The point of this MOMDP is that really is that its easy to know optimal CH from inspection, and should be relatively easy to solve for any tree search algorithm.

To motivate why contextual tree search is a necessary component of MOMCTS, consider $s_0 \to a_0 \to s_1$ TODO: clean notation and that from state s_1 onwards, we know that the optimal action must be to continue taking a_0 until the trial ends. In some sense, the actions for the remainder of the trial should be consistent with any actions previously taken in the trial. Otherwise, the search policy will be essentially flipping between trying to optimse for different combinations of objectives and in effect optimising for none of them. TODO: This paragraph very brain dump.

The proposal in this thesis is for every trial to sample a *context* weight vector, where for now it will be assumed that $\mathbf{w}^i \sim U(\Delta^D)$ TODO: just realised the Delta in BTS section might be overloaded notation? ALSO double check what notation used for simplex. ALso clean this equation up. This context is then used to allow algorithms to follow a consistent objective over all actions in a trial. TODO: There may be other ways to achieve this, but the proposal of contextual tree search solves this problem relatively simply

TODO: Some experiments showing that the prior work on MOMCTS fails horribly on this MOMDP.

BD: As the work in this chapter will follow the typical pattern of taking a multi-armed bandit problem and applying it to a sequential decision making problem, the theoretical quantity that will be considered in this chapter is contextual (cumulative) regret.

TODO: Define the typical MAB problem figure, and contextual regret equation.

TODO: HERE HERE HERE HERE. NExt = write up the typical MAB problem figure and defn contextual regret. With new command local. Then move onto defining contextual zooming for trees and CHMCTS. Doesn't matter if the notation is a bit off, as long as the equations which are slow to type out are typed out.

TODO: make defn Given that the multi-objective problem is now being posed as a contextual problem, it is necessary to allow for policies to depend on the context weight. Given a weight $\mathbf{w} \in \Delta^D$, the probability of samping action a from policy π at state s will now be written $\pi(a|s;\mathbf{w})$.

Additionally, *context dependent* policy will refer to policies that can vary for different weights, and *context independent* policies are ones that give the same distribution for all weights. That is, a policy π is context independent if $\pi(a|s;\mathbf{w}) = \pi(a|s;\mathbf{w}')$ for all $\mathbf{w}, \mathbf{w}' \in \Delta^D$. TODO: How to handle that context dependent could learn a context independent? Maybe rephrase?

5.2 Contextual Zooming for Trees

TODO: part 1: say that a common approach from tree search is to think about each node as solving a (non-stationary) multi-armed bandit problem

TODO: part 2: following a similar approach, contextual zooming is an approach for contextual bandits

TODO: part 3: have a diagram of a box, UCB right to UCT, UCB down to CZ, UCT down to CZT, CZ right arrow to CZT

TODO: part 4: describe the algorithm. say that it's for a fixed number of trials. talk about doubling trick and how it can be used to bound contextual regret for unknown number of trials. In practise just use the equations without thinking about it.

TODO: part 5: subsection on CH-CZT, which is just using the convex hull backups for the recommendation policy.

A common design choice in tree search for sequential decision making is to use a relevant multi-armed bandit algoroithm at each deicison node of the tree. For example, the UCT algorithm adapts the UCB algorithm at each node, to run on a non-stationary MAB problem at each node.

Since Section **??** motivated viewing multi-objective MCTS from a contextual viewpoint. This Section considers utilising the *Contextual Zooming* alogirithm TODO: ref in ch2 for contextual multi-armed bandits to give the *Contextual Zooming for Trees* (CZT) algorithm.

BD: Following a similar methodology to UCT — which runs UCB on a non-stationary multi-armed bandit problem at each node in the search tree — *Contextual Zooming for Trees* (CZT) runs *Contextual Zooming* (Chapter 2, TODO REF) on a non-stationary contextual multi-armed bandit problem at each node in the search tree.

It's worth noting that because the CZT algorithm is following the "UCT formula", and that the contextual zooming algorithm is motivated in the BD: standard reinforcement learning setting, as opposed to the exploration setting. BD: As such CZT in some sense is not designed with the exploration setting in mind.

TODO: Consdier this algo for two reasons: follow same design as UCT and contextual search policy baseline (as non exist really).

TODO: Box diagram

TODO: Revise contextual zooming algorithm

5.3 Convex Hull Monte Carlo Tree Search

TODO: part 1: give generalised CHMCTS backups

TODO: part 2: say that this generalises the prior works from ch3

TODO: part 3: state the search policies of the prior works, and label them

TODO: part 4: dont know if can be bothered to implement, but updating N(s) by w and making it a visitation vector now makes more sense than just a count.

This section introduces *Convex Hull Monte Carlo Tree Search* (CHMCTS), which generalises the prior work in multi-objective MCTS TODO: ref ch3 to handle stochastic environments by using backups over *Convex Hull Value Sets* from Chapter 2 TODO: ref ch2. First the backups, which will be common across all of the following algorithms.

Let ALG be a substitute acronym for any of the following algorithms. At each decision node s a value set is maintained $\hat{\boldsymbol{\mathcal{V}}}_{\text{ALG}}(s)$, and at chance nodes (s, a) the value set $\hat{\boldsymbol{\mathcal{Q}}}_{\text{ALG}}(s)$ is maintained. The search policy, $\pi_{\text{ALG}}(\cdot|s, \mathbf{w})$, is *contextual* and will depend on a randomly sampled $\mathbf{w} \in \Delta^D$ for each trial. BD: Assume for now that the search policy is defined, as it will differ for each algorithm.

Given a context objective $\mathbf{w} \in \Delta^D$ and a trajectory $\tau = (s_0, a_0, \mathbf{r}_0, ..., s_{h-1}, a_{h-1}, \mathbf{r}_{h-1}, s_h) \sim \pi_{\text{ALG}}$ TODO: want the w in the pialg bit somehow? Maybe make it superscript? But dont want it to mess with notation for theory. the value estimates are updated for $t = h - 1, ..., 0$:

$$\hat{\boldsymbol{\mathcal{V}}}_{\text{ALG}}(s_t) \leftarrow \texttt{cvx_prune}\left(\bigcup_{a_t \in \mathcal{A}} \hat{\boldsymbol{\mathcal{Q}}}_{\text{ALG}}(s_t, a_t)\right), \tag{5.8}$$

$$\hat{\boldsymbol{\mathcal{Q}}}_{\text{ALG}}(s_t, a_t) \leftarrow \mathbf{R}(s_t, a_t) + \sum_{s' \in \text{Succ}(s_t, a_t)} \frac{N(s')}{N(s_t, a_t)} \hat{\boldsymbol{\mathcal{V}}}_{\text{ALG}}(s'). \tag{5.9}$$

where the `cvx_prune` operation is defined in (**??**), BD: but repeated here for clarity over an arbitrary set of vectors $\boldsymbol{\mathcal{V}}$:

$$\texttt{cvx_prune}(\boldsymbol{\mathcal{V}}) = \{\mathbf{v} \in \boldsymbol{\mathcal{V}} | \exists \mathbf{w} \in \Delta^D . \forall \mathbf{v}' \in \boldsymbol{\mathcal{V}} - \{\mathbf{v}\} . \mathbf{w}^\top \mathbf{v} > \mathbf{w}^\top \mathbf{v}'\}. \tag{5.10}$$

Additionally, because the search policies are contextual TODO: in the following subsections (todo ref), it will be useful to define *contextual value estimates*, given a context objective $\mathbf{w} \in \Delta^D$ as follows:

$$\hat{V}^{\mathbf{w}}_{\text{ALG}}(s) = \max\{\mathbf{w}^\top \mathbf{v} | \mathbf{v} \in \hat{\boldsymbol{\mathcal{V}}}_{\text{ALG}}(s)\}, \tag{5.11}$$

$$\hat{Q}^{\mathbf{w}}_{\text{ALG}}(s, a) = \max\{\mathbf{w}^\top \mathbf{q} | \mathbf{q} \in \hat{\boldsymbol{\mathcal{Q}}}_{\text{ALG}}(s)\}. \tag{5.12}$$

TODO: define the policies for the algorithms in chapter 3. also think want to justify the equivalence, because they will use some $\bar{\mathbf{R}}$ TODO: in the updates to the convex hulls. Decompose $\bar{\mathbf{R}}(s_t) = \mathbf{R}(s_t, a_t) + \bar{\mathbf{R}}(s_{t+1})$, and say that this would only get added to CHVS at s_t iff at $t+1$ the second part was added to the convex hull.

TODO: should just write up the ch3 stuff before writing this tidbit

The algorithms described in Chapter 3 TODO: ref can be realised using the above backups, with the search policies taking the following form:

$$\pi_{\text{ALG}}(s|\mathbf{w}) = \arg\max_{a \in \mathcal{A}} \left[\zeta_{\text{ALG}}(\boldsymbol{\mathcal{Q}}(s, a)) + c_{\text{ALG}} \sqrt{\frac{\log(N(s))}{N(s, a)}} \right] \tag{5.13}$$

TODO: write up the policies for the ch3 algorithms by defining zeta for the different algorithms. And highlight the context independence from not depending on the weight

TODO: correct that pareto ucb uses a different search policy too

Below = previous writing on this

TODO: list

- Give convex hull monte carlo tree search
- Contextual zooming with the convex hull backups

TODO: Discuss property of extracting from convex hull

TODO: A range of action selection methods will be used.

Now that contextual tree search has been motivated, *Convex Hull Monte Carlo Tree Search* is given in a form that generalises the works of TODO: cite lots.

TODO: CHMCTS backups

TODO: Contextual Search policy

TODO: Restate and name the prior works search policies

The algorithms considered in TODO: ref ch4 can be converted into contextual multi-objective algorithms by using the contextual values $\hat{V}^{\mathbf{w}}_{\text{ALG}}(s)$ and $\hat{Q}^{\mathbf{w}}_{\text{ALG}}(s,a)$ for decision making.

5.3.1 CH-CZT

TODO: move

Additionally, there is nothing preventing CZT from keeping convex hull value set estimates at each of it's nodes. This algorithm is referred to as Contextual Zooming for Trees with Convex Hull backups (CH-CZT). In such a case, the backups from TODO: ref eqns are included, to update the value set estimates TODO: notation, which are then used for recommnedations TODO: as follows. In essence, this algorithm runs CZT, but adds an aditional convex hull backup which is used for recommendations.

TODO: this should mostly be copying out from the chmcts paper.

5.3.2 CH-UCT

TODO: part 1: give action selection for CH-UCT and talk about it

TODO: part 2: discuss more?

A natural way to extend the UCT search policy for contextual tree search is to use the contextual value estimates (**??**) and (**??**) in the UCT search policy. Call this algorithm *UCT with Convex Hulls* (CH-UCT). This gives the following search policy:

$$\pi_{\text{CH-UCT}}(s|\mathbf{w}) = \arg\max_{a\in\mathcal{A}} \left[\hat{Q}^{\mathbf{w}}_{\text{CH-UCT}}(s,a) + c_{\text{CH-UCT}} \sqrt{\frac{\log(N(s))}{N(s,a)}} \right]. \tag{5.14}$$

BD: One issue is that the confidence interval terms are shared between all possible context objectives, and may assign a high confident, or tight bound, for contexts that haven't been explored down this path much. A hueristic way to solve this is to a vectorised visit counts $\mathbf{M}(s)$, $\mathbf{M}(s,a)$. These values are backed up after a trial for context $\mathbf{w}$ as follows:

$$\mathbf{M}(s) \leftarrow \mathbf{M}(s) + \mathbf{w}, \tag{5.15}$$

$$\mathbf{M}(s,a) \leftarrow \mathbf{M}(s,a) + \mathbf{w}. \tag{5.16}$$

Then a contextual visit count can be defined, and used in the search policy as follows:

$$M(s;\mathbf{w}) = \mathbf{w}^\top \mathbf{M}(s) \tag{5.17}$$

$$M(s,a;\mathbf{w}) = \mathbf{w}^\top \mathbf{M}(s,a) \tag{5.18}$$

$$\pi_{\text{CH-UCT}}(s|\mathbf{w}) = \arg\max_{a\in\mathcal{A}} \left[\hat{Q}^{\mathbf{w}}_{\text{CH-UCT}}(s,a) + c_{\text{CH-UCT}} \sqrt{\frac{\log(M(s;\mathbf{w}))}{M(s,a;\mathbf{w})}} \right]. \tag{5.19}$$

Note that the original value of $N(s)$ is still needed in the convex hull backups for Q-value sets, to empirically estimate the transition distribution $p(s'|s,a) \approx \frac{N(s')}{N(s,a)}$, however this can be recovered from the vectorised visit count by summing over the values of the vector:

$$N(s) = \sum_{i=1}^{D} M(s)_i \tag{5.20}$$

$$N(s,a) = \sum_{i=1}^{D} M(s,a)_i. \tag{5.21}$$

TODO: notation above isn't clean.

TODO: Why doesnt a CH-UCT withe confidence bounds around the DP values you get from the convex hulls still work? Couldn't we do a k-d tree of visit counts for contexts, or some other way of counting how often a context has been searched for? There would be some lemma that the number of visits, is correct, for some epsilon ball around the weight, which the epsilon gives a small enough error in the value to be fine, except for the visit from the nodes above it in the kd tree. ANd then require exp many more trials to count for each level of the tree.

TODO: Add to ORIMenu to just submit some arxiv shit for something like "an addendum to convex hull monte carlo tree search", where we basically copy past this chapter. Actually, I think just take this chapter and publish it with the definitions cut down. Say for litreview either look at the CHMCTS or Thesis literature reviews.

TODO: Quote theory results

5.3.3 CH-BTS

TODO: part 1: give action selection for CH-DENTS and talk about it

TODO: part 2: say CH-BTS is same without entropy backup

TODO: part 2: discuss entropy, say that it's just an approximate exploration bonus now

5.3.4 CH-DENTS

TODO: part 1: give action selection for CH-DENTS and talk about it

TODO: part 2: say CH-BTS is same without entropy backup

TODO: part 2: discuss entropy, say that it's just an approximate exploration bonus now

5.4 Results

TODO: part 0: discuss the evaluation prodceedure, and how the results are evaluated.

TODO: part 1: run algorithms on the tree envs that show the context dependent stuff better

TODO: part 1a: run algorithms with and without vectorised visit counts and show this helps with exploration?

TODO: part 2: run on suite of mo-gym environments

TODO: part 3: find environments where the convex hull blows up to motivate ch6. Give the CH fully and plot it out.

TODO: part 3a: also consider showing some of the optimal convex hulls where possible

TODO: list

- Results from CHMCTS paper
- Get same plots from C++ code, but compare expected utility, rather than the confusing hypervolume ratio stuff

BD: A range of algorithms will be considered in this experimental section. Firstly, for baselines, the convex hull backup versions of algorithms from Ch3 (REF) will be used. Additionally, CZT, CH-CZT, CH-UCT, CH-BTS and CH-DENTS will be considered

This section provides an empirical evalutation of the algorithms developed in this chapter. The baseline algorithms considered will also be CHMCTS algorithms, using the action selection methods outline in Sections TODO: ref litrev and above.

BD: This para is very braindump. It is worth noting that in multi-objective MCTS algorithms that the accuracy and quality of the convex hull at the root node is the primary concern, even when a multi-objective MCTS algorithm is used in an *online* manor, where tree search is interleaved between actions taken by the agent. Once a policy has been selected at the initial state it implicitly defines a weighting over the objectives as detailed in Section TODO: ref sec 2. And following the discussion from TODO: ref above, to follow an optimal policy the actions need to be taken consistently with respect to the same objective weights. As such, a more efficient online algorithm will be to run the multi-objective tree search from the initial state, and run a single-objective MCTS algorithm in a scalarised single-objective MDP for the remaining actions. Where in the scalarised reward is $R(s, a; \mathbf{w}) = \mathbf{w}^\top \mathbf{R}(s, a)$.

5.4.1 Evaluation Proceedure

To evaluate the quality of the convex hull at the root node, the following metrics from the multi-objective literature [**?**] are considered: *Expected Utility Metric*, *Hypervolume Metric* and a *Sparsity Metric*. TODO: Add EUM and HV and Sparsity here or in ch2?

To evaluate the quality of the convex hull at the root node, where the optimal convex hull is not known a-priori, the *Expected Utility Metric*,

Firstly, given a context $\mathbf{w}$ and CHMCTS search tree $\mathcal{T}$ that defines a partial recommendation poilicy ψ_{alg}, the policy will be made complete similarly to Section TODO: ref eval proceedure in ch4:

$$\texttt{complete}(\psi_{\text{alg}})(a|s;\mathbf{w}) = \begin{cases} 1 & \text{if } s \in \mathcal{T} \text{ and } a = \psi_{\text{alg}}(s;\mathbf{w}), \\ 0 & \text{if } s \in \mathcal{T} \text{ and } a \neq \psi_{\text{alg}}(s;\mathbf{w}), \\ \frac{1}{|\mathcal{A}|} & \text{otherwise.} \end{cases} \tag{5.22}$$

TODO: should reword to account for CZT and how it does the same thing

The Convex Hull (of policies) can then be written as

$$\Psi_{\text{alg}} = \{\texttt{complete}(\psi_{\text{alg}}) \mid \mathbf{w} \in \Delta^D\}. \tag{5.23}$$

This Convex Hull of policies is then evaluated using the *Expected Utility Metric* (EUM), which is defined as:

$$\text{EUM}(\Psi_{\text{alg}}) = \mathbb{E}_{\mathbf{w}\sim U(\Delta^D)}\left[\max_{\psi\in\Psi_{\text{alg}}} \mathbf{w}^\top \mathbf{V}^\psi(s_0)\right], \tag{5.24}$$

where $U(\Delta^D)$ is the uniform distribution over vectors in Δ^D. A monte-carlo estimate of $\text{EUM}(\Psi_{\text{alg}})$ will be computed, following TODO: cite the deep MORL paper, let $E(\Delta^D)$ be an evenly spaced set of points in Δ^D. Then the EUM can be approximated as:

$$\text{EUM}(\Psi_{\text{alg}}) \approx \sum_{\mathbf{w}\in E(\Delta^D)} \mathbf{w}^\top \mathbf{V}^{\tau(\mathbf{w})}, \tag{5.25}$$

where $\tau(\mathbf{w}) \sim \texttt{complete}(\psi_{\text{alg}})$ is a sampled trajectory for the point $\mathbf{w} \in \Delta^D$, and $\mathbf{V}^{\tau(\mathbf{w})}$ is the cumulative return from the trajectory. That is, if TODO: double check indices are consistent with how been doing it before the trajectory is $\tau(\mathbf{w}) = (s_0, a_0, \mathbf{r}_0, ..., s_{h-1}, a_{h-1}, \mathbf{r}_{h-1}, s_h)$, then $\mathbf{V}^{\tau(\mathbf{w})} = \sum_{i=0}^{h-1} \mathbf{r}_i$.

For the results given in TODO: ref, the number of sampled used to approximate the EUM was XXX. That is $|E(\Delta^D)| = XXX$.

TODO: double check this is consistent with the litrev paper on MORL and also cite it. Also what are the other metrics?

Experiments are run with `single_node_expansion` turned off, as no memory issues were encountered during the evalutation.

Similarly to the gridworld environments in Chapter TODO: ref, the BayesOpt package [**?**] was used to tune the hyperparameters of all algorithms in this chapter. Full details on the hyperparameter tuning and the selected parameters are given in Appendix TODO: new appendix? join the appendix?.

5.4.2 Environments

The MO-Gymnasium TODO: cite suite of environments was used to evaluate the algorithms.

TODO: Pick a couple envs to describe in more detail, and refer to cite for full details

Additionally, the Deep Sea Treasure is used, and the TODO: extended? (whatever the updated one is) are used. TODO: Also mention that we can generate maps of varying size to perform a scalability analysis

5.4.3 Results and Discussion

TODO: Paste in results and talk about them

5.5 Theory

TODO: part 1: all the (relevant) search policies will select actions infinitely often.

TODO: part 1a: BTS and DENTS follows from ch4. For UCT can argue that if select one argument a finite number of times. Then use the confidence interval term to arrive at contradiction

TODO: part 2: convex hull backups perform the scalarised backups for all possible weights over the objective.

TODO: write up proofs wrote up on ipad. Used this gpt convo in cleaning up details: https://chatgpt.com/c/67fa9295-0cb8-8007-88fd-5bec1ac5393b

TODO: part 2a: refer back to convergence of backups in ch4, clean up and DKW stuff need to

TODO: part 3: if values converge to optimal, then contextual simple regret tends to zero

TODO: part 4: mention that CZT proofs would be much more complex, and given the other algorithms do better. Mention a lower bound which comes from the CZ regret bound. And that this algorithm could be preferable in not in exploring context.

TODO: part 4a: similar with CH-UCT to CZT

TODO: part 5: any proofs for CH-UCT and/or CZT? Which wouldn't be a disaster trying to proove?

Below is what we initially wrote in this section.

TODO: See if with chatgpt help can crack out a rough proof outline? Because assuming no noise in the reward function, the leaf nodes clearly converge with same rate. Just really need the induction step. Feel like it should be very do-able, by bootstrapping on the CZT proofs.

TODO: Can easily show that CH converges, by using every action sampled infinitly often still, and bootstrap off the convex hull convergence proof.

TODO: One thing need to prove is that our form of backups converges, because we're using the sample averages our backups are slightly different. Should

TODO: Remainder of properties bootstrap off the proofs in Ch4. Every action is sampled inf often, hence the convex hull backups converge. Don't bother showing convergence rates. Just show that they are consistent.

TODO: Also want to show that given any convex hull value estimates, that following the policy extracted from convex hulls does give the same multi-objective value.

basically this would say

$$\pi(s;w) = \arg\max_{a\in\mathcal{A}} \circ\mathbf{Q}(s,a;w) \tag{5.26}$$

$$w^\top \mathbf{V}^\pi(s;w) = w^\top \mathbb{E}_{a\sim\pi}[\mathbf{Q}^\pi(s,a;w)] \tag{5.27}$$

$$= \circ\mathbf{Q}(s,a;w) \tag{5.28}$$

$$w^\top \mathbf{Q}^\pi(s,a;w) = ... \tag{5.29}$$

So say that the value of the policy by taking the maimum from value estimates, will achieve the value estimates. We've been handwavy around empirical distriubtion vs actual transition distribution. But basically the argument is inductive, and holds trivially for leaf nodes.

6

Simplex Maps for Multi-Objective Monte Carlo Tree Search

Contents

6.1 Introduction **124**
6.2 Simplex Maps **124**
6.3 Simplex Maps in Tree Search **125**
6.4 Theoretical Results **125**
6.5 Empirical Results **125**

New Ch5 plan:

- Intro, introduce and motivate simplex maps, motivate by move to weight space, more efficient lookups and ability to use entropy again
- Define simplex map interface and implementation
- Discussion around the complexity of simplex maps
- Use a hypercube example to illustrate that the neighbourhood can still be with respect to dimension and this is seems unnavoidable in general
- However, lookups are much more efficient in practise

- And sharing values between vertices will cut out a lot of computation when there is no improvement
- Algos, BTS and DENTS using simplex maps as a baseline
- Empirical results
- Theory, contextual regret, non-contextual policies garunteed to have a linear contextual regret
- Maybe some theory, probably bootstrap of the ch4 + ch5 results

6.1 Introduction

TODO: list

- high level overview of simplex maps work
- discuss how simplex maps answer the research questions from introduction chapter
- staying in multi-objective land now
- Motivated by CHMCTS being slow

6.2 Simplex Maps

TODO: list

- Define simplex map interface
- Give details on how to efficiently implement the interface with tree structures
- (Good diagram is everything here I think)

6.3 Simplex Maps in Tree Search

TODO: list

- Come up with better title for section
- Use simplex maps interface to create algorithms from the dents work
- Give a high level idea of what δ parameter is (used in theory section)

6.4 Theoretical Results

TODO: list

- Convergence can build ontop of DENTS results
- Runtime bounds (better than $O(2^D)$ which is what using convex hulls has)
- Simplex map has a diameter δ (i.e. the furthest away a new context could be from a point in the map)
- Bounds can then come from that diameter (which is a parameter of the simplex map/algorithm) and DENTS results

6.5 Empirical Results

TODO: list

- Results from MO-Gymnasium
- Compare algorithms using expected utility

7
Conclusion

Contents

7.1 Summary of Contributions 127
7.2 Future Work 127

TODO: Something about we'll conclude by looking back at contributions and possible future work.

7.1 Summary of Contributions

TODO: go through each of the research questions and contributions, and write about how the work answers the research questions

7.2 Future Work

TODO: outline some avenues of potential future work

- Decaying TENTS, the idea can be applied directly to Tsallis entropy
- Applying DENTS logic to deep learning
- Applying BTS/DENTS in alpha zero style loops

- value curves instead of convex hulls for where convex hull is better than simplex map
- adding entropy as an additional objective in simplex maps work
- multi-objective alpha zero (simplex map/convex hull), cite some of the work about catastrophic forgetting where learned policy $pi(a|s;w)$ degrades to the same policy for many weights w, maybe ensuring that the policy is updated for many weights w in the loss could help with this

TODO: Also mention the people that have used/cited DENTS work

Appendices

A

Additional Algorithm Details For Chapter 4

This appendix gives the full equations for AR-BTS and AR-DENTS for reference. Each algorithm is specified minimally using the `THTS++` schema given in Section **??**.

A.1 AR-BTS

Value Estimates and Parameters.

- $\bar{V}_{\text{AR-BTS}}(s)$ - The value estimate for state s;
- $\bar{Q}_{\text{AR-BTS}}(s, a)$ - The Q-value estimate for state s and action a;
- $N(s)$ - The visit count for state s;
- $N(s, a)$ - The visit count for state s and action a;
- $\alpha_{\text{AR-BTS}}(x) > 0$ - The search temperature function;
- $\epsilon_{\text{AR-BTS}} \in [0, \infty)$ - The uniform exploration parameter;
- `normalised_Q` - A flag indicating whether the Q-values should be normalised when used in the search policy;
- $N_{\text{AR-BTS}}^{\hat{V}_{\text{heuristic}}}$ - The number of pseudo-visits to use for biasing the backups;

- $N_{\text{AR-BTS}}^{\hat{Q}_{\text{heuristic}}}$ - The number of pseudo-visits to use for biasing the Q-values;
- $\hat{V}_{\text{heuristic}}(s)$ - The heuristic value estimate for state s;
- $\hat{Q}_{\text{heuristic}}(s,a)$ - The heuristic Q-value estimate for state s and action a;

Initialisation.

$$N(s) \leftarrow 0, \tag{A.1}$$

$$N(s,a) \leftarrow 0, \tag{A.2}$$

$$\bar{V}_{\text{AR-BTS}}(s) \leftarrow 0, \tag{A.3}$$

$$\bar{Q}_{\text{AR-BTS}}(s,a) \leftarrow 0. \tag{A.4}$$

Search Policy. Intermediate Q-values:

$$\bar{Q}_{\text{AR-BTS}}^{\text{bias}}(s,a) = \frac{N(s,a)\bar{Q}_{\text{AR-BTS}}(s,a) + N_{\text{AR-BTS}}^{\hat{Q}_{\text{heuristic}}}\hat{Q}_{\text{heuristic}}(s,a)}{N(s,a) + N_{\text{AR-BTS}}^{\hat{Q}_{\text{heuristic}}}}, \tag{A.5}$$

$$\hat{Q}_{\text{BTS}}^{\text{norm}}(s,a) = \begin{cases} \frac{\bar{Q}_{\text{AR-BTS}}^{\text{bias}}(s,a) - \min_{a\in\mathcal{A}} \bar{Q}_{\text{AR-BTS}}^{\text{bias}}(s,a)}{\max_{a\in\mathcal{A}} \bar{Q}_{\text{AR-BTS}}^{\text{bias}}(s,a) - \min_{a\in\mathcal{A}} \bar{Q}_{\text{AR-BTS}}^{\text{bias}}(s,a)} & \text{if } \texttt{normalised_Q}, \\ \bar{Q}_{\text{AR-BTS}}^{\text{bias}}(s,a) & \text{otherwise.} \end{cases} \tag{A.6}$$

Search policy:

$$\pi_{\text{AR-BTS}}(a|s) = (1 - \lambda(s, \epsilon_{\text{AR-BTS}}))\rho_{\text{AR-BTS}}(a|s) + \frac{\lambda(s, \epsilon_{\text{AR-BTS}})}{|\mathcal{A}|}, \tag{A.7}$$

$$\rho_{\text{AR-BTS}}(a|s) \propto \exp\left(\frac{\bar{Q}_{\text{AR-BTS}}^{\text{norm}}(s,a)}{\alpha_{\text{AR-BTS}}(N(s))}\right), \tag{A.8}$$

$$\lambda(s, \epsilon_{\text{AR-BTS}}) = \min\left(1, \frac{\epsilon_{\text{AR-BTS}}}{\log(e + N(s))}\right). \tag{A.9}$$

Backups. Let $\tau_{:h} = (s_0, a_0, r_0, \ldots, s_{h-1}, a_{h-1}, r_{h-1}, s_h) \sim \pi_{\text{AR-BTS}}$:

$$\bar{Q}_{\text{AR-BTS}}(s_t, a_t) \leftarrow R(s_t, a_t) + \sum_{s' \in \text{Succ}(s_t, a_t)} \left(\frac{N(s')}{N(s_t, a_t)} \bar{V}_{\text{AR-BTS}}(s')\right), \tag{A.10}$$

$$\bar{V}_{\text{AR-BTS}}(s_t) \leftarrow \frac{N_{\text{AR-BTS}}^{\hat{V}_{\text{heuristic}}}\hat{V}_{\text{heuristic}}(s_t)}{N(s_t) + N_{\text{AR-BTS}}^{\hat{V}_{\text{heuristic}}}} + \sum_{a\in\mathcal{A}} \frac{N(s_t, a)}{N(s_t) + N_{\text{AR-BTS}}^{\hat{V}_{\text{heuristic}}}} \bar{Q}_{\text{AR-BTS}}(s_t, a). \tag{A.11}$$

Recommendation Policy.

$$\psi_{\text{AR-BTS}}(s) = \arg\max_{a \in \mathcal{A}} \bar{Q}_{\text{AR-BTS}}(s, a), \tag{A.12}$$

$$\texttt{mv}_{\text{AR-BTS}}(s) = \arg\max_{a \in \mathcal{A}} N(s, a). \tag{A.13}$$

A.2 AR-DENTS

Value Estimates and Parameters.

- $\bar{V}_{\text{AR-DNTS}}(s)$ - The value estimate for state s;
- $\bar{Q}_{\text{AR-DNTS}}(s, a)$ - The Q-value estimate for state s and action a;
- $N(s)$ - The visit count for state s;
- $N(s, a)$ - The visit count for state s and action a;
- $\alpha_{\text{AR-DNTS}}(x) > 0$ - The search temperature function;
- $\epsilon_{\text{AR-DNTS}} \in [0, \infty)$ - The uniform exploration parameter;
- `normalised_Q` - A flag indicating whether the Q-values should be normalised when used in the search policy;
- $N_{\text{AR-DNTS}}^{\hat{V}_{\text{heuristic}}}$ - The number of pseudo-visits to use for biasing the backups;
- $N_{\text{AR-DNTS}}^{\hat{Q}_{\text{heuristic}}}$ - The number of pseudo-visits to use for biasing the Q-values;
- $\hat{V}_{\text{heuristic}}(s)$ - The heuristic value estimate for state s;
- $\hat{Q}_{\text{heuristic}}(s, a)$ - The heuristic Q-value estimate for state s and action a;

Initialisation.

$$N(s) \leftarrow 0, \tag{A.14}$$

$$N(s, a) \leftarrow 0, \tag{A.15}$$

$$\bar{V}_{\text{AR-DNTS}}(s) \leftarrow 0, \tag{A.16}$$

$$\bar{Q}_{\text{AR-DNTS}}(s, a) \leftarrow 0. \tag{A.17}$$

Search Policy. Intermediate Q-values:

$$\hat{Q}^{\text{sft}}_{\text{AR-DNTS}}(s,a) = \bar{Q}_{\text{AR-DNTS}}(s,a) + \beta_{\text{AR-DNTS}}(N(s))\bar{\mathcal{H}}_{Q,\text{AR-DNTS}}(s,a), \tag{A.18}$$

$$\hat{Q}^{\text{sft,bias}}_{\text{AR-DNTS}}(s,a) = \frac{N(s,a)\hat{Q}^{\text{sft}}_{\text{AR-DNTS}}(s,a) + N^{\hat{Q}_{\text{heuristic}}}_{\text{AR-DNTS}}\hat{Q}_{\text{heuristic}}(s,a)}{N(s,a) + N^{\hat{Q}_{\text{heuristic}}}_{\text{AR-DNTS}}}, \tag{A.19}$$

$$\hat{Q}^{\text{sft,norm}}_{\text{AR-DNTS}}(s,a) = \begin{cases} \frac{\hat{Q}^{\text{sft,bias}}_{\text{AR-DNTS}}(s,a) - \min_{a\in\mathcal{A}} \hat{Q}^{\text{sft,bias}}_{\text{AR-DNTS}}(s,a)}{\max_{a\in\mathcal{A}} \hat{Q}^{\text{sft,bias}}_{\text{AR-DNTS}}(s,a) - \min_{a\in\mathcal{A}} \hat{Q}^{\text{sft,bias}}_{\text{AR-DNTS}}(s,a)} & \text{if } \texttt{normalised_Q}, \\ \hat{Q}^{\text{sft,bias}}_{\text{AR-DNTS}}(s,a) & \text{otherwise.} \end{cases} \tag{A.20}$$

Search policy:

$$\pi_{\text{AR-DNTS}}(a|s) = (1 - \lambda(s, \epsilon_{\text{AR-DNTS}}))\rho_{\text{AR-DNTS}}(a|s) + \frac{\lambda(s, \epsilon_{\text{AR-DNTS}})}{|\mathcal{A}|}, \tag{A.21}$$

$$\rho_{\text{AR-DNTS}}(a|s) \propto \exp\left(\frac{\hat{Q}^{\text{sft,norm}}_{\text{AR-DNTS}}(s,a)}{\alpha_{\text{AR-DNTS}}(N(s))}\right), \tag{A.22}$$

$$\lambda(s, \epsilon_{\text{AR-DNTS}}) = \min\left(1, \frac{\epsilon_{\text{AR-DNTS}}}{\log(e + N(s))}\right). \tag{A.23}$$

Backups. Let $\tau_{:h} = (s_0, a_0, r_0, \ldots, s_{h-1}, a_{h-1}, r_{h-1}, s_h) \sim \pi_{\text{AR-DNTS}}$:

$$\bar{\mathcal{H}}_{Q,\text{AR-DNTS}}(s_t, a_t) \leftarrow \sum_{s'\in\text{Succ}(s_t,a_t)} \frac{N(s')}{N(s_t,a_t)} \bar{\mathcal{H}}_{V,\text{AR-DNTS}}(s'), \tag{A.24}$$

$$\bar{\mathcal{H}}_{V,\text{AR-DNTS}}(s_t) \leftarrow \mathcal{H}(\pi_{\text{AR-DNTS}}(\cdot|s_t)) + \sum_{a\in\mathcal{A}} \pi_{\text{AR-DNTS}}(a|s_t)\bar{\mathcal{H}}_{Q,\text{AR-DNTS}}(s_t,a), \tag{A.25}$$

$$\bar{Q}_{\text{AR-DNTS}}(s_t, a_t) \leftarrow R(s_t, a_t) + \sum_{s'\in\text{Succ}(s_t,a_t)} \left(\frac{N(s')}{N(s_t,a_t)} \bar{V}_{\text{AR-DNTS}}(s')\right), \tag{A.26}$$

$$\bar{V}_{\text{AR-DNTS}}(s_t) \leftarrow \frac{N^{\hat{V}_{\text{heuristic}}}_{\text{AR-DNTS}}\hat{V}_{\text{heuristic}}(s_t)}{N(s_t) + N^{\hat{V}_{\text{heuristic}}}_{\text{AR-DNTS}}} + \sum_{a\in\mathcal{A}} \frac{N(s_t,a)}{N(s_t) + N^{\hat{V}_{\text{heuristic}}}_{\text{AR-DNTS}}} \bar{Q}_{\text{AR-DNTS}}(s_t,a). \tag{A.27}$$

Recommendation Policy.

$$\psi_{\text{AR-DNTS}}(s) = \arg\max_{a\in\mathcal{A}} \bar{Q}_{\text{AR-DNTS}}(s,a), \tag{A.28}$$

$$\texttt{mv}_{\text{AR-DNTS}}(s) = \arg\max_{a\in\mathcal{A}} N(s,a). \tag{A.29}$$

B

Additional Experimental Details for Chapter 4

This appenix gives additional details for the experiments conducted in Chapter 4. It is structured as follows:

- Section **??** gives additional details for the environemnts used in TODO: ref FL and Sailing results;
- Section **??** gives details on the hyperparameters used for the experiments in TODO: ref FL and Sailing results and the optimisation process used to select them;
- Section **??** gives additional details for the Go experiments conducted in Section TODO: ref go results, including hyperparameters and how they were selected;

B.1 Additional DeterministicGridWorld, FrozenLake details

Figure **??** shows the maps used in the DeterministicGridWorld and FrozenLake environments. The 4x4 and 8x8 maps are the standard maps from the Gymnasium library TODO: cite. The remaining maps are randomly generated, where the starting position is always the top left corner, the goal position is always the bottom

```
SFFFFFHF
FFFFFFFF
FHFHFFFF
FFFFFFHH
FFFHFFFF
FHHHFFFF
FFFFFHFF
FFFFFFFG
```

(a) 8x8 FrozenLake.

```
SFHFFFHFFFFF
FFFFFFFHFFFF
HFFFFFHFFFFF
FHFFHFFFFFFF
HHFFFFFFFFFF
FHFFFFHFFFFF
FHFFFHHFHFFF
FFFFFFFFFHHG
```

(b) 8x12 FrozenLake.

Figure B.1: Maps used for the DeterministicGridWorld and FrozenLake environments. `S` is the starting location for the agent, `F` represents floor that the agent can move too, `H` are holes that end the agents trial and `G` is the goal location. TODO: Add 4x4, 5x5 or 4x6, 6x6 or 4x8 maps for stoch envs TODO: Add 8x8, 8x12 or 8x16, 12x12 or 16x16 maps TODO: update these maps to the final maps used and update caption.

right corner and the remaining positions are randomly sampled with a $\frac{1}{5}$ probability of being a hole. Rejection sampling is used to ensure that there is always a path from the starting position to the goal.

B.2 Hyperparameter Details for DeterministicGridWorld, FrozenLake and Sailing Experiments

This section is organised as follows:

- Section **??** gives a list of shared and BD: specific to a subset of algorithms hyperparameters used for the experiments;
- Section **??** describes how the common hyperparameters are integrated into prior algorithms, using UCT as an example;
- Section **??** gives details on the hyperparameter optimisation and the final hyperparameters that were selected.

B.2.1 Common Hyperparameters Reference

The following parameters were used by all algorithms considered in section TODO: ref gridworlds results:

- `single_node_expansion` - a `THTS++` parameter that specifies whether the selection phase should end when a new decision node is added to the tree or not;
- `transposition_table` - a `THTS++` parameter that specifies whether a transposition table should be used to store and re-use the decision nodes of the search tree when they are reached again via another path;
- `normalised_Q` - if Q-values should be normalised before being used in the search policy;
- `init_v` - the heuristic value function;
- `init_q` - the heuristic Q-value function;
- $N^{\hat{V}_{\text{heuristic}}}$ - the number of pseudo-visits assigned to `init_v` in backups;
- $N^{\hat{Q}_{\text{heuristic}}}$ - the number of pseudo-visits assigned to `init_q` to the Q-value estimate;

The following parameters are used by specific algorithms:

- ϵ - the (uniform) exploration parameter used by algorithms using a Boltzmann search policy, e.g. MENTS, BTS;
- α - the temperature parameter for algorithms using a maximum entropy objective, e.g. MENTS, RENTS, TENTS;
- $\alpha(x)$ - the temperature schedule for Boltzmann distributions in BTS and DENTS;
- $\beta(x)$ - the entropy weighting function for DENTS;
- b - the bias term used in UCB style policies, e.g. UCT, H-MCTS;
- `uct_budget` - TODO: check if its seq halving into uct or uct into seq halving, the budget for running UCT before swtiching to Sequential Halving in H-MCTS;

BD: To be compatible with hyperparameter optimisation, the functional form of α and β BD: need to be specified in a parameterised form. For the experiments in Section TODO: ref gridworlds results, the following forms were used:

$$\alpha(x) = \frac{\alpha_0}{\sqrt{1 + \alpha_{\text{decay}} x}} \tag{B.1}$$

$$\beta(x) = \begin{cases} \beta_c - \frac{\beta_c}{\beta_{\text{thresh}}} x & \text{if } x \leq \beta_{\text{thresh}}, \\ 0 & \text{otherwise.} \end{cases} \tag{B.2}$$

These functions are chosen so that the entropy weighting becomes negligible, and more specifically so that $\frac{\beta(x)}{\alpha(x)}$ tends to zero as x tends to infinity.

B.2.2 Integration of Common Hyperparameters into Prior Algorithms

Consider an algorithm with value estimates $\bar{V}(s)$ and $\bar{Q}(s,a)$. To use the $N^{\hat{Q}_{\text{heuristic}}}$ and `normalised_Q` parameters, the biased Q-values $\bar{Q}^{\text{norm}}(s,a)$ used in the search policy are given by:

$$\bar{Q}^{\text{bias}}(s,a) = \frac{N(s,a)\bar{Q}(s,a) + N^{\hat{Q}_{\text{heuristic}}}\hat{Q}_{\text{heuristic}}(s,a)}{N(s,a) + N^{\hat{Q}_{\text{heuristic}}}}, \tag{B.3}$$

$$\bar{Q}^{\text{norm}}(s,a) = \begin{cases} \frac{\bar{Q}^{\text{bias}}(s,a) - \min_{a\in\mathcal{A}} \bar{Q}^{\text{bias}}(s,a)}{\max_{a\in\mathcal{A}} \bar{Q}^{\text{bias}}(s,a) - \min_{a\in\mathcal{A}} \bar{Q}^{\text{bias}}(s,a)} & \text{if } \texttt{normalised_Q}, \\ \bar{Q}^{\text{bias}}(s,a) & \text{otherwise.} \end{cases} \tag{B.4}$$

All visit counts and value estimates are initialised to zero: $N(s) = 0$, $N(s,a) = 0$, $\bar{V}(s) = 0$ and $\bar{Q}(s,a) = 0$. While MCTS algorithms are often presented as initialising new leaf nodes directly with a visit and the heuristic value TODO: ref UCT init, this biasing term can instead be encorporated into the backups instead, allowing the algorithm to be run with an without the `single_node_expansion` parameter.

Given a trajectory $\tau = (s_0, a_0, r_0, \ldots, s_{h-1}, a_{h-1}, r_{h-1}, s_h)$, the *full backups* for the average returns $\bar{V}$ and $\bar{Q}$ including the biasing term for $t = h-1, \ldots, 0$ are given by:

$$\bar{Q}(s_t, a_t) \leftarrow R(s_t, a_t) + \sum_{s' \in \text{Succ}(s_t, a_t)} \left(\frac{N(s'|s_t, a_t)}{N(s_t, a_t)} \bar{V}(s') \right), \tag{B.5}$$

$$\bar{V}(s_t) \leftarrow \frac{N^{\hat{V}_{\text{heuristic}}}\hat{V}_{\text{heuristic}}(s_t)}{N(s_t) + N^{\hat{V}_{\text{heuristic}}}} + \sum_{a\in\mathcal{A}} \frac{N(s_t, a)}{N(s_t) + N^{\hat{V}_{\text{heuristic}}}} \bar{Q}(s_t, a). \tag{B.6}$$

BD: In the main thesis the visit counts $N(s'|s,a)$ BD: are breifly mentioned in Section XXX, but not used for the general presentation to follow prior works presentation. When using `transposition_table`, all backups need to be of the *full backups*, as discussed in Section TODO: ref and argued by TODO: cite kata go.

BD: This is of particular importance for algorithms that used average returns that are typically computed using incremental backups. Note that the benefits of using `transposition_table` BD: come when the unique state assumption does not hold. The correctness of using full backups when the unique state assumption does not hold is beyond the scope of this thesis TODO: cite katago.

B.2.2.1 UCT Implementation with Full Backups

As a worked example, the version of UCT used in experiments that utilises all of the parameters from Section **??** is given. The algorithm is specified minimally using the `THTS++` schema given in Section **??**.

Value Estimates and Parameters.

- $\bar{V}_{\text{UCT}}(s)$ - The value estimate for state s;
- $\bar{Q}_{\text{UCT}}(s,a)$ - The Q-value estimate for state s and action a;
- $N(s)$ - The visit count for state s;
- $N(s,a)$ - The visit count for state s and action a;
- `normalised_Q` - A flag indicating whether the Q-values should be normalised when used in the search policy;
- $N_{\text{UCT}}^{\hat{V}_{\text{heuristic}}}$ - The number of pseudo-visits to use for biasing the backups;
- $N_{\text{UCT}}^{\hat{Q}_{\text{heuristic}}}$ - The number of pseudo-visits to use for biasing the Q-values;
- $\hat{V}_{\text{heuristic}}(s)$ - The heuristic value estimate for state s;
- $\hat{Q}_{\text{heuristic}}(s,a)$ - The heuristic Q-value estimate for state s and action a;
- b_{UCT} - the bias parameter used in the UCT search policy;

Initialisation.

$$N(s) \leftarrow 0, \tag{B.7}$$

$$N(s,a) \leftarrow 0, \tag{B.8}$$

$$\bar{V}_{\text{UCT}}(s) \leftarrow 0, \tag{B.9}$$

$$\bar{Q}_{\text{UCT}}(s,a) \leftarrow 0. \tag{B.10}$$

Search Policy. Intermediate Q-values:

$$\bar{Q}^{\text{bias}}_{\text{UCT}}(s,a) = \frac{N(s,a)\bar{Q}_{\text{UCT}}(s,a) + N^{\hat{Q}_{\text{heuristic}}}_{\text{UCT}}\hat{Q}_{\text{heuristic}}(s,a)}{N(s,a) + N^{\hat{Q}_{\text{heuristic}}}_{\text{UCT}}}, \tag{B.11}$$

$$\bar{Q}^{\text{norm}}_{\text{UCT}}(s,a) = \begin{cases} \frac{\bar{Q}^{\text{bias}}_{\text{UCT}}(s,a) - \min_{a\in\mathcal{A}} \bar{Q}^{\text{bias}}_{\text{UCT}}(s,a)}{\max_{a\in\mathcal{A}} \bar{Q}^{\text{bias}}_{\text{UCT}}(s,a) - \min_{a\in\mathcal{A}} \bar{Q}^{\text{bias}}_{\text{UCT}}(s,a)} & \text{if } \texttt{normalised_Q}, \\ \bar{Q}^{\text{bias}}_{\text{UCT}}(s,a) & \text{otherwise.} \end{cases} \tag{B.12}$$

Search policy:

$$\pi_{\text{UCT}}(s) = \arg\max_{a\in\mathcal{A}} \bar{Q}^{\text{norm}}_{\text{UCT}}(s,a) + b_{\text{UCT}}\sqrt{\frac{\log(N(s))}{N(s,a)}}, \tag{B.13}$$

Backups. Let $\tau_{:h} = (s_0, a_0, r_0, \ldots, s_{h-1}, a_{h-1}, r_{h-1}, s_h) \sim \pi_{\text{UCT}}$:

$$\bar{Q}_{\text{UCT}}(s_t,a_t) \leftarrow R(s_t,a_t) + \sum_{s'\in\text{Succ}(s_t,a_t)} \left(\frac{N(s'|s_t,a_t)}{N(s_t,a_t)} \bar{V}_{\text{UCT}}(s') \right), \tag{B.14}$$

$$\bar{V}_{\text{UCT}}(s_t) \leftarrow \frac{N^{\hat{V}_{\text{heuristic}}}\hat{V}_{\text{heuristic}}(s_t)}{N(s_t) + N^{\hat{V}_{\text{heuristic}}}} + \sum_{a\in\mathcal{A}} \frac{N(s_t,a)}{N(s_t) + N^{\hat{V}_{\text{heuristic}}}} \bar{Q}_{\text{UCT}}(s_t,a). \tag{B.15}$$

Recommendation Policy.

$$\psi_{\text{UCT}}(s) = \arg\max_{a\in\mathcal{A}} \bar{Q}_{\text{UCT}}(s,a), \tag{B.16}$$

$$\texttt{mv}_{\text{UCT}}(s) = \arg\max_{a\in\mathcal{A}} N(s,a). \tag{B.17}$$

B.2.3 Hyperparameter Optimisation

The parameters shared between all algorithms were set manually to the following values for all environments:

- `single_node_expansion` = False - the selection phase continues until the planning horizon or a terminal state is reached;

Parameter	Minimum	Maximum	Scaling
ϵ	10^{-6}	1	Logarithmic
α	10^{-5}	10^2	Logarithmic
α_0	10^{-5}	10^2	Logarithmic
α_{decay}	10^{-6}	10^2	Logarithmic
β_c	10^{-5}	10^2	Logarithmic
β_{thresh}	10^3	10^12	Logarithmic
b	10^{-5}	10^2	Logarithmic
`uct_budget`	1	10^5	Linear

Table B.1: The hyperparameter search space for gridworld experiments (Section TODO: ref). Recall that α is the temperature used for maximum entropy algorithms (e.g. MENTS), and α_0, α_{decay} β_c and β_{thresh} are parameters for $\alpha(x)$ and $\beta(x)$ used in BTS and DENTS. The search space for α_{decay} and β_{thresh} are selected such that the function spaces for $\alpha(x)$ and $\beta(x)$ include functions that are nearly constant for the duration of the search.

- `transposition_table` = True - transposition tables are used to store and re-use decision nodes when they are reached again via another path;
- `normalised_Q` = True - Q-values are linearly normalised before being used in the search policy;
- $N^{\hat{V}_{\text{heuristic}}} = 0$ - the number of pseudo-visits assigned to `init_v` in backups;
- $N^{\hat{Q}_{\text{heuristic}}} = 1$ - the number of pseudo-visits assigned to `init_q` to the Q-value estimate.

Using the principle of optimism in the face of uncertainty, the heuristic value functions were set to a trivial upper bound on the values that could be achieved in each environment. For the DeterministicGridWorld and FrozenLake environments, this meant the value functions were set to $\hat{V}_{\text{heuristic}}(s) = \hat{Q}_{\text{heuristic}}(s, a) = 1.0$ for all states s and actions a. For the Sailing environments, because all rewards are negative, the value functions were set to $\hat{V}_{\text{heuristic}}(s) = \hat{Q}_{\text{heuristic}}(s, a) = 0.0$ for all states s and actions a.

To select the remaining hyperparameters for the experiments detailed in Section TODO: ref the BayesOpt library TODO: cite was used. Table **??** gives the space searched over for each parameter, and if BD: parameters were sampled using a linear or logarithmic scalarised.

Algorithm	ϵ	α	10^{-5}
10^2	Logarithmic		
α_0	10^{-5}	10^2	Logarithmic
α_{decay}	10^{-6}	10^2	Logarithmic
β_c	10^{-5}	10^2	Logarithmic
β_{thresh}	10^3	10^{12}	Logarithmic
b	10^{-5}	10^2	Logarithmic
`uct_budget`	1	10^5	Linear height

Table B.2: The hyperparameter search space for gridworld experiments (Section TODO: ref). Recall that α is the temperature used for maximum entropy algorithms (e.g. MENTS), and α_0, α_{decay} β_c and β_{thresh} are parameters for $\alpha(x)$ and $\beta(x)$ used in BTS and DENTS. The search space for α_{decay} and β_{thresh} are selected such that the function spaces for $\alpha(x)$ and $\beta(x)$ include functions that are nearly constant for the duration of the search.

Parameters were optimised using the TODO: ref map for DeterministicGridWorld, the TODO: ref map for FrozenLake and the TODO: ref map for Sailing.

To evaluate the performance of a given set of parameters, the algorithm was run for 50,000 trials, and 1024 trials used to evaluate the recommendation policy. At least 10 runs were performed for each set of parameters, and runs were repeated until the standard deviation across the runs was less than 0.025 for DeterministicGridWorld and FrozenLake, and less than 1.0 for Sailing.

The selected hyperparameters for DeterministicGridWorld, FrozenLake, Sailing(North) and Sailing(South-East) are given in Tables **??**, **??**, **??** and **??** respectively.

TODO TODO TODO TODO TODO brief description of hp opt table giving ranges for parameters optimised over give table for gridworld, frozen lake and sailing for the optimised parameters used in experiments

TODO TODO TODO edit go stuff

UCT is a good example of how the common hyperparameters are integrated into existing algorithms. UCT is a tree search algorithm that uses a Boltzmann distribution to select actions at each decision node. The search policy that UCT follows is given by:

TODO: This is copy and pasted from neurips at the moment. It doesn't account for the changes made to the algorithms or now using BayesOpt. Need to describe the

parameter space searched over for each algorithm, how many Bayes trials were used and how each bayes tirla evaluated. And also give the final values used in tables. Manually setting Qinit function, but no prior knowledge used other than this.

To select hyper-parameters for the experiments detailed in Section **??**, we performed a hyper-parameter search. The search was run on the 8x12 Frozen Lake environment from Figure **??**, and the results in Section **??** were run on the 8x12 Frozen Lake envrionment from Figure **??**. For the Sailing problem, we performed the search using an initial wind direction of North, and the result in Section **??** used an initial wind direction of South-East.

To avoid the search space from becoming too large, we set some parameters manually. A good rule of thumb for initial values is to assure that $Q^{\text{init}}_{\text{sft}}(s,a) < Q^{*}_{\text{sft}}(s,a)$ and $Q^{\text{init}}(s,a) < Q^{*}(s,a)$. Explicitly this means that an initial value of zero is *not* a good choice for the Sailing problem, as rewards are negative (i.e. it has costs). In the Sailing environment, we actually set the initial values to -200, so that they were equal to the lowest possible return from a trial (the trial length was set to 50, and an agent can incur a cost of at most -4 per timestep). To simplify the search space, we initially set the decay function in DENTS to $\beta(m) = \alpha/\log(e+m)$ and tune it after.

For the remaining parameters, we considered all combinations of the following values:

- *UCT Bias*: TODO: cite, 100.0, 10.0, 1.0, 0.1;
- *MENTS exploration coefficient*: 2.0, 1.0, 0.3, 0.1, 0.03, 0.01;
- *Temperature*: 100.0, 10.0, 1.0, 0.1, 0.01, 0.001;
- *HMCTS UCT budget*: 100000, 30000, 10000, 3000, 1000, 300, 100, 30, 10.

Where a UCT bias of TODO: citerefers to the adaptive bias introduced by Keller and Eyerich TODO: cite, and 'temperature' refers to the relevant temperature for the algorithm (i.e. search temperature in BTS and DENTS, and the temperature for Shannon/Relative/Tsallis entropy in MENTS/RENTS/DENTS). After that search,

Algorithm	Exploration Parameter (ϵ)	Temperature (α)	Initial Values (Q^{init}, $Q^{\text{init}}_{\text{sft}}$)
MENTS	1.0	0.001	0
RENTS	2.0	0.001	0
TENTS	1.0	0.001	0
BTS	2.0	0.1	0
DENTS	1.0	0.1	0

Table B.3: Final hyperparameters used for Frozen Lake in Section **??**. Not included in the table: TODO: citewas selected for the bias in UCT, TODO: citewas selected for the bias and 3000 for the UCT budget in HMCTS and $\beta_{\text{init}} = 1.0$ was selected as the initial entropy temperature for DENTS.

Algorithm	Exploration Parameter (ϵ)	Temperature (α)	Initial Values (Q^{init}, $Q^{\text{init}}_{\text{sft}}$)
MENTS	1.0	10.0	-200
RENTS	1.0	10.0	-200
TENTS	2.0	0.1	-200
BTS	1.0	10.0	-200
DENTS	1.0	10.0	-200

Table B.4: Final hyperparameters used for the Sailing Problem in Section **??**. Not included in the table: TODO: citewas selected for the bias in UCT, TODO: citewas selected for the bias and 30 for the UCT budget in HMCTS and $\beta_{\text{init}} = 10.0$ was selected as the initial entropy temperature for DENTS.

for DENTS we considered the decay functions of the form $\beta(m) = \beta_{\text{init}}/\log(e + m)$ and considered the following values:

- *DENTS initial entropy temperature* (β_{init}): 100.0, 10.0, 1.0, 0.1.

The final set of hyperparameters is given in Tables **??** and **??**, which were used in the gridworld experiments in Section **??**. Not included in the tables: TODO: citewas selected for the UCT bias in both Frozen Lake and the Sailing problem.

B.3 Additional Go details, results and discussion

TODO: Everything in this section this in this appendix is C and P from NeruIPS Go parameter tuning section.

Recall from Appendix **??** that we initialised values with the neural networks as $Q^{\text{init}}(s, a) = \log \tilde{\pi}(a|s) + B$ and $V^{\text{init}}(s) = \tilde{V}(s)$, where B is a constant (adapted from Xiao TODO: fixcites) For these experiments we set a value of $B = \frac{-1}{|\mathcal{A}|} \sum_{a \in \mathcal{A}} \log \tilde{\pi}(a|s)$. Initialising the values in such a way tends to lead to the values of $Q^{\text{init}}(s, a)$ being

in the range $[-20, 20]$, to account for this, we scaled the results of the game to 100 and -100, which means that any parameters selected are about 100 times larger than they would be if we had not have used this scaling.

In these experiments we used a recommendation policy that recommends the action that was sampled the most, i.e. $\psi(s) = \max_a N(s, a)$, as this tends to be more robust to any noise from neural network outputs.

To select each parameter we ran a round robin tournament where we varied the value of one parameter. The agent that won the most games was used to select the parameter moving forward, and in the case of a tie we used the agent which won the most games. If the winning agent had the largest or smallest value, then we ran another tournament adjusting the values accordingly.

We tuned all of these algorithms using the game of 9x9 Go with a komi of 6.5, and giving each algorithm 2.5 seconds per move. For our final results we used the same parameters on 19x19 Go with a komi of 7.5, giving each algorithm 5.0 seconds per move.

B.3.1 Parameter selection for Go and supplimentary results

In this section we work through the process used to select parameters for the Go round robin tournament used in Section **??**. Predominantly parameters were chosen by playing out games of Go between agents. The parameters used for PUCT were copied from Kata Go and Alpha Go Zero TODO: fix cites

Initially we set the values of $\epsilon, \epsilon_{\tilde{\lambda}}$ to $0.03, 1.0$. In Tables **??** and **??** we give results of tuning the search temperature for BTS and AR-BTS. For AR-BTS we tried different values of α_{init} in $\alpha(m) = \alpha_{\text{init}}/\sqrt{m}$.

We then tuned the weighting of the prior policy $\epsilon_{\tilde{\lambda}}$. In Tables **??** and **??** we give results of tuning the prior policy weight for BTS and AR-BTS.

Following that, we then tuned MENTS exploration parameter ϵ. In Tables **??** and **??** we give results of tuning the exploration parameter for BTS and AR-BTS. Although we selected the lowest value we tried here, we note that with such a value

Black \White	3	1	0.3	0.1	0.03
3	-	7-8	9-6	12-3	12-3
1	15-0	-	12-3	15-0	15-0
0.3	13-2	11-4	-	14-1	15-0
0.1	14-1	10-5	11-4	-	15-0
0.03	13-2	5-10	8-7	13-2	-

Table B.5: Results for round robin to select the temperature parameter α for BTS. The value of 1.0 won all four of its matches so was selected.

Black \White	3	1	0.3	0.1	0.03
3	-	3-12	4-11	3-12	1-14
1	14-1	-	8-7	9-6	7-8
0.3	14-1	8-7	-	7-8	11-4
0.1	15-0	11-4	9-6	-	9-6
0.03	14-1	8-7	4-11	8-7	-

Table B.6: Results for round robin to select the temperature parameter α_{init} for AR-BTS. The value of 0.1 won all four of its matches so was selected.

Black \White	10	5	3	2	1
10	-	5-10	3-12	1-14	3-12
5	14-1	-	12-3	11-4	11-4
3	15-0	15-0	-	12-3	12-3
2	15-0	12-3	12-3	-	10-5
1	15-0	14-1	12-3	9-6	-

Table B.7: Results for round robin to select the weighting of the prior policy $\epsilon_{\tilde{\lambda}}$ for BTS. The value of 2.0 won the most matches so was selected.

Black \White	3	2	1	0.75	0.5
3	-	14-1	9-6	13-2	14-1
2	12-3	-	12-3	15-0	10-5
1	12-3	13-2	-	11-4	13-2
0.75	11-4	10-5	12-3	-	14-1
0.5	11-4	13-2	9-6	7-8	-

Table B.8: Results for round robin to select the weighting of the prior policy $\epsilon_{\tilde{\lambda}}$ for AR-BTS. The value of 1.0 won the most matches so was selected.

Black \White	0.1	0.03	0.01	0.003	0.001
0.1	-	12-3	10-5	8-7	7-8
0.03	9-6	-	8-7	13-2	12-3
0.01	11-4	9-6	-	7-8	12-3
0.003	13-2	9-6	11-4	-	11-4
0.001	12-3	11-4	8-7	9-6	-

Table B.9: Results for round robin to select the exploration parameter ϵ for BTS. The value of 0.003 won the most matches so was selected.

Black \White	0.1	0.03	0.01	0.003	0.001
0.1	-	12-3	14-1	12-3	13-2
0.03	15-0	-	11-4	14-1	14-1
0.01	15-0	11-4	-	13-2	13-2
0.003	15-0	14-0	14-1	-	13-2
0.001	14-1	14-1	14-1	15-0	-

Table B.10: Results for round robin to select the exploration parameter ϵ for AR-BTS. The value of 0.001 won the most matches so was selected.

that a random action would have been sampled very few times, so the result of the hyperparameter selection was essentially that ϵ should be as low as possible.

Then we tuned the (fixed and constant) search temperatures for MENTS, AR-MENTS, RENTS, AR-RENTS, TENTS and AR-TENTS in tables **??**, **??**,**??**, **??**,**??** and **??**.

Finally, we considered entropy temperatures of the form $\beta(m) = \beta_{\text{init}} \frac{1+\exp(-5)}{1+\exp(m-5)}$ for DENTS and AR-DENTS, and tuned the value of β_{init}, in Tables **??** and **??** for DENTS and AR-DENTS respectively.

After tuning all of the algorithms, we compared each algorithm to their AR version in table **??**, and the AR versions universally outperformed their counterparts. We used all of the selected parameters in 19x19 Go to run our final experiments

Black \White	3	1	0.3	0.1	0.03
3	-	9-6	9-6	11-4	10-5
1	11-4	-	11-4	9-6	10-5
0.3	9-6	7-8	-	11-4	6-9
0.1	4-11	4-11	0-15	-	4-11
0.03	2-13	2-13	0-15	0-15	-

Table B.11: Results for round robin to select the temperature parameter α for MENTS. The value of 1.0 won the most matches so was selected.

Black \White	3	1	0.3	0.1	0.03
3	-	2-13	1-14	3-12	5-10
1	14-1	-	11-4	13-2	15-0
0.3	15-0	12-3	-	12-3	14-1
0.1	15-0	9-6	9-6	-	15-0
0.03	15-0	8-7	9-6	14-1	-

Table B.12: Results for round robin to select the temperature parameter α for AR-MENTS. The value of 0.3 won the most matches so was selected.

Black \White	3	1	0.3	0.1	0.03
3	-	7-8	9-6	10-5	9-6
1	13-2	-	12-3	11-4	12-3
0.3	10-5	11-4	-	10-5	5-10
0.1	3-12	2-13	1-14	-	3-12
0.03	3-12	0-15	0-15	0-15	-

Table B.13: Results for round robin to select the temperature parameter α for RENTS. The value of 1.0 won the most matches so was selected.

Black \White	3	1	0.3	0.1	0.03
3	-	4-11	2-13	9-6	3-12
1	15-0	-	12-3	13-2	13-2
0.3	15-0	13-2	-	14-1	15-0
0.1	15-0	10-5	13-2	-	15-0
0.03	13-2	13-2	12-3	14-1	-

Table B.14: Results for round robin to select the temperature parameter α for AR-RENTS. The value of 0.3 won the most matches so was selected.

Black \White	300	100	30	10	3
300	-	3-12	5-10	7-8	9-6
100	6-9	-	9-6	12-3	12-3
30	8-7	7-8	-	9-6	11-4
10	0-15	2-13	2-13	-	6-9
3	1-14	1-14	2-13	5-10	-

Table B.15: Results for round robin to select the temperature parameter α for TENTS. The value of 30.0 won the most matches so was selected.

Black \White	30	10	3	1	0.3
30	-	14-1	15-0	14-1	14-1
10	15-0	-	13-2	15-0	15-0
3	15-0	14-1	-	14-1	15-0
1	15-0	15-0	14-1	-	15-0
0.3	15-0	15-0	14-1	15-0	-

Table B.16: Results for round robin to select the temperature parameter α for AR-TENTS. The value of 3.0 won the most matches so was selected.

Black \White	0.1	0.03	0.01	0.003	0.001
0.1	-	10-5	12-3	8-7	11-4
0.03	13-2	-	11-4	13-2	10-5
0.01	12-3	11-4	-	10-5	11-4
0.003	7-8	13-2	13-2	-	10-5
0.001	13-2	13-2	11-4	11-4	-

Table B.17: Results for round robin to select the initial entropy temperature β_{init} for DENTS. The value of 0.3 won the most matches so was selected.

given in Table **??**.

Finally, we also ran AR-BTS against Kata Go directly limiting each algorithm to 1600 trials. KataGo beat AR-BTS by 31-19, confirming that the aditional exploration is outweighed by the information contained in the neural networks, and in Go the Boltzmann search algorithms gain their advantage via the Alias method and being able to run more trials quickly.

Black \White	3	1	0.3	0.1	0.03
3	-	11-4	12-3	12-3	14-1
1	13-2	-	11-4	13-2	13-2
0.3	14-1	13-2	-	13-2	14-1
0.1	12-3	12-3	12-3	-	11-4
0.03	12-3	13-2	13-2	14-1	-

Table B.18: Results for round robin to select the initial entropy temperature β_{init} for AR-DENTS. The value of 0.3 won the most matches so was selected.

Black \White	MENTS	AR-MENTS	BTS	AR-BTS	DENTS	AR-DENTS
MENTS	-	0-25				
AR-MENTS	25-0	-				
BTS			-	16-9		
AR-BTS			22-3	-		
DENTS					-	21-4
AR-DENTS					22-3	-
Black \White	RENTS	AR-RENTS	TENTS	AR-TENTS		
RENTS	-	2-23				
AR-RENTS	24-1	-				
TENTS			-	8-17		
AR-TENTS			23-2	-		

Table B.19: Results for the matches of each algorithm against its AR version.

Bibliography

[] Bruce Abramson. Expected-outcome: A general model of static evaluation. *IEEE transactions on pattern analysis and machine intelligence*, 12(2):182–193, 1990.

[] P Auer. Finite-time analysis of the multiarmed bandit problem, 2002.

[] David Auger, Adrien Couetoux, and Olivier Teytaud. Continuous upper confidence trees with polynomial exploration–consistency. In *Machine Learning and Knowledge Discovery in Databases: European Conference, ECML PKDD 2013, Prague, Czech Republic, September 23-27, 2013, Proceedings, Part I 13*, pages 194–209. Springer, 2013.

[] Leon Barrett and Srini Narayanan. Learning all optimal policies with multiple criteria. In *Proceedings of the 25th international conference on Machine learning*, pages 41–47, 2008.

[] Andrew G Barto, Steven J Bradtke, and Satinder P Singh. Learning to act using real-time dynamic programming. *Artificial intelligence*, 72(1-2):81–138, 1995.

[] Richard Bellman. *Dynamic Programming*. Dover Publications, 1957. ISBN 9780486428093.

[] Vincent Berthier, Hassen Doghmen, and Olivier Teytaud. Consistency modifications for automatically tuned monte-carlo tree search. In *International Conference on Learning and Intelligent Optimization*, pages 111–124. Springer, 2010.

[] Lilian Besson and Emilie Kaufmann. What doubling tricks can and can't do for multi-armed bandits. *arXiv preprint arXiv:1803.06971*, 2018.

[] Patrick Billingsley. *Probability and measure*. John Wiley & Sons, 2017.

[] Cameron B Browne, Edward Powley, Daniel Whitehouse, Simon M Lucas, Peter I Cowling, Philipp Rohlfshagen, Stephen Tavener, Diego Perez, Spyridon Samothrakis, and Simon Colton. A survey of monte carlo tree search methods. *IEEE Transactions on Computational Intelligence and AI in games*, 4(1):1–43, 2012.

[] Sébastien Bubeck, Rémi Munos, and Gilles Stoltz. Pure exploration in multi-armed bandits problems. In *Algorithmic Learning Theory: 20th International Conference, ALT 2009, Porto, Portugal, October 3-5, 2009. Proceedings 20*, pages 23–37. Springer, 2009.

[] Tristan Cazenave, Flavien Balbo, Suzanne Pinson, et al. Monte-carlo bus regulation. In *12th international IEEE conference on intelligent transportation systems*, volume 340345, 2009.

[] Benjamin E Childs, James H Brodeur, and Levente Kocsis. Transpositions and move groups in monte carlo tree search. In *2008 IEEE Symposium On Computational Intelligence and Games*, pages 389–395. IEEE, 2008.

[] Pierre-Arnaud Coquelin and Rémi Munos. Bandit algorithms for tree search. *arXiv preprint cs/0703062*, 2007.

[] Adrien Couëtoux, Jean-Baptiste Hoock, Nataliya Sokolovska, Olivier Teytaud, and Nicolas Bonnard. Continuous upper confidence trees. In *International conference on learning and intelligent optimization*, pages 433–445. Springer, 2011.

[] Tuan Q Dam, Carlo D'Eramo, Jan Peters, and Joni Pajarinen. Convex regularization in monte-carlo tree search. In *International Conference on Machine Learning*, pages 2365–2375. PMLR, 2021.

[] Aryeh Dvoretzky, Jack Kiefer, and Jacob Wolfowitz. Asymptotic minimax character of the sample distribution function and of the classical multinomial estimator. *The Annals of Mathematical Statistics*, pages 642–669, 1956.

[] Zohar Feldman and Carmel Domshlak. Monte-carlo planning: Theoretically fast convergence meets practical efficiency. *arXiv preprint arXiv:1309.6828*, 2013.

[] William Feller et al. *An introduction to probability theory and its applications*. Wiley New York, 1971.

[] Sylvain Gelly and David Silver. Combining online and offline knowledge in uct. In *Proceedings of the 24th international conference on Machine learning*, pages 273–280, 2007.

[] Sylvain Gelly and David Silver. Monte-carlo tree search and rapid action value estimation in computer go. *Artificial Intelligence*, 175(11):1856–1875, 2011.

[] Tuomas Haarnoja, Haoran Tang, Pieter Abbeel, and Sergey Levine. Reinforcement learning with deep energy-based policies. In *International conference on machine learning*, pages 1352–1361. PMLR, 2017.

[] Eric A Hansen and Shlomo Zilberstein. Lao*: A heuristic search algorithm that finds solutions with loops. *Artificial Intelligence*, 129(1-2):35–62, 2001.

[] Conor F Hayes, Roxana Rădulescu, Eugenio Bargiacchi, Johan Källström, Matthew Macfarlane, Mathieu Reymond, Timothy Verstraeten, Luisa M Zintgraf, Richard Dazeley, Fredrik Heintz, et al. A practical guide to multi-objective reinforcement learning and planning. *Autonomous Agents and Multi-Agent Systems*, 36(1):26, 2022.

[] Edwin T Jaynes. Information theory and statistical mechanics. *Physical review*, 106(4):620, 1957.

[] Thomas Keller and Patrick Eyerich. Prost: Probabilistic planning based on uct. In *Proceedings of the International Conference on Automated Planning and Scheduling*, volume 22, pages 119–127, 2012.

[] Thomas Keller and Malte Helmert. Trial-based heuristic tree search for finite horizon mdps. In *Proceedings of the International Conference on Automated Planning and Scheduling*, volume 23, pages 135–143, 2013.

[] Robert Kleinberg, Aleksandrs Slivkins, and Eli Upfal. Multi-armed bandits in metric spaces. In *Proceedings of the fortieth annual ACM symposium on Theory of computing*, pages 681–690, 2008.

[] Levente Kocsis and Csaba Szepesvári. Bandit based monte-carlo planning. In *European conference on machine learning*, pages 282–293. Springer, 2006.

[] Levente Kocsis, Csaba Szepesvári, and Jan Willemson. Improved monte-carlo search. *Univ. Tartu, Estonia, Tech. Rep*, 1:1–22, 2006.

[] Tze Leung Lai and Herbert Robbins. Asymptotically efficient adaptive allocation rules. *Advances in applied mathematics*, 6(1):4–22, 1985.

[] Tor Lattimore and Csaba Szepesvári. *Bandit algorithms*. Cambridge University Press, 2020.

[] Tyler Lu, Dávid Pál, and Martin Pál. Contextual multi-armed bandits. In *Proceedings of the Thirteenth international conference on Artificial Intelligence and Statistics*, pages 485–492. JMLR Workshop and Conference Proceedings, 2010.

[] Ruben Martinez-Cantin. Bayesopt: A bayesian optimization library for nonlinear optimization, experimental design and bandits. Technical report, 2014.

[] Volodymyr Mnih, Adria Puigdomenech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous methods for deep reinforcement learning. In *International conference on machine learning*, pages 1928–1937. PmLR, 2016.

[] Thomas M Moerland, Joost Broekens, Aske Plaat, and Catholijn M Jonker. A0c: Alpha zero in continuous action space. *arXiv preprint arXiv:1805.09613*, 2018.

[] Mausam Natarajan and Andrey Kolobov. *Planning with Markov decision processes: An AI perspective*. Springer Nature, 2022.

[] Michael Painter. THTS++, 2025. URL `https://github.com/MWPainter/thts-plus-plus`. Available at `https://github.com/MWPainter/thts-plus-plus`.

[] Laurent Péret and Frédérick Garcia. On-line search for solving markov decision processes via heuristic sampling. *learning*, 16:2, 2004.

[] Herbert Robbins. Some aspects of the sequential design of experiments. 1952.

[] Diederik Marijn Roijers, Shimon Whiteson, and Frans A Oliehoek. Computing convex coverage sets for faster multi-objective coordination. *Journal of Artificial Intelligence Research*, 52:399–443, 2015.

[] Christopher D Rosin. Multi-armed bandits with episode context. *Annals of Mathematics and Artificial Intelligence*, 61(3):203–230, 2011.

[] Christopher D Rosin. Multi-armed bandits with episode context. *Annals of Mathematics and Artificial Intelligence*, 61(3):203–230, 2011.

[] Abdallah Saffidine, Tristan Cazenave, and Jean Méhat. Ucd: Upper confidence bound for rooted directed acyclic graphs. *Knowledge-Based Systems*, 34:26–33, 2012.

[] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[] Claude Elwood Shannon. A mathematical theory of communication. *The Bell system technical journal*, 27(3):379–423, 1948.

[] David Silver and Joel Veness. Monte-carlo planning in large pomdps. *Advances in neural information processing systems*, 23, 2010.

[] David Silver, Aja Huang, Chris J Maddison, Arthur Guez, Laurent Sifre, George Van Den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, et al. Mastering the game of go with deep neural networks and tree search. *nature*, 529(7587):484–489, 2016.

[] David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, et al. Mastering the game of go without human knowledge. *nature*, 550(7676):354–359, 2017.

[] David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharshan Kumaran, Thore Graepel, et al. A general reinforcement learning algorithm that masters chess, shogi, and go through self-play. *Science*, 362(6419):1140–1144, 2018.

[] Aleksandrs Slivkins. Contextual bandits with similarity information. In *Proceedings of the 24th annual Conference On Learning Theory*, pages 679–702. JMLR Workshop and Conference Proceedings, 2011.

[] Dennis JNJ Soemers, Chiara F Sironi, Torsten Schuster, and Mark HM Winands. Enhancements for real-time monte-carlo tree search in general video game playing. In *2016 IEEE Conference on Computational Intelligence and Games (CIG)*, pages 1–8. IEEE, 2016.

[] Richard S Sutton. Reinforcement learning: An introduction. *A Bradford Book*, 2018.

[] David Tolpin and Solomon Shimony. Mcts based on simple regret. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 26, pages 570–576, 2012.

[] Marko Tot, Michelangelo Conserva, Diego Perez Liebana, and Sam Devlin. Turning zeroes into non-zeroes: Sample efficient exploration with monte carlo graph search. In *2022 IEEE Conference on Games (CoG)*, pages 300–306. IEEE, 2022.

[] Michael D Vose. A linear algorithm for generating random numbers with a given distribution. *IEEE Transactions on software engineering*, 17(9):972–975, 1991.

[] Alastair J Walker. An efficient method for generating discrete random variables with general distributions. *ACM Transactions on Mathematical Software (TOMS)*, 3(3):253–256, 1977.

[] Ronald J Williams and Jing Peng. Function optimization using connectionist reinforcement learning algorithms. *Connection Science*, 3(3):241–268, 1991.

[] David J Wu. Accelerating self-play learning in go. *arXiv preprint arXiv:1902.10565*, 2019.

[] David J. Wu. Monte-carlo graph search from first principles. `https://github.com/lightvector/KataGo/blob/master/docs/GraphSearch.md`, 2023. Accessed: 2026-03-19.

[] Chenjun Xiao, Ruitong Huang, Jincheng Mei, Dale Schuurmans, and Martin Müller. Maximum entropy monte-carlo planning. *Advances in Neural Information Processing Systems*, 32, 2019.

[] Brian D Ziebart. *Modeling purposeful adaptive behavior with the principle of maximum causal entropy*. Carnegie Mellon University, 2010.

[] Brian D Ziebart, Andrew L Maas, J Andrew Bagnell, Anind K Dey, et al. Maximum entropy inverse reinforcement learning. In *Aaai*, volume 8, pages 1433–1438. Chicago, IL, USA, 2008.